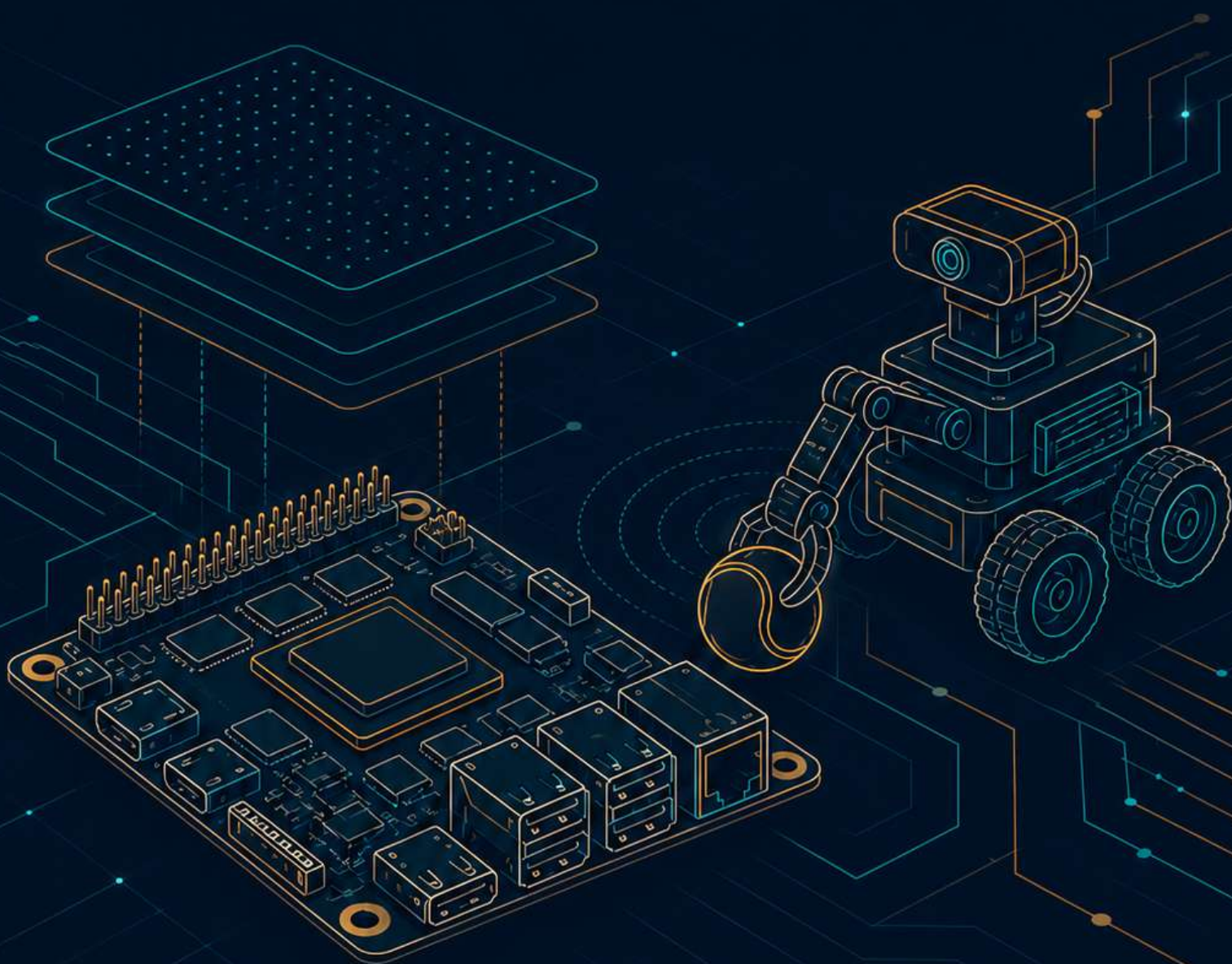


功能挑战 · proj4

面向边缘智能的 AIOS 设计与优化

决赛设计方案

Posad · 清华大学



摘要

本工作的起点，是将一台网球拾取机器人的操作系统由基于 C 语言的 Linux 迁移至采用 Rust 实现的 StarryOS，运行平台为 RK3588 (OrangePi-5-Plus)。工作设定两个目标：其一，使机器人在 StarryOS 上恢复运行；其二，使整套 StarryOS 在与机器人无关的通用负载上同样追平 Linux。两侧采用同一份遵循 Linux ABI 的可执行文件进行对照测试，8 线程 sysbench cpu 吞吐达到 Linux 的 0.96 倍，单核 A55 达到 1.03 倍并小幅领先，THP first-touch 建立映射较 Linux 快 2.87 倍，机器人自上电至首帧推理的 TTFI 为 9.83s，优于 Linux 的 11.28s。上述结果均源自内核侧对调度、DVFS、分页与启动路径的改造。基线为 rcore-os/tgoskits 的 dev 分支 (commit 73409e079)，全部工作由 Team Posad (清华大学) 完成。

目录

第一部分 背景与贡献	5
1 引言与目标	5
2 系统总览与测量方法	5
3 基础版本与增量贡献	7
第二部分 应用层：视觉感知与推理链路	9
4 视觉感知流水线	9
4.1 流水线各阶段	9
4.2 在真机上的结果	11
5 推理链路优化	12
5.1 推理链路各阶段	12
5.2 两种输入尺寸的口径	13
5.3 基线剖析与瓶颈归因	14
5.4 将最重的推理绑定到 A76 大核	14
5.5 以整数路径取回输出	15
5.6 RGA 硬件解码并零拷贝直写 NPU 输入	15
5.7 原生分辨率模型与 ReLU 激活	16
5.8 JPU 硬件 MJPEG 解码	16
5.9 已否定与在设计中的方案	16
5.10 结果	16
5.11 剖析方法	17
6 机器人拾球流程与本体侧优化	17
6.1 拾球状态机	18
6.2 可换真实执行器后端	19
6.3 里程计引导回桶	20
6.4 工程加固与延迟工程	21
第三部分 系统层：内核优化与系统能力	23
7 性能观测 perf	23
7.1 硬件基础：ARM PMUv3 与硬件计数	23
7.2 perf_event_open ABI 与直接运行上游 perf	24
7.3 多核与大小核	24
7.4 事件类型	25
7.5 采样与归属	26
7.6 调用栈采样	26
7.7 动态跟踪：kprobe 与 tracepoint	26
7.8 ftrace 函数跟踪	27
7.9 板级验证	27
7.10 硬件受限特性	27
7.11 板级运行中发现的 axtask 跨核互唤醒死锁	28
8 调度与大小核	28
8.1 调度器与运行队列	28
8.2 大小核与既有 axtask 的放置	29
8.3 跨核派生与唤醒分发 (#1656)	30

8.4	占用感知的放置	30
8.5	完整工作窃取均衡器的反面结论	31
8.6	wake_affine 就近唤醒	31
8.7	并发工作线程的唤醒铺开	31
8.8	-T 线程负载的用户拷贝快路径	32
8.9	对等阶梯与核驻留	32
8.10	单核算力与残余差距	34
9	系统调用入口开销	34
9.1	陷入路径的构成	34
9.2	每次调用固定支付的三处成本	35
9.3	三处优化	35
9.4	tickacct 精确记账特性	36
9.5	结果	36
10	内存与分页	37
10.1	分页、首次触碰、多级页表与 TLB	38
10.2	首次触碰路径的既有每页开销	39
10.3	三项首次触碰优化	39
10.4	页表与 TLB 的一类正确性修复	40
11	频率与电压 DVFS/PVTPLL	42
11.1	电压耦合的 PVTPLL 与 BL31 保护的时钟	42
11.2	既有的 SCMI 时钟通路	43
11.3	ondemand 按簇调频 (#1657)	44
11.4	PMIC 电压标定	44
11.5	PVTPLL 环长切换	44
11.6	逐核对齐与 governor 上限	45
11.7	内存带宽的 DDR 变频固件外因	46
12	启动与首帧推理 TTFI	47
12.1	启动链的组成	47
12.2	基线的构成：19.22 s 的分解	48
12.3	相机初始化在流负载下的 32 s 停滞	48
12.4	设备探测段的两处内核停顿消除	49
12.5	启动到 shell 的三处架构改造	49
12.6	应用冷启动的两项通用改动	49
12.7	init 自动执行与真正的对等差量	50
12.8	中性与否定结果	50
12.9	QEMU 验证	51
12.10	结果与诚实边界	51
13	原生显示栈 VOP2 / HDMI-QP / HDPTX	51
13.1	显示通路：四段链路及其机制	52
13.2	像素时钟归属	53
13.3	既有与新增	53
13.4	193 条 PHY 写序及其判据	54
13.5	两条启用路径与安全取舍	55
13.6	主机与板级验证	55

14 图形合成器栈：Weston 于 StarryOS	56
14.1 合成器压在内核上的四个子系统	57
14.2 逐层的内核启用与修复	58
14.3 验证	58
15 QEMU RKNPU 功能级仿真与模型结构协同优化	59
15.1 RKNPU 与功能级仿真的必要性	59
15.2 设备接入与命令流水线	60
15.3 IOMMU 接入	60
15.4 算子的功能执行	61
15.5 QTest 回归	61
15.6 多核协同	61
15.7 基于模拟器的模型结构协同优化	61
16 板级 I/O 与外设驱动	62
16.1 子系统构成与既有骨架	63
16.2 控制台与串口：UART 与 USB 串口 tty	63
16.3 执行器控制：PWM 的 sysfs 接口	64
16.4 双系统切换：reboot 系统调用	64
16.5 视觉取流前端：UVC 相机异步传输	64
16.6 从网卡上电到板上 SSH	65
16.7 使能效果与刷写通路	65
第四部分 方法、分工与展望	66
17 评估后未采用的尝试与适用条件	66
18 AI 辅助开发方法与正确性保障	68
18.1 方法的架构	68
18.2 四类判定门	68
18.3 真机闭环与带外电源控制	69
18.4 测量驱动的对抗式验证否定了若干设想	70
18.5 团队掌控架构与验收	71
19 队员分工	71
20 开发过程与时间线	72
21 可复现性与后续工作	74
21.1 硬件接线与板上运行	74
21.2 后续方向	75

第一部分 背景与贡献

1 引言与目标

在边缘设备上运行推理已成常态，模型、调度、驱动、供电各层集中于同一块 SoC（片上系统）之上，操作系统既要为加速器持续供给数据，又要在多核之间分配处理器时间。将这样一台系统从以 C 语言实现、运行于 Linux 的形态迁移到以 Rust 实现的内核，收益在于内存安全与模块化，前提是通用负载的性能不因迁移而下降。所迁移的负载是一台网球拾取机器人，程序移植自 pengzechen/aka-rk3588，原基于 C 语言、运行于 Linux。它的完整回路包含追球、抓取、寻找红桶、趋近、投放五个阶段，衡量口径为自相机成帧到执行器指令的帧到指令延迟，这也是竞赛所采用的口径。

工作分为两部分。应用层一侧，让机器人在 StarryOS 上运行并优化其推理链路，依靠取球循环与数据布局，将帧到指令延迟降低至接近 Linux 的水平。系统层一侧，令整套 StarryOS 作为通用操作系统，在调度、系统调用、内存、频率这些通用的子系统上与 Linux 直接对比。据此，机器人只是众多负载中的一个，所考核的对象是操作系统的通用能力。

比较遵循统一准则。每个子系统采用各自的基准套件，各配一条独立的 Linux 基线，两侧运行同一份遵循 Linux ABI 的可执行文件，一份负载在两侧的差异因此只反映操作系统本身。后文先展开机器人应用，再依次呈现观测、调度、系统调用、内存、频率、启动等系统层子系统。对等类子系统各以对应的 Linux 基线为参照，并叙述相应改动；显示、图形合成、QEMU NPU 模型与板级 I/O 属使能与仿真类贡献，不作对等主张。

2 系统总览与测量方法

StarryOS 复用 ArceOS 的模块层，在其上补齐 Linux 兼容的系统调用、进程与信号，并按设备类型对驱动分层。系统内核侧的改动集中于调度与大小核、系统调用入口、分页与内存、频率电压以及观测；仿真与板级侧新增 QEMU 的 RKNPU 功能级模型、若干 RK3588 驱动与显示栈，并承担全部真机验证。两条主线的落点见图 1。



图 1: StarryOS 在 RK3588 上的分层结构，本队改动集中于调度、DVFS、分页与观测等层

机器人负载的闭环起始于相机成帧，依次经 JPU 解码、RGA 预处理、NPU 推理，直至定位与运动控制，再返回相机。帧到指令延迟即该闭环运行一周的端到端时间，各子系统的改动最终都汇入这一个数字。



图 2: 跨子系统对 Linux 的性能比值仪表盘 (越接近或超过 1.0 越好)

每个子系统各用一套业界基准测量，并各自配一条同机同频的 Linux 基线：sysbench cpu 考察调度分发与 DVFS 频率，sysbench memory 的 first-touch 考察分页与 THP，其写带宽考察 DDR 侧 DVFS (频率电压)，hackbench 考察高并发下的 fork 与调度，schbench 考察唤醒延迟，getpid 等空系统调用单独测量系统调用入口的开销。基准套件与子系统的对应见图 3。

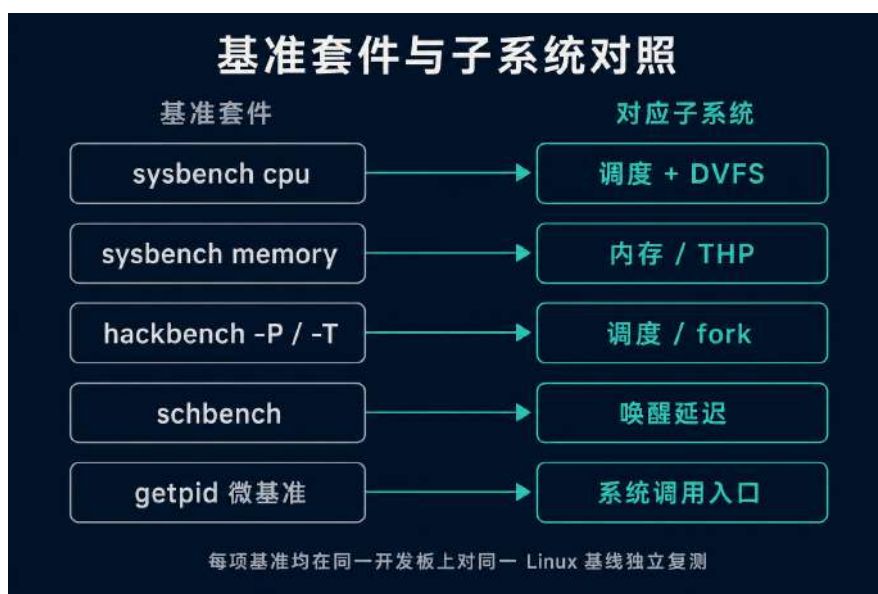


图 3: 各子系统的基准套件与其所测子系统的对应关系 (每项均配同机 Linux 基线)

测量口径本身亦予固定。基线版本冻结后，每轮改动均重新复测；两次读数漂移超过 $\pm 2\%$ 即判为环境噪声并重测。两侧使用同一份 ELF，以免编译差异混入结论。每个数据点取 $N \geq 5$ 次的 p50，不采用最优值。per-core 与聚合分别测量，先验证单核算力对齐，再考察多核堆叠。主线采

用累计阶梯，逐层叠加改动以观察每一步的增量；对相互影响的改动项，再单独做正交消融，将各自的贡献逐一厘清。

下表按能力域汇总前置目标的完成情况，九个能力域的既定目标均已达成；结果列直接引用全局数据宏，口径列标明各项的测量方法。

能力域	结果	状态	口径
观测 (perf/fttrace)	硬件 PMU perf 与 fttrace 已在真机运行，big.LITTLE 全量通过	已达	板级 perf-validate (39/0、56/0)
调度与大小核	cpu t8 4987ev/s，为 Linux 5222 的 0.96×	已达	sysbench cpu 8 线程 p50
系统调用入口	getpid 由 1200 降至 431ns，逼近陷入往返的架构地板，对 Linux 167ns 由约 7× 收窄至 2.6×	已达	getpid 微基准 p50
内存与分页	THP first-touch 0.030s，Linux 为 0.086s，快 2.87×（领先）	已达	128 MB first-touch
频率电压	CPU 调频调压经 PVTPLL 环长对齐 Linux，逐核持平；内存写带宽 0.61×，DDR 变频属主线固件限制	已达	逐核 cpu p50 + memory 写 8 线程
启动 TTFI	9.83s，优于 Linux 11.28s（领先）	已达	上电到首帧推理
显示栈	VOP2 与 HDMI-QP 上板点亮，ROPLL 像素时钟锁定验证通过	已达	主机寄存器测试 + 板级 ROPLL 锁定与 /dev/fb0 彩条
QEMU NPU 模型	RKNPU 功能级模型已合并，106 项 qtest	已达	qtest 回归
机器人应用	网球拾取机器人五阶段闭环在 RK3588 真机完整运行，感知到运动控制全链路打通	已达	整机闭环端到端运行

表 1：前置目标完成情况

3 基础版本与增量贡献

本节汇总增量工作及其相对基线的位置与状态；原生显示驱动栈（VOP2 与 HDMI）已完成主机侧验证与板级点亮（ROPLL 锁定），正整理为可独立拆分的 PR，稍后列入。基线取 rcore-os/tgoskits 的 dev 分支，commit 73409e079，上游仓库为 github.com/rcore-os/tgoskits。其上运行的机器人应用移植自 pengzechen/aka-rk3588，模型转换与量化工具取自 airockchip/rknn-toolkit2。集成分支的首个提交即把基线固定于该提交点，此后每一处改动都以它派生，作为一个原子 PR 回到上游、对应的跨内核共享 crate 或 QEMU fork。改动拆到一次即可独立回归的粒度，每条都能对照固定基线单独核对得失，评审也只需关注单一处逻辑。

全部改动分为两条并行推进的线。系统内核侧覆盖性能观测、调度、内存与页表正确性、CPU DVFS 与基准套件；仿真与板级侧覆盖 QEMU RKNPU 仿真器、加速器与图形驱动、串口与外设、网络及机器人应用。表 2 按子系统列出全部条目，状态分三档：已合并表示已并入对应目标仓库（上游或 QEMU fork），在审表示已提交并处于评审中，已准备表示已在集成分支实现、待

拆分为独立 PR。

贡献	位置/PR	状态
单核 ARM PMUv3 硬件 perf	rcore#1395	已合并
多核与 big.LITTLE 硬件 PMU perf	rcore#1577	在审
软件事件与 HW_CACHE 缓存事件	rcore#1601	在审
采样归属真实 tid 与丢样记录	rcore#1602	在审
事件组与组采样读	rcore#1603	在审
跨核互唤醒死锁修复	rcore#1495	已合并
新建与被唤醒线程跨核分发	rcore#1656	在审
sysbench big.LITTLE 套件与板级 harness	rcore#1658	在审
RK3588 CPU DVFS 与 PMIC 电压标定 (cpufreq)	rcore#1657	已合并
RK3588 RGA2 二维加速器	rcore#1388	已合并
RK3588 JPU (VDPU720) 硬件 JPEG 解码	rcore#1456	已合并
rknpu DRM ioctl 辅助函数整合	rcore#1351	已合并
RKNPU GEM mmap 遵循 cache 标志	rcore#1364	已合并
RK3588 RKNPU 功能级仿真	qemu#19	已合并
RKNPU 多核协同执行	qemu#26	已合并
reboot(2) 系统调用	rcore#1358	已合并
USB-serial tty	rcore#1378	已合并
RK3588 UART 初始化	rcore#1921	已合并
TTY 输入 flush 与唤醒修复	rcore#1922	已合并
RK3588 PWM sysfs	rcore#1468	已合并
UVC 摄像头异步传输生命周期	rcore#1924	已合并
UVC 测试迁移到 ax-sync	rcore#1961	已合并
rtnetlink IPv4 与多网卡 regulator GPIO	rcore#1497	已合并
ax-net 数据报睡眠互斥修复	rcore#1963	已合并
图形合成器栈: Weston 于 StarryOS (DRM/KMS、evdev 输入、AF_UNIX 传递、epoll/timerfd、Xwayland)	rcore#503-516、#1160	已合并
THP 首次触碰与 fork/拆分正确性	starry-mm	已准备
页表与 TLB 正确性修复	page_table_multiarch	已准备
机器人应用 (底盘里程计定位)	apps/starry	已准备

表 2: 增量贡献一览, qemu 前缀指 processmission/qemu fork, 其余为 rcore-os/tgoskits 上游

三项已准备的工作已在集成分支实现并经 RK3588 板级验证, 并将按同一原则拆分为独立 PR 回馈上游。内存侧为 **THP 2MB** 私有匿名首次触碰及其在 fork 与拆分路径上的正确性修复; 页表侧集中于跨内核共享的 `page_table_multiarch` 与 `page_table_entry`, 含一类释放后使用、一处内存序缺陷与一处 SMP 广播漏项; 应用侧为基于底盘电机编码器里程计的机器人桶位定位。

第二部分 应用层：视觉感知与推理链路

4 视觉感知流水线

网球机器人的视觉感知从一帧图像开始，到网球在画面中的坐标结束。相机取来一帧，解码、缩放与色彩转换把它整理成检测网络要求的输入，NPU 推理给出一组候选框，后处理在这些框里筛出网球并算出它的位置。这条流水线的每一个阶段都有两条落点，既可以留在 CPU 上由通用核逐像素完成，也可以交给 RK3588 上对应的专用硬件单元。JPEG 解码、二维缩放与色彩转换、张量推理这三步在 CPU 上各自的开销都与一次推理相当，若都压在通用核上，感知与控制会争抢同一组核，流水线随即被拖住。把它们逐级交给 JPU、RGA、NPU 三个专用单元，通用核才能腾出来去做决策与运动控制。

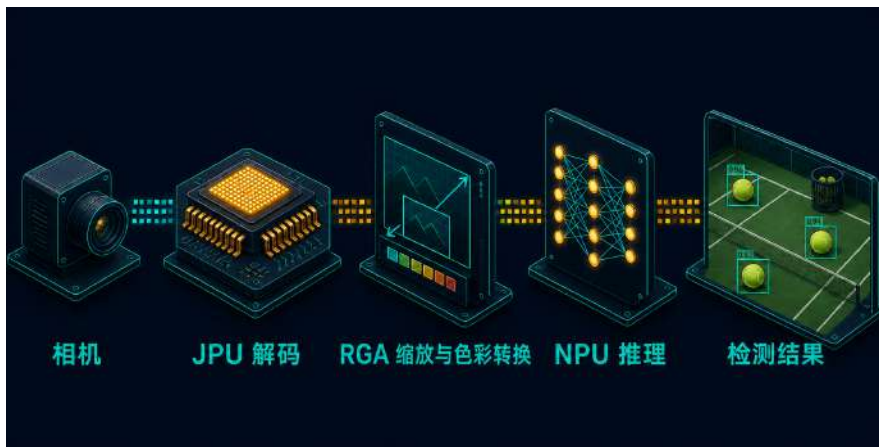


图 4：视觉感知流水线：相机取流 → JPU 解码 → RGA 缩放与色彩转换 → NPU 推理 → 后处理

图 4 给出这条流水线的分级结构。其中相机的 USB/xHCI 主机栈与 NPU 的 `/dev/card1` 节点为既有能力，JPU 驱动、RGA 驱动、dma-heap 分配器与 NPU 的 GEM 缓冲修复为本队新增。以下逐级说明各硬件单元与对应的工作。

4.1 流水线各阶段

相机 流水线的第一级是接在 RK3588 USB3 端口上的相机。它符合 USB 视频类（UVC）规范，经 Starry 既有的 xHCI 主机栈以等时传输取流，无需专用驱动。这只相机交付两种格式，YUYV 是未压缩的 4:2:2 原始像素，取来只需一次色彩转换；MJPEG 把视频的每一帧各自压成一张独立的 JPEG，取来还要先解码。两种格式后续都进入 RGA 与 NPU，区别只在前端是否多一步解码。这一级复用既有主机栈，取流通路本身不引入新驱动。xHCI 端点上下文的 `Interval` 字段决定控制器多久为等时端点取一次数据，它并不直接等于描述符里的 `bInterval`，须按端点类型与连接速度换算。对高速及更高速度的等时端点，正确换算是 `Interval` 取 `bInterval` 减一，含义为每 2^{Interval} 个 $125\mu\text{s}$ 微帧服务一次；`usb-host` 的 `calculate_xhci_interval` 按速度与端点类型分类固定这一换算。若图省事以一个固定下界把该值夹到 1，`bInterval` 为 1 的高带宽端点会被编码成每两个微帧才服务一次，可用带宽当场减半，相机内部缓冲随即溢出，等时帧被成片截断。同一段端点配置还显式把等时端点的错误计数置零（等时传输本不重传），并按每微帧包数算出突发大小与一次服务区间的最大载荷（max ESIT payload）一并写入上下文，控制器才会按

相机申报的带宽真正预约总线。对实到字节短于申报长度的等时帧，驱动逐包归类为足额、偏短或为零，并与缺失完成事件的包数分开统计，从而把事件投递丢失与控制器侧限速或 FIFO 截断两类成因区分开，这是在板上诊断一条被截断的等时输入流时唯一能凭代码坐实的判据。

JPU MJPEG 帧的解码交给 JPU，即 RK3588 集成的硬件 JPEG 解码单元（设备树节点 `jpegd@fdb90000`）。在 CPU 上软解一帧约需 25ms，几乎与一次推理相当；JPU 在硬件中完成熵解码与反变换，直接把一张 JPEG 还原为 NV12 帧写入物理内存，使这一步从计算路径上让出。Starry 此前没有这块硬件的驱动，现已新增其平台无关的驱动核心与 `/dev/mpp_service` 设备节点，对齐瑞芯微 MPP 库的会话协议，使未经改动的 MPP 程序照常驱动这块解码器。MPP 与内核之间是一串按字节对齐、经 `ioctl` 传入的请求记录，每条为一个 24 字节结构，带命令号、标志位与一个指向各自负载的用户态指针，多条以 `MULTI_MSG` 标志串成一批至 `LAST_MSG` 为止；会话先以 `PROBE_HW_SUPPORT` 询问本节点支持的编解码并据实回报只支持 JPEG 解码，再以 `QUERY_HW_ID` 读硬件版本、以 `INIT_CLIENT_TYPE` 把会话绑定到 JPEG 解码客户端类型，随后 `SET_REG_WRITE` 送入整组寄存器、`SET_REG_READ` 声明完成后回读的寄存器窗口，最后 `POLL_HW_FINISH` 触发一次提交并阻塞到这一帧解码完成。提交前节点按厂商 `trans_tbl_jpgdec` 给定的位置逐一扫描地址槽，把 `dma-buf` 文件描述符经共享的 `/dev/dma_heap` 解析为其连续缓冲的物理基址、再加声明的偏移写回寄存器，量化表、霍夫曼最小码表与霍夫曼值表共处一段表缓冲、分别落在偏移 384 与 704 字节处。设备初始化时把前置的 IOMMU 置为直通：其寄存器在同一寄存器页偏移 0x480 处，先写入强制复位命令清掉前一个系统可能遗留的页表，再把页表基址寄存器 `RK_MMU_DTE_ADDR` 清零、写入停用分页命令 `RK_MMU_CMD_DISABLE_PAGING` 并屏蔽全部中断，此后引擎按物理地址直接读写内存。运行时把整组寄存器写入硬件时跳过只读的 `ID` 寄存器，并把带启动位的控制寄存器 `SWREG1` 留到最后一个写，从而保证几何、格式、表长与五个地址槽都就位之后引擎才被拉起，绝不会有输入尚未配齐时就开始取数。开发板上单帧解码中位约 1ms，输出与 Linux 逐字节一致。该驱动以合并请求 #1456 提交至上游，已合并。

RGA 缩放与色彩转换交给 RGA，即 RK3588 集成的二维栅格图形加速器，它在硬件中完成色彩空间转换、缩放与图层拼接。相机交付的是 YUV 排布的像素，而检测网络要求连续的 RGB 平面，尺寸还需缩放到模型的输入分辨率；在 CPU 上逐像素做这件事，一帧的开销与一次推理相当。Starry 此前没有这条通路，现已新增平台无关的 **RGA2** 驱动核心与 `/dev/rga` 设备节点，按 `librga` 的真实二进制接口实现 `rga_req` 结构体布局与 `RGA_BLIT_SYNC` 约定，使既有的 `librga` 二进制无需改动即可驱动 RGA2 核。`rga_req` 在 LP64 下恰为 504 字节，内嵌三份各 56 字节的图像描述符，缓冲导入用的 `rga_external_buffer` 为 288 字节，这些尺寸以编译期断言固定，任何字段增删都会在编译时暴露；早期镜像漏掉 `rotate_mode`、`rd_mode` 等较新字段，使其后所有字段整体偏移，引擎据此取到错位的旋转矩阵与地址。`librga` 在打开设备时查询驱动版本与硬件版本以选择结构体布局，因此设备节点须如实报告 `MultiRGA v1.3.1` 与 RK3588 上 RGA2 核的版本号，`librga` 才会按这一布局组包并把请求投向承载 RGA2 的那个核。RGA2 的执行时序对寄存器写入顺序敏感：引擎须先被置入主模式，命令起始信号才会触发一次命令 DMA 取指，且表示全部命令完成的状态位只有在对应中断使能于起始之前被置位时才会锁存，最初省略这一步会使引擎跑完而完成位始终为零、轮询一路超时；据厂商时序在主模式与起始之间补上对中断使能位的读改写后完成位正常锁存，既有的忙等轮询即可观察到它，无需真正布线中断。缩放系数以

16.16 定点表示, 缩小时正确系数是目标维度比源维度、其值不超过一并写入低半字, 最初误用其倒数使源指针每步跨过有效区域, 扫描器随即停在忙状态约 50 ms 不再前进, 按缩小与放大两种情形分别依厂商参数计算后缩放回到一次提交即完成。该驱动以合并请求 #1388 提交至上游, 已合并。

NPU 整理好的一帧送进检测网络, 承担这一步的是 RK3588 片上集成的神经网络加速器 NPU。运行时以用户态共享库 `librknnrt.so` 加载 RKNN 模型、提交一帧、取回结果, 经 `/dev/card1` 这个 DRM 字符设备下达到内核。这个节点及其 RKNPU 驱动本已存在, 但要承载一帧接一帧的连续推理, GEM 缓冲对象在映射与销毁两处的行为须被修正。映射时若不正确处理这段区域的缺页, 第一次访问就触发无法收场的异常, 紧随其后的推理取不到输入; 销毁时若引用计数处理有误, 被提前回收的那段内存会在下一帧被再次创建时拿到错位的地址。本队修正了缺页处理与引用计数这两处, 连续推理随之稳定, 首次推理的检测结果与 Linux 一致。这部分对 GEM 缓冲对象的修复以合并请求 #1364 提交至上游, 已合并。

零拷贝底座 三个加速器各有独立的总线主控通道, 若每一级都把上一级的结果先搬回 CPU、再拷贝给下一级, 单帧仅在搬运上就要付出与一次推理相当的内存带宽与缓存抖动。让它们读写同一段物理内存是整条流水线能够流动起来的前提。Starry 此前没有对应机制, 现已新增 **dma-heap 分配器与 dma-buf 零拷贝导入**: 一段物理连续内存经 `/dev/dma_heap` 只分配一次, 以文件描述符在 JPU、RGA、NPU 之间传递, 每个部件凭描述符解析出同一个物理地址, 整条 MJPEG → JPU → RGA → NPU 通路因此全程零拷贝。应用以一次 `DMA_HEAP_IOCTL_ALLOC` 提交一份 `dma_heap_allocation_data` 请求长度, 该结构在 LP64 下恰为 24 字节, `ioctl` 编号由内核侧按 `_IOWR('H', 0, ...)` 推算并以编译期断言固定为 `0xC0184800`, 与主线 Linux 的取值逐位一致, 任何字段或编号的偏移都会在编译时暴露。分配落在 4 GiB 以下的物理空间、按页对齐且物理连续, 这两条约束各有来由: 物理连续是因为 RGA2 与 NPU 在本阶段以设备侧地址翻译部件 (IOMMU) 旁路、物理直通的方式沿一个起始物理地址顺序访问整段缓冲, 中途若被分页打断便会读到错误的页; 32 位地址上限则对应 RGA2 把一个裸 32 位物理基址直接写入自身寄存器的约束, 分配器因此以 `u32::MAX` 作为 DMA 掩码并把请求长度限制在 32 位以内。设备物理直通带来缓存一致性问题: 在把缓冲交给引擎之前沿写入设备的方向同步一次、将脏的 CPU 缓存行刷回内存, 待引擎完成、读取结果之前再沿读回 CPU 的方向同步一次、使目标缓冲对应的缓存行失效, 这两次方向相反的同步只针对实际参与本次操作的源与目标缓冲, CPU 与设备据此看到一致的内容。

4.2 在真机上的结果

在 OrangePi-5-Plus 上按分级测量整条流水线, 端到端时延阶梯见推理链路优化一节。

分级来看, RGA 承担的图像准备阶段较 CPU 缩放快约 7.4 倍, 色彩空间转换与缩放接近归零。JPU 单帧解码中位约 1 ms, 解出的帧与 Linux 逐字节相同。推理本身 (run) 因加速器时钟固定在 1.0 GHz 不做动态降频, 与 Linux 在同一时钟下处于同一水平, 约 4.7 ms。

5 推理链路优化

推理链路是这套视觉应用的主体。相机取来一帧，做颜色空间转换与缩放补边，送入 NPU 前向推理，再取回输出并解码为检测框，逐帧给出结果。整条链路由一个用户态 C++ 应用串起，各阶段依次为 capture (取帧)、decode (YUYV 转 RGB)、letterbox (缩放补边)、inputs_set (写入 NPU 输入并维护缓存)、run (NPU 前向)、outputs_get (取回输出并重排) 与 postprocess (DFL 解码与 NMS)。链路本身、RKNN 运行时 librknnrt、用户态 librga 库、NPU 硬件以及 /dev/card1 上的 RKNPU 驱动均为既有部分；相机取流复用 Starry 既有的 xHCI 主机栈。两侧 OS 对测的是同一份未经改动的 ELF，观测到的差异因此只归于运行环境，应用本身两侧相同。StarryOS 此前在这份负载上端到端落后 Linux 约 6.3 倍，热点散布在链路各阶段，无从直接读出，本节先逐阶段说明各阶段的作用与开销来源，再说明两种输入尺寸的口径，随后以真机剖析把成本归因到确切位置，最后逐项说明现已实现的优化及其机制与收益。

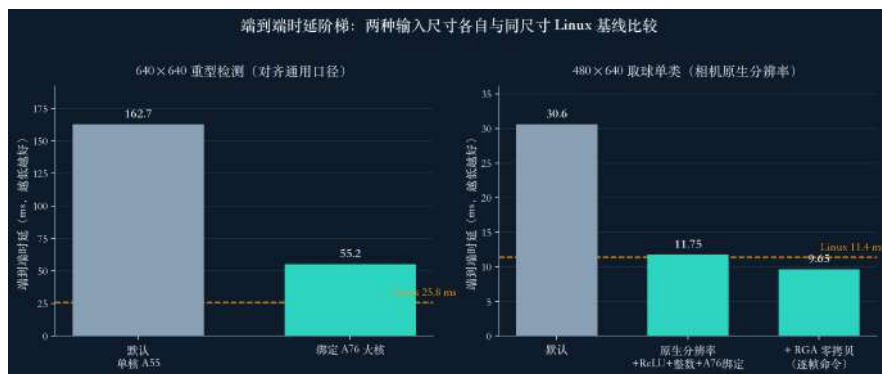


图 5: 两种输入尺寸各自的端到端时延阶梯，各与对应尺寸的 Linux 基线比较

图 5 分别给出 640×640 与 480×640 两种输入尺寸各自的端到端时延阶梯，每条阶梯从 StarryOS 默认配置起，逐项叠加本节的优化，各与对应尺寸的 Linux 基线比较。其中链路的运行时、用户态库与 NPU 硬件为既有部分，四项优化以及配套的内核 /dev/rga 驱动、JPU 驱动与全链路剖析插桩为本队新增。

5.1 推理链路各阶段

一帧从相机到检测框依次经过七个阶段，各阶段承担一件事，其开销来源各不相同，以下逐阶段说明。

capture 取帧 链路的第一级是经 xHCI 主机栈以等时传输从 UVC 相机取一帧。相机交付两种格式：YUYV 是未压缩的 4:2:2 原始像素，每帧约 614KB，取来只需一次颜色转换；MJPEG 把每帧各自压成一张独立的 JPEG，每帧约 30KB，取来须先解码。取帧速率受相机与 USB 供帧上限约束，在 USB2 上 YUYV 的供帧上限约 30fps。这一级本身不做计算，其开销体现在 USB 带宽与供帧节奏上；在 StarryOS 未绑核时，取帧线程与后续阶段争抢同一个核，实测取帧速率被拖到约 12fps。

decode 颜色转换 相机交付的是 YUV 排布的像素，检测网络要求连续的 RGB 平面，因此须做一次 YUYV 到 RGB888 的颜色空间转换。在 CPU 上这是对整帧逐像素做定点算术，A76 大

核上约 5.7 ms。开销正比于像素数与每像素的转换算术量，是绑核之后 CPU 侧最高的阶段。

letterbox 缩放补边 模型只接受固定的输入分辨率，因此须把相机帧等比缩放到模型尺寸并以灰边补齐 (letterbox)，使长宽比不失真。CPU 路径逐像素做双线性重采样，一帧的开销与一次推理相当，在 StarryOS 未接入用户态 RGA 时，A55 上高达 75.5 ms，是两侧 OS 之间绝对差距最大的单一阶段 (Linux 上走 RGA 硬件为 1.38 ms)。开销正比于重采样的像素数。

inputs_set 写入输入并维护缓存 推理前须把整理好的输入帧写入 NPU 的输入缓冲，并做一次缓存维护，把 CPU 侧写入的数据刷回内存，使 NPU 读到最新内容。开销由一次内存拷贝加一次缓存刷回构成。稳态下这一阶段很轻，真机实测 0.84 ms，内核侧的缓存维护 ioctl (mem_sync) 仅 0.082 ms；批量定像素跑法下曾观察到的 51.73 ms 是首帧 DMA 分配的冷启动伪量，长跑稳态并不出现。

run NPU 前向 整理好的一帧送入 NPU 做前向推理，这一阶段的计时包住向内核提交作业的 ioctl 与等待完成。640×640 模型的 NPU 纯计算约 16 ms，算子级瀑布显示其为卷积主导，因此只有模型级改动能压缩它。这一阶段的墙钟时间不只含 NPU 计算，还含主机侧的调度等待：内核侧的提交 ioctl (submit) 仅 0.36 ms，而在 StarryOS 未绑核、与取帧解码争抢同一个 A55 小核时，实测 run 膨胀到 71 ms，反映的是被测线程迟迟得不到调度以观察完成，绑定到空闲的 A76 核后随即回落到接近 NPU 计算底线。

outputs_get 取回并重排 推理完成后须把 NPU 的输出张量取回。NPU 以硬件分块布局 NC1HWC2 写出张量，而解码器要求标准的 NCHW 排布，因此每个元素都要按新布局重新聚拢；若再要求浮点输出，还须把全部张量从 INT8 反量化为 FP32。640×640 模型有 9 个输出张量、约 40 万个数值，其中三个 [1,64,H,W] 的框 DFL 张量占大头。这一重排与反量化在 CPU 上完成，在 StarryOS 默认单核上高达 14.3 ms，是仅次于 letterbox 与 run 的第三高阶段，其开销正比于被重排与反量化的元素总数。

postprocess 后处理 最后一阶段把 NPU 输出解码为检测框：先做 DFL 解码，将每个网格单元的框分布对数按加权求和还原为坐标，再做 NMS 抑制相互重叠的候选框。这一阶段很轻，约 1 ms，其开销正比于通过分数门限的候选单元数，而密集网格中通过门限的单元通常只有约 30 个。

5.2 两种输入尺寸的口径

链路涉及两种输入尺寸，对应两条互不通约的口径，任何图表都不能把它们并入同一条曲线。一条是 640×640 的重型 YOLOv8 检测模型，8400 个锚点，用于与通用检测口径对齐，其 Linux 深剖基线端到端 25.8 ms，其中 NPU 前向占推理时间的 79%。另一条是与相机分辨率一致的 480×640，6300 个锚点，用于取球应用的单类检测，其 Linux 基线端到端 11.4 ms。两者的模型规模、锚点数量、检测头算量与后处理量都不同，各自的 Linux 参照也不同，因此本节始终按尺寸分列，绝对时延只在同一尺寸的两侧 OS 之间比较。

5.3 基线剖析与瓶颈归因

优化之前先在真机上把成本归因到确切阶段。最初一份二手口径的对比表给出了若干直觉性归因（letterbox 33 倍出自缺少 RGA、outputs_get 35 倍出自缓存同步、run 2 倍出自 DVFS），这些归因一律作为待测假设处理，逐条以真机测量核对，其中两条被测量修正。

NPU 算子瀑布 对 640×640 模型做算子级剖析，108 个算子中 ConvExSwish 一类占 72%，Concat、Split、Add 等合计不足两成，模型为卷积主导。这一结论确立了一条边界：run 阶段的时间由模型的乘加量决定，任何软件层改动都无法压缩它，只有模型级改动（分辨率与激活）能触及，直接指向后文的原生分辨率与 ReLU 两项。

消融测量 逐次只改一个变量。NPU 核掩码从 1 核到全 3 核，2 核相对 1 核提速 11%，第 3 核起不再有增益，说明这颗小模型有效只用一到两个 NPU 核。调频策略在 ondemand 与 performance 之间读数一致，NPU 全程锁在 1.0 GHz，NPU 时钟不是波动来源。letterbox 走 RGA 硬件为 1.72 ms，走 CPU 为 15.82 ms，硬件路径省约 14 ms、约 9.2 倍，这正是两侧 OS 之间 letterbox 差距的机制来源。张量 I/O 的拷贝与零拷贝两种模式在 Linux 上分别为 23.83 与 24.65 ms，零拷贝并未更快，因为它虽消去了 inputs_set，却把 NC1HWC2 到 NCHW 的重排推给了 CPU。

冷启动、调度与内存带宽 首次推理的冷启动为 450 ms，其中模型初始化 15 ms、取流初始化 93 ms、首帧等待 300 ms、首次推理 42 ms；首帧等待占大头，来自相机流协商，属硬件行为。调度器剖析显示被测进程的调度延迟平均 0.021 ms、最大 0.162 ms，因此端到端的队龄长尾出自取最新帧的槽位策略，与 CPU 饥饿无关。内存带宽方面，NPU 流量约 1.06 GB/s，CPU 缓存缺失率 2.3%，远低于 LPDDR 约 17 至 32 GB/s 的上限，链路不受内存带宽约束。

内核 ioctl 计时 在内核侧对 NPU 的 ioctl 单独累计计时：提交作业 submit 平均 0.36 ms、缓存维护 mem_sync 平均 0.082 ms、DMA 分配 mem_create 平均 2.41 ms。submit 远小于 run 的约 20 ms，说明内核 NPU 通路不是成本中心；mem_sync 仅 0.082 ms，说明定像素跑法下 inputs_set 与 outputs_get 上观察到的几十毫秒是用户态的缓冲处理与重排，不是内核缓存同步。这条测量把成本准确定位在用户态取回与重排，直接指向后文的整数取回路径。真机默认配置下 StarryOS 端到端 162.7 ms，相对 Linux 的 25.8 ms 为 6.3 倍，letterbox 与 outputs_get 两阶段占据了这段差距的主体。

5.4 将最重的推理绑定到 A76 大核

StarryOS 的 axtask 把新建与唤醒的线程放在当前核上，且此前无负载均衡；应用不做任何绑定时，取帧到后处理的整条链路串行落在 cpu0，即一个 A55 小核上，与之竞争的取帧、解码线程也在同一核上排队。将推理线程绑定到 A76 大核簇（4-7）后，sched_setaffinity 在 StarryOS 上被如实执行。这里有一个次序约束：rknn_init 与 USB 初始化在非引导核上会挂起，因此亲和性须在模型与相机初始化完成之后、进入推理循环之前设置。绑定后 640×640 端到端由 162.7 ms 降到 55.2 ms，约 3 倍，其中 run 71.1→24.5、letterbox 75.5→26.4、outputs_get 14.3→2.7，仍高于 Linux 的 25.8 ms。核驻留直方图确认推理 100% 落在 cpu4：掩码被如实执行，但无均衡器

时整条链路仍堆在单一 A76 核上，残差因此集中在这一核上的用户态 letterbox 与重排，进而引出后两项优化。

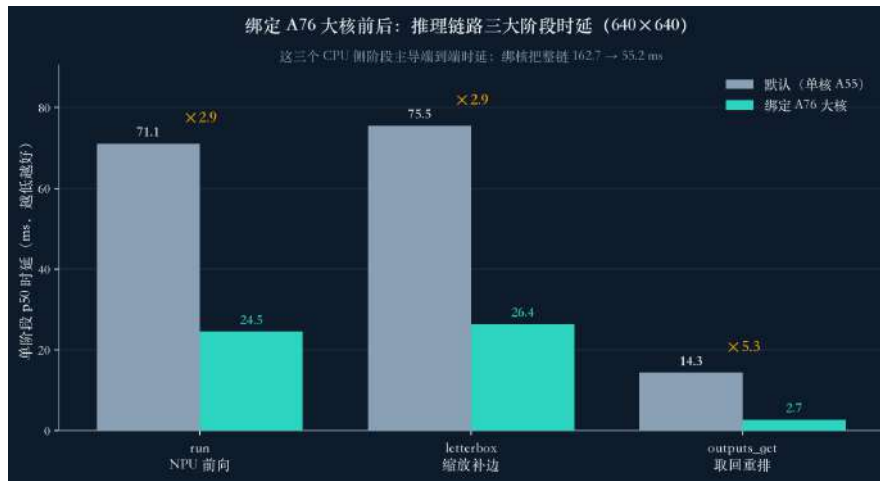


图 6：绑定 A76 大核前后推理链路三大阶段的 p50 时延 (640×640)：run、letterbox、outputs_get 三个 CPU 侧阶段主导端到端时延，绑核把整链由 162.7 ms 压到 55.2 ms，三段各降约 2.9、2.9、5.3 倍

5.5 以整数路径取回输出

YOLOv8 的密集网格中超过 99% 为背景，640 尺寸下 8400 个单元里通常只有约 30 个携带真实检测。原先的单类解码器以 `want_float=1` 调用 `rknn_outputs_get`，迫使运行时把全部 9 个输出张量从 INT8 反量化为 FP32 并从 NC1HWC2 重排为 NCHW，约 40 万个数值几乎全是背景。改以 `want_float=0` 取回原始 INT8：先在 INT8 域以一次单字节比较对分数分支做门限筛选背景，仅对通过门限的存活单元的约 64 个 DFL 值做仿射反量化，反量化量由约 40 万降到约 30 个单元、每单元 64 值。outputs_get 由约 12ms 降到约 1ms，端到端由约 23ms 降到约 11ms，285 张测试图的检测与浮点路径逐张一致，无精度损失。这是纯软件改动，不依赖任何硬件或内核支持。

5.6 RGA 硬件解码并零拷贝直写 NPU 输入

绑定大核之后，YUYV 到 RGB888 的颜色转换 (decode) 成为最高的 CPU 阶段，A76 上约 5.7ms，已超过这颗小模型的 NPU 计算。将其交给 **RGA2** 二维引擎 (`imcvtcolor`)，并以 Path-A 真零拷贝把结果直接写入 NPU 的输入 dma-buf：NPU 关闭 IOMMU 运行，其 `dma_addr` 即物理地址，RGA 可直接以该物理地址为目的地写入，这一步同时消去了 `inputs_set`。decode 由约 5.7ms 降到约 0.77ms，为 7.4 倍，inputs_set 归零，检测全程一致 (415/415 与 336/336 逐帧一致)。这条路径经过 14 轮在板调试才打通：跨分配器的文件描述符导入失败，改以物理地址导入；设备树暴露三个 RGA 核，拼接一度锁到 RGA3 核，须按核选择修正；最终的决定性问题是 `librga` 会填入一个恒等旋转矩阵 (`cosa=65536`，即 16.16 定点下的 $\cos 0^\circ$)，ABI 解析误将其判为非法，修正为仅在 `rotate_mode != 0` 时才走旋转分支。内核 `/dev/rga` 驱动以合并请求 #1388 提交至上游并已合并，通过 5/5 项硬件自检；驱动内部见视觉感知流水线一节。

5.7 原生分辨率模型与 ReLU 激活

这一项针对 run 阶段，因为算子瀑布已表明只有模型级改动能压缩 NPU 前向。一个 RKNN 只为固定输入形状编译，把 480×640 的相机帧送进 640×640 的模型会把它再缩放补边放大，NPU 反而要在灰边上做卷积。将模型按 480×640 原生分辨率导出后，像素数由 409,600 降到 307,200，锚点由 8400 降到 6300，检测头约 0.75 倍算量；训练仍用方形 640 加马赛克增强，仅在导出时设定矩形形状。同时以 ReLU 替换 SiLU 激活：SiLU 为 $\text{sigmoid}(x) \cdot x$ ，在 NPU 上更慢且 INT8 量化有损，而 ReLU 的 NPU 前向约快 2.5 倍且量化干净，mAP50 为 0.881 对 0.885，二者相当，INT8 余弦保真度 0.998 至 1.000。两项与前述整数取回路径、A76 绑定叠加后， 480×640 端到端 p50 由 30.6 ms 降到 11.75 ms。

5.8 JPU 硬件 MJPEG 解码

取帧格式改为 MJPEG 可把 USB 字节数压缩约 20 倍（614KB 降到约 30KB），抬高约 14fps 的取帧上限，解码则交给 RK3588 集成的硬件 JPEG 单元 JPU（jpegd@fdb90000，位于 IOMMU 之后），经瑞芯微 MPP 库的 /dev/mpp_service 会话协议下达，输出 NV12/NV16，理想情形下直接喂给 NV12 输入的 RKNN 以省去颜色转换。独立验证中，未经改动的 mpi_dec_test 在 StarryOS 上经本队新增的 /dev/mpp_service 解码，输出与 Linux 逐字节一致（OUT_CHECKSUM 2712426383），持续 395fps、约 2.5ms/帧，驱动本身经真机验证；其中一处 dma32 修复解决了 32 位引擎无法触及 4GiB 以上堆缓冲导致的取值失败。JPU 驱动内部见视觉感知流水线一节。

5.9 已否定与在设计中的方案

张量 I/O 零拷贝：已否定 一度尝试以 rknn_create_mem 把 NPU 的输入与输出都绑为 DMA 缓冲，以省去主机与 NPU 之间的拷贝和缓存同步。测量结果为两侧 OS 均净亏：Linux 由 23.83 升到 24.65ms，StarryOS 由 217.63 升到 266.08ms。原因是零拷贝把 NC1HWC2 到 NCHW 的输出重排强行压到慢的单核上，而内核缓存维护本就只有 0.082ms/次，可省的拷贝成本很小。据此保留拷贝路径，零拷贝二进制仅作为 A/B 对照留存；由于它绑定的是每上下文私有的 DMA 缓冲，必须拒绝多 worker，否则共享输入缓冲会被覆写。

全链路多 worker 并行：在设计中 推理路径是单线程串行地做解码到后处理，即便内核均衡器完美也只能把它放到一个核上。设想把它表达为 N 个相互独立的整链 worker，每个 worker 持有自己的 rknn_dup_context（在 cpu0 上创建后再迁移），分别绑定到不同的 A76 核。已识别的上限是：NPU 的 run 阶段无论是否 dup_context，都在 rdrive 的单一设备锁上串行（Error::Busy），CPU 阶段（解码、缩放补边、后处理）仍可在 A76 各核上并行，因此在 RGA 之前 letterbox 与 run 相当时，多 worker 仍能提升吞吐；真正的 NPU 并发须依赖内核作业队列。此项已成设计，为推理侧横向扩展的下一步。

5.10 结果

两条口径分别收敛，见图 5。 640×640 上，端到端由 162.7 ms 经大核绑定降到 55.2 ms，相对 Linux 的 25.8 ms 由 6.3 倍收窄到约 2.1 倍，残差为单一 A55/A76 核上的用户态 letterbox 与

输出重排。480×640 上，A76 绑定、整数取回路径与原生分辨率加 ReLU 模型三项叠加后，端到端 p50 为 11.75 ms，与 Linux 的 11.4 ms 持平；再加 RGA 硬件解码零拷贝后，逐帧命令时延 p50 降到 9.65 ms、约 28 fps，转为受相机供帧上限约束。综合而言，重型 640 模型上的差距由 6.3 倍收窄到约 2.1 倍，小模型经由 Linux 同样具备的用户态优化从约 **2.68** 倍收敛到持平，残差是单核上的用户态工作。本节不主张快于 Linux，主张的是一个研究型 OS 在真实加速器硬件上达到与 Linux 的功能与性能对等。

5.11 剖析方法

全部数据以同一份未经改动的 ELF 在两侧 OS 上对测，观测差异因此归于运行环境。全链路按阶段记录分位数并逐帧留痕，端到端与各阶段均取 p50 比较。带算子级插桩的诊断运行与干净运行分开采集，诊断插桩会把总量抬高到约 23 ms（干净约 15.4 ms），因此只读其份额、不读绝对值。内核侧对 NPU 的 ioctl 单独累计计时，把内核成本与用户态成本分离开，据此判定成本集中在用户态取回与重排。真机运行在 OrangePi-5-Plus 上，由机器可判定的判定门驱动，一次运行在同一次启动内扫过默认、绑 A55、绑 A76 三种配置并记录核驻留直方图。上游同步 76 个提交后按 ±2% 容差复测，端到端 25.81 对 25.90、漂移 0.3%，RGA 约 9.2 倍与张量零拷贝不更快两项消融结论均复现。

6 机器人拾球流程与本体侧优化

网球拾球机器人是本报内核侧工作的应用载体与验证对象。调度亲和、DVFS 与启动路径上的改动，最终都要落到这台机器人上电、追球、抓取、回桶的完整动作里接受检验。本体侧是一份用户态 C++ 可执行文件，主机 dry-run 与 RK3588 板端运行同一份状态机源码，仅替换电机与机械臂后端；因此感知链路的端到端时延在两处口径一致、可复现。感知在 480×640 分辨率上的端到端时延与 **Linux 持平 (11.75 ms 对 11.4 ms)**，上电到首帧推理 (TTFI) 领先 Linux (9.83 s 对 11.28 s)，二者机理分别见推理链路一节与 TTFI 一节。把推理线程固定到 A76 大核簇、把解码交给 JPU、把色彩转换与缩放交给 RGA，是内核侧工作落到本体的直接抓手，其加速见推理链路与视觉感知两节。本节只讲本体侧的应用逻辑：纯逻辑拾球状态机、可换真实执行器后端，以及里程计引导的回桶。

运行时的数据流是一条闭环。应用从 UVC 相机取帧，感知按当前状态在两个检测器之间二选一：追球阶段把整理好的帧经 RGA 与 JPU 的零拷贝路径送入 NPU 上的单类 YOLOv8 检测头，DFL 解码后取置信度最高的球；寻桶阶段改用整型 HSV 红色掩膜，对满足色相与饱和度阈值的像素以四连通并查集求最大连通域来定位桶。该 YOLOv8 头会自动识别单类结构，其 DFL 解码此前只支持框通道数 64 (reg_max 为 16) 的检测头，现已推广到框通道数 **32 (reg_max 为 8)** 的紧凑头，解码时按框通道数除以 4 推得 reg_max，无需重训或更换后处理即可兼容更小的导出模型。感知结果进入状态机生成底盘与舵机指令，经去重后下发给差速底盘与 ZP10S 机械臂。perception_mode 依状态发布感知模式，CHASE BALL 与 GRAB 走球检测器，其余状态走桶检测器，感知因此不会跑错检测器。以下按状态机、执行器后端、里程计回桶、工程加固四部分分述。

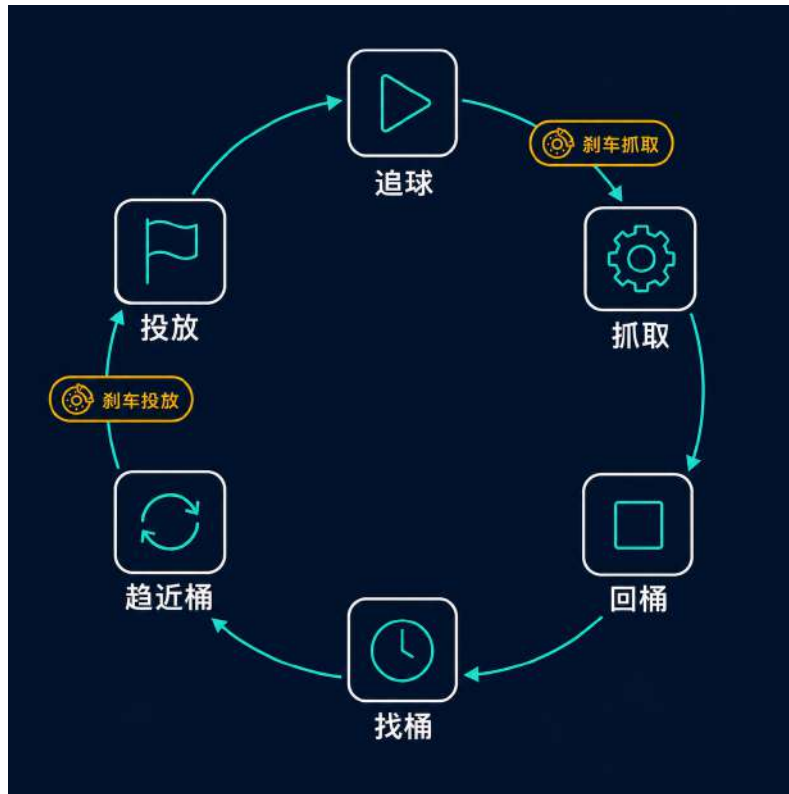


图 7: 拾球状态机：六状态非阻塞闭环，两个刹车时相由单调时钟推进，定时时相时长与相机供帧频率解耦

6.1 拾球状态机

拾球逻辑是一台六状态有限状态机，一轮完整循环沿 CHASE_BALL、GRAB、RETURN_TO_BUCKET 或 FIND_BUCKET、APPROACH_BUCKET、DEPOSIT 推进，投放完成后复位里程计并回到 CHASE_BALL。两个刹车时相 BrakeForGrab 与 BrakeForDeposit 穿插在抓取与投放之前，让底盘在动作前停稳。状态机的推进是纯逻辑的：单次 step 只依据当前检测与时钟给出下一步抽象指令，本身不做任何 I/O、不睡眠，串口写入与舵机阻塞等待都隔离在 dispatch 一侧。这样一来单帧感知无论快慢都不会阻塞状态推进，同一份状态机在主机 dry-run 与板端运行完全一致。

定时时相以单调时钟的绝对到期时刻计时。刹车保持 350 ms、抓取 settle 100 ms、释放 settle 500 ms 各记为一个 deadline，由每帧的 step 与帧间约 1 ms 一次的 tick 共同推进。因此定时时相的时长只取决于挂钟，与相机供帧频率解耦；相机偶发丢帧或帧率波动时，刹车与 settle 的时长保持不变。

追球对准 CHASE_BALL 采用固定速度的原地转向对准，按优先级从高到低逐条判定，先成立者即返回。其一，球丢失时朝上一次可见方向以固定速度原地转向搜索，转向由上一帧记下的横向偏移符号播种。其二，球过近（面积占比达到 area_reverse 阈值 0.50）时直线倒车让开，倒车判定的优先级高于抓取，避免贴脸抓空。其三，横向偏移超过对准窗口 20 像素时以固定速度原地转向，把球对到目标列：夹爪在相机光心右侧约 90 像素，故目标列取列坐标 410，即在画面中心 320 之外右移 90 像素。其四，球尚远（面积占比低于 area_stop 阈值 0.28）时直行趋近。其五，当面积占比落在 0.28 与 0.50 之间且偏移进入对准窗口，累计确认帧并输出刹车。追球全程只下

发固定速度的离散可执行指令，此前的比例控制器与失速踢腿动作已删除，以适配只认固定速度的真实底盘。抓取触发经三帧确认防抖，连续三帧成立才启动 `BrakeForGrab` 刹车时相，刹车到期进入 `GRAB`。

抓取与两次脉冲确认 `GRAB` 首次进入下发一次抓取动作并置 100 ms 的 settle deadline。抓取是否成功由两次脉冲阈值确认：机械臂夹爪在贴地夹合与提起两个位姿各回读一次夹爪舵机的脉冲宽度，两次读数均达到 1260 微秒才判为夹住。其原理是球撑住夹爪、使其合不到空爪应有的脉冲值，故夹住时读数偏高。若判为空抓，状态机经 `on_grab_empty` 回到 `CHASE_BALL` 重新搜索，避免空爪去桶。settle 到期后若里程计有效则进入 `RETURN_TO_BUCKET` 盲导回桶，否则进入 `FIND_BUCKET` 靠视觉找桶。

找桶、趋近与投放 `FIND_BUCKET` 原地旋转搜索，以固定速度 12 转向；一旦检测到桶便累计确认帧，连续三帧成立后进入 `APPROACH_BUCKET`。`APPROACH_BUCKET` 是唯一使用比例转向的状态：偏置由桶心的列偏移按增益折算（等效于列偏移除以 16）并钳位到正负 8，死区 15 像素以内不转；前进速度在桶较远时取 25、面积占比达到 0.70 后降到 5 爬行。连续超过 10 帧丢失桶则退回 `FIND_BUCKET`。桶的面积占比达到投放阈值 0.90 时启动 `BrakeForDeposit` 刹车时相，到期进入 `DEPOSIT`。`DEPOSIT` 首次进入下发一次释放动作并置 500 ms 的 settle deadline，到期后令机械臂复位、复位里程计锚点并回到 `CHASE_BALL`，闭合一轮拾球。

6.2 可换真实执行器后端

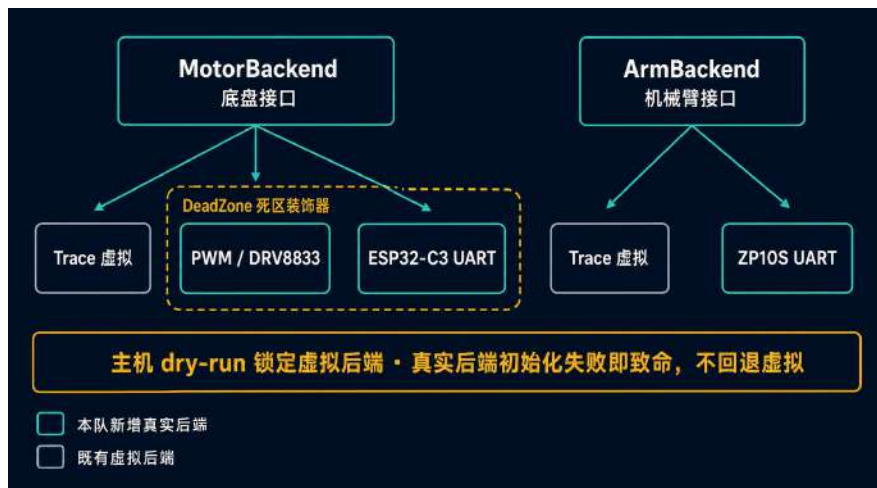


图 8：执行器分层：MotorBackend 与 ArmBackend 接口下挂 Trace、PWM/DRV8833、ESP32-C3 UART 与 ZP10S UART 四个后端，主机 dry-run 锁定虚拟路径，真实后端初始化失败即致命

状态机产生的是抽象指令，底盘一侧为 Drive、Brake、Standby，机械臂一侧为 Grab、Release、Ready；指令的物理实现被收敛到两个接口后面，即 **MotorBackend** 与 **ArmBackend**。一个字符串分派工厂按配置选定后端，共四种：底盘一侧有 Trace 虚拟、PWM/DRV8833 与 ESP32-C3 UART 三种，机械臂一侧有 Trace 虚拟与 ZP10S UART 两种。两种真实电机后端外层统一套一层死区装饰器，把任何非零指令抬升到电机的最小可动速度之上。本体应用此前只有 Trace 虚拟后端，把

指令打成轨迹供主机基准与 CI 使用；现新增上述三个真实后端，从纯虚拟基准变为可驱动真机，同时保留虚拟路径让端到端延迟保持纯计算、可复现。

同一份二进制既能在主机上以虚拟后端做 dry-run，又能在板上驱动真机。真实后端只在板端构建中编入，主机 dry-run 路径被锁定在虚拟后端；工厂在 ready 阶段一旦初始化失败即视为致命并终止本次运行，不回退虚拟后端，以免真机在无人察觉时空爪空转。部署由配置驱动，`--config` 加载器先读配置文件再由命令行覆盖，一份二进制既能在 CI 做 dry-run，又能在板端部署，并对物理上不可执行的速度强校验拒绝。

ESP32-C3 底盘协议 ESP32-C3 底盘走 115200 8N1 的 `/dev/ttyS6`。帧格式为帧头 0xAA、0x55 接命令字、载荷长度、载荷与异或校验，在线长度为 5 加载荷长度，接收侧先扫 0xAA 再扫 0x55 重同步。下发速度用命令 0x13，载荷为一对大端有符号百分比、钳位到正负 100，采用不等应答的即发即弃方式。里程计遥测另走命令 0x20，下位机以两帧 0x90 回报左右轮转速，供下一子节的里程计死推使用。

ZP10S 机械臂协议 ZP10S 机械臂走 115200 的 `/dev/ttyS3`，使用阻塞式 ASCII 舵机协议。移动命令形如 `#000P1611T1000!`，字段依次为舵机号、脉冲宽度与运动时间；位置回读命令形如 `#002PRAD!`，回报的脉冲值正是上一子节抓取确认所依据的量。角度到脉冲的映射为 500 微秒加角度按 270 度满量程折算的分量，钳位到 500 至 2500 微秒。抓取被重写为一段标定位姿序列：张开、伸到位、合爪、回读贴地脉冲、抬起、回读提起脉冲，两次脉冲均达到 1260 微秒阈值才判为夹住。每步之间阻塞约 1 s，据此抓取、释放、复位分别约 5 s、3 s、1 s，这些阻塞落在 dispatch 一侧，不进入纯逻辑的状态推进。

PWM/DRV8833 底盘 PWM/DRV8833 后端经 sysfs 驱动 H 桥，四个 pwmchip 分别对应左右轮的正反向输入，周期固定为 1 kHz，占空比按速度百分比线性折算。刹车令四路全部拉高，使 H 桥两端同高、主动短接电机；standby 令四路全关、任其滑行。右轮固定叠加正负 2 的偏置以补偿两侧电机差异。

6.3 里程计引导回桶

回桶依靠一套差速轮里程计。两轮的 RPM 遥测按差速模型死推位姿，轮半径取 0.03 m、轮距取 0.18 m，积分出机体的平面坐标与航向。投放动作完成时调用 `reset_anchor` 把当前位姿清零，桶就此成为里程计原点；此后 `estimate` 随时给出到锚点的直线距离（坐标的欧氏模）与方位角。里程计默认关闭，仅在板端的 live 配置中开启。

抓球完成后若里程计有效，状态机进入 `RETURN_TO_BUCKET` 盲导回桶：按到锚点方位角与当前航向之差决定动作，航向误差超过 15 度时以固定速度原地转向收敛，误差在容差以内时以固定速度直行趋近，直到进入 0.50 m 的到达半径。回桶全程视觉具有最高优先级，一旦检测到桶立即抢占，转入 `FIND_BUCKET` 交由视觉闭环趋近，桶位因此被记住而无需每轮重新搜索。超时 15 s、里程计失效或推算距离超出 10 m 的合理上限时，同样退回视觉搜索。若抓取完成时里程计无效，状态机直接进入 `FIND_BUCKET` 靠视觉找桶。

为不让遥测阻塞控制，里程计采样被移出控制回路。一次同步的 RPM 读取需约 300 ms（一帧下发加两帧各 150 ms 超时回读），留在控制回路里会卡住电机决策。现改由一个后台线程按固

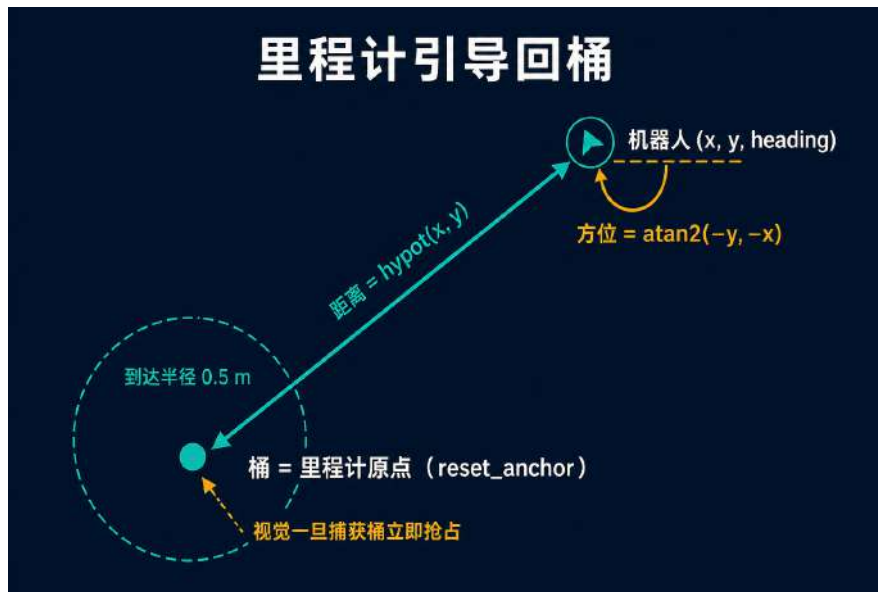


图 9: 里程计引导回桶: 投放时 `reset_anchor` 把桶设为里程计原点, 抓球后按到锚点的距离与方位盲导回桶, 视觉一旦捕获桶立即抢占

定周期 100 ms 读取轮速并更新位姿快照, 控制回路每轮只取一次快照。同时对串口写入加锁、读取只清输入缓冲, 以免遥测误丢尚未发出的驱动帧。慢速遥测与电机决策因此各行其道。

6.4 工程加固与延迟工程



图 10: 机器人感知到控制的闭环: 相机 latest-frame 锁存、感知二选一、状态机到 dispatch 的通路, 无帧时经约 1 ms 的 tick 旁路推进定时时相

现场运行要求这套闭环在异常下安全收场, 本体侧为此叠加了若干加固。执行器接口的每个方法都返回成功与否, 失败沿 `dispatch` 逐层冒泡, 任一执行器调用失败即终止本次运行并令底盘 `standby` 停车。相机侧设两道保护: 启动时的 `warmup` 要求连续三帧结构有效才进入主循环, 超时上限 3000 ms; 运行中的无帧看门狗在 2000 ms 内取不到新帧即停机退出。进程收尾由 `RAII` 接管 `SIGINT` 与 `SIGTERM`, 每条退出路径都先令底盘 `standby`。下发侧对指令去重, 相同的 `Brake`、`Standby` 或相同左右轮速的 `Drive` 不重复发送, 减少串口流量与下位机抖动。推理侧以 `rknn_set_core_mask` 配置 `NPU` 的核掩码启用双核并发, 把检测吞吐摊到两个计算核上 (第三核经消融确

认无增益，详见推理链路一节）。

感知到控制的取帧路径也为低延迟设计。相机采集线程只把最新一帧锁存进单一互斥槽，消费者慢时永远取到最新帧、不积压旧帧。主循环有帧时跑感知加控制，无帧时每约 1 ms 打一次 tick、仅推进定时时相，控制节奏因此不被相机拖住。帧的时间戳在取帧时以单调时钟打点，未采用相机 PTS（该版本采集库不提供该字段），故端到端延迟统计少算了采集线程到取帧之间的排队，板端引用相关延迟时需注明这一口径。

本体侧的功能与加固逻辑均已由主机侧单元测试覆盖：状态机的时相边界精确到毫秒断言，执行器协议经伪终端断言，里程计积分经纯数学断言。就代码结构而言，机械臂的抓取、释放、复位分别由五步、三步、一步逐步阻塞的位姿序列组成，每步之间阻塞约 1 s，据此给出抓取约 5 s、释放约 3 s、复位约 1 s 的阻塞下界。

第三部分 系统层：内核优化与系统能力

7 性能观测 perf

perf 是性能剖析器。它读取 CPU 内建的硬件性能计数器，把周期、指令、缓存访问与分支这些事件归因到具体的函数与代码行，是在真机上定位性能瓶颈的标准手段。Linux 自带 perf; StarryOS 此前没有对应能力，调度与内存的行为只能靠打印与粗粒度计时间接推断，热点落在哪一层无从读出，本报告其余各节所依赖的板级测量也就缺少一件称手的工具。现已在 StarryOS 上实现完整的硬件 PMU perf，覆盖单核 ARM PMUv3 计数、按簇的 big.LITTLE 拓扑、软件与缓存与组事件、带真实进程身份的采样、帧指针与 DWARF 调用栈、kprobe 与 tracepoint 动态跟踪以及 ftrace 函数跟踪，直至未经修改的上游 perf 6.1.0 二进制可直接在 StarryOS 上运行，产出有效的 perf.data、perf report 与 perf top。本节先说明 perf 的硬件基础与各项特性的机制，再说明各项的实现与板级验证。

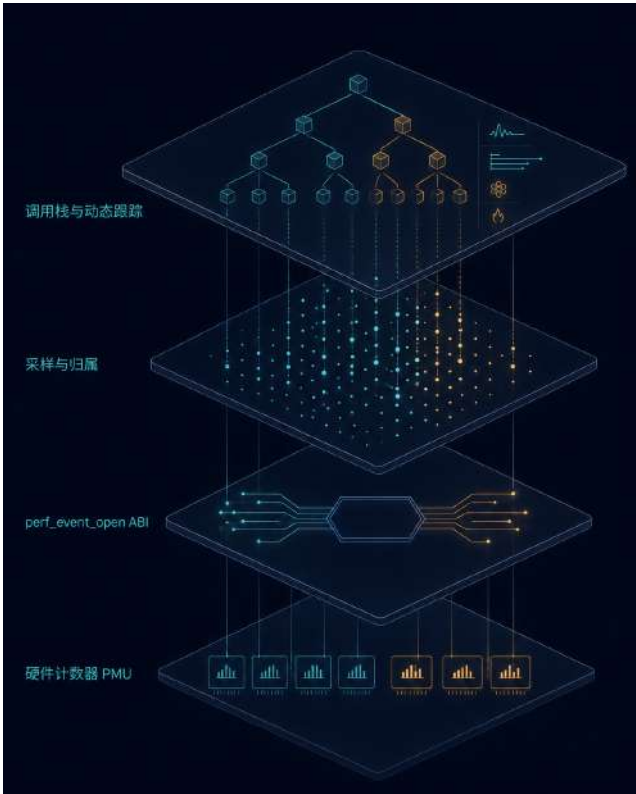


图 11: perf 的分层结构，各层均为新增的内核侧实现

图 11 给出这套 perf 的分层结构。最底层是硬件计数器的编程，往上依次是 perf_event_open ABI 与其 mmap 环形缓冲、采样与归属、调用栈与动态跟踪。其中 perf_event_open ABI 与内核侧的 PMU、采样、跟踪逻辑为本队新增，用户态运行的是未经改动的上游 perf 二进制。

7.1 硬件基础：ARM PMUv3 与硬件计数

硬件计数的载体是 ARM 的性能监控单元 PMUv3。它在每个核上提供一组可编程的事件计数器，外加一个专用的 64 位周期计数器。计数器的数量由 PMCR 的 N 字段给出，RK3588 上 PMCR.N=6，即每核 6 个通用计数器加 1 个专用周期计数器。硬件计数的原理是：把一个待测事件

的编号写入某个通用计数器对应的事件类型寄存器，此后该核每发生一次这类事件，硬件就在对应计数器上自增一次，软件只需在测量前后读取计数器差值，无需任何采样开销即可得到精确的事件总数。周期数走专用的周期计数器，指令、分支、缓存访问等则占用通用计数器。

内核在这一层承担三件事。其一是计数器分配，把用户请求的事件编号绑定到某个空闲的通用计数器，用尽 6 个通用计数器后再打开的事件转入下文的时分复用。其二是用户态与内核态的区分：事件类型寄存器中的 `exclude` 位可将计数限定于用户态或内核态，`perf` 借此只统计被测程序自身而不含内核路径。这条过滤在真机上以极端比值得到验证，同一负载下用户态计数读到 160000055，内核态计数读到 28455，相差约 **5600** 倍，其间一处 `off-by-one` 也在真机上被订正。其三是溢出中断采样：`perf stat` 只需测量前后各读一次计数器，而 `perf record` 需要按事件密度定期采样，做法是给计数器预置一个接近溢出的初值，计数溢出时硬件抬起每核的 PMU 溢出中断（per-PE INTID 23），中断处理记录一次样本并重装初值，采样频率因此与事件发生率成正比。单核阶段还验证了周期计数器为满 64 位、不发生 32 位回绕。这一层以合并请求 #1395 提交至上游，已合并。

7.2 `perf_event_open` ABI 与直接运行上游 `perf`

`perf` 的用户态接口是 `perf_event_open` 系统调用与其配套的 `mmap` 环形缓冲。用户态用 `perf_event_open` 传入一份事件描述（事件类型、编号、过滤位、采样周期等），拿回一个文件描述符，再 `mmap` 出一段环形缓冲；内核把采样记录连续写入这段缓冲，用户态顺序读出并解码。符号化不落在内核，用户态凭 `/proc/kallsyms` 把内核地址还原为函数名，凭 `/proc/<pid>/maps` 把用户地址还原为可执行文件与偏移。这一层为本队新增，也是整项工作的核心结论所在：StarryOS 的 `perf_event_open` ABI 与 Linux 对齐到足以让未经改动的上游静态 `musl` 版 `perf 6.1.0` 二进制直接运行，无需为 StarryOS 定制一份 `perf`。该二进制在 CI 中由 `linux-6.1` 源码可复现地重建，其内置的 `tools/lib/traceevent` 静态链入，因而 `perf report` 与 `perf script` 能就地解码跟踪数据，剥离符号后约 2.78 MB。

直接运行上游二进制的端到端结果如下。`perf stat true` 在真机上计出约 19.3M 周期、IPC 0.91（QEMU 因只建模周期一项计出 74027146 周期）。`perf record` 采得 751 个样本、产出 0.031 MB 的 `perf.data`；`perf report` 将内核热点逐行符号化，给出 `UserContext::run` 占 7.73%、`memcpy` 占 5.57%、`on_exec_sideband` 占 3.02%，符号经 `/proc/kallsyms` 解析。`perf top --stdio` 在 4 核上以 `drop: 0/10115` 无丢样滚动，首行为 100.00% [kernel] [k] `ax_task::run_idle`。ABI 对齐使工程量显著下降，`perf` 全部功能的验证都可直接以真实 `perf` 的输出为准。

7.3 多核与大小核

RK3588 是大小核架构，4 个 A55 小核（`cpu0` 至 `cpu3`）与 4 个 A76 大核（`cpu4` 至 `cpu7`）分属两簇，PMU 计数器为每核私有，跨核测量必须逐核编程。内核为每个核经 `percpu::ALLOC` 维护独立的计数器池，并以 `ensure_core_initd()` 在首次触及某核时惰性初始化其池。两簇的事件编码并不一致，同一事件在 A55 上存在而在 A76 上可能缺失，内核据 MIDR 识别核心身份加以区分：A55 的 `part` 号为 0xd05、归为 `type 9`，A76 的 `part` 号为 0xd0b、归为 `type 10`，事件的支持性检查因此按打开事件所在核所属的簇来判定。

系统级测量 `perf stat -a` 需要同时在所有在线核上计数，内核经核间中断 (IPI) 把编程与读取请求按核扇出，每个核在本地完成计数器操作后回汇。每个事件绑定一个 home core，其计数器的编程与读取都在该核串行完成 (home-core lock)，跨核读取经 IPI 转发到 home core，避免两核并发触碰同一计数器。当某核上打开的事件数超过 6 个通用计数器时，内核在调度 tick 上按时间片轮转这些事件到有限的物理计数器，读数再按占用时间比例放大，得到与全程独占等效的估计，这与 Linux 的计数器复用一致。该层还实现了 `PERF_FORMAT_LOST`，把复用与缓冲写满造成的丢失计入读数。这部分以合并请求 #1577 提交至上游，在审。

7.4 事件类型

`perf` 把可测事件按来源分为若干类，各类的取值口径与开销不同，以下逐类说明其机制与实现。

硬件事件 硬件事件直接落到 PMUv3 的通用计数器，包括周期、指令、分支、各级缓存访问与命中等。它们由硬件精确计数，开销近乎为零，是 `perf stat` 与 `perf record` 的默认事件来源，机制见前文硬件基础一节。

软件事件 软件事件不占用硬件计数器，由内核在关键路径上以纯原子操作按任务累加，包括 `cpu-clock`、`task-clock`、`context-switches`、`cpu-migrations` 与 `page-faults`。`cpu-clock` 计逝去的挂钟时间，`task-clock` 只计任务实际在核上运行的时间，故 `task-clock` 恒不超过 `cpu-clock`；`context-switches` 计上下文切换次数，`cpu-migrations` 计任务跨核迁移次数，`page-faults` 计缺页次数。实现落在新增的 `perf/sw.rs`，每个软件事件是一个每任务的原子量，在切换、迁移、缺页等钩子处自增。一次 QEMU 测量给出 `cpu-clock` 247 ms、`task-clock` 173 ms (小于 `cpu-clock`)、`page-faults` 513、`context-switches` 24、`cpu-migrations` 可读，语义自治。软件事件在无 PMU 或 PMU 被占满时仍可用，是纯计算负载画像的基础。这部分随合并请求 #1601 提交至上游，在审。

缓存事件 `PERF_TYPE_HW_CACHE` 以三元组编码一个缓存事件：缓存层级 (L1D、L1I、LL、TLB 等)、访问类型 (读、写、预取) 与结果 (访问、缺失)。内核以 `hw_cache_to_arm(config)` 把这个组合映射到 PMUv3 上对应的原始事件编号，无对应硬件事件的组合如实报告为不支持。一组 20 项组合的解码单元测试通过，L1D-prefetch 这类无对应事件的组合被干净拒绝而不误路由。真机上 CACHE-2 的 L1D-load 计出 40524820 (大于零)，缓存计数只能在真机上取得，QEMU 的 PMU 仅建模周期一项。这部分同随合并请求 #1601 提交至上游，在审。

事件组 事件组把多个事件绑定为一组，保证它们被同时调度上同一组计数器，其读数才可在同一时间窗内相互换算，如以 `instructions` 除以 `cycles` 求 IPC。组的成员经 `group_fd` 指向组长建立，`PERF_FORMAT_GROUP` 令一次 `read` 原子地取回全组读数，`PERF_SAMPLE_READ` 令组长的每个样本携带全组计数，组长采样时即可把成员计数一并快照。一次 QEMU 软件组测量得 `nr=2` 的 `{task-clock,cpu-clock}`，各成员按启用传播计数、id 正确解复用；采样读的样本值严格递增 (819 个样本、末值 81900000，即 819 乘 100000)。组长采样在一处跨线程用后释放 (`use-after-free`) 上曾有隐患：组长把成员的每任务原子量裸指针烘入自己的每核样本槽，成员线程若退出并释放这些原子量，组长的硬中断处理仍会解引用。该处以一道同线程不变式 (`owner_tid` 同线程门，跨

线程成员返回 `EINVAL`) 封堵, 对齐 Linux 中组长与成员须同属一个上下文的约束。事件组随合并请求 #1603 提交至上游, 在审。

7.5 采样与归属

采样记录必须归属到正确的进程与线程, 否则 `perf report` 的按进程聚合便失真。采样在硬中断上下文中完成, 此时 `current()` 未必仍是被采样的任务, 尤其在多核抢占下。为此内核在把某任务的时间片装载上核 (slice-arm) 时就记下它真实的进程号与线程号 (`tgid,tid`), 样本据此归属, 跨核采样的归属因此准确。真机上 TID-1 以 1024 个样本验证, 每个样本的 `tid` 与 `tgid` 均正确 (`pid=47`、`worker_tid=158`)。

环形缓冲写满时新样本会丢失, 若不记录丢失量, 用户态无从判断画像是否被截断。内核以带内的 `PERF_RECORD_LOST` 记录在丢失发生处补入丢样计数, 与后续样本一同顺序读出。真机上 LOST-1 得 2033 个有效样本、15 条 LOST 记录、累计丢失 3416426, 丢样在压力下被如实计入而不静默。这部分随合并请求 #1602 提交至上游, 在审。

7.6 调用栈采样

只知道热点函数不足以定位问题, 还需知道它经由哪条调用链被触及, 即调用栈采样 (`PERF_SAMPLE_CALLCHAIN`), 其可视化即火焰图。内核实现两条互补的展开路径。

第一条是帧指针路径。它是一个无分配的展开器, 从中断入口发布的陷入帧取出帧指针 `FP` 与栈指针 `SP`, 沿栈上串起的 `FP` 链逐帧上行读回各级返回地址, 全程不申请内存, 可安全运行于硬中断上下文。`FP` 与 `SP` 由 `axcpu` 在 `IRQ` 入口发布到每核槽位供展开器读取。裸解引用内核 `FP` 在硬中断中有风险, 一个落在合法范围却已失效的 `FP` 会触发不可收场的异常, 故内核 `FP` 的读取改走一条 `IRQ` 安全、不触发缺页的四级页表走查, 并以物理内存范围核验拦住指向用户映射的 `RGA`、`NPU`、`JPU` 寄存器的伪地址, 另设总步数上限防止畸形链导致的耗时失控。QEMU 上该路径可展开 8 层用户帧。用户帧展开有两处前置: 用户测试须以 `-fno-optimize-sibling-calls` 编译, 否则 `-O2` 下尾调用链会塌成 3 帧; 深层内核展开须用带 `BACKTRACE=y` 的帧指针内核。真机上 CHAIN-USER 得 580 个样本、最深 7 层用户帧 (至少 4 层 `FP` 帧), CHAIN-KERNEL 得 583 个样本、577 条内核链。

第二条是 `DWARF` 路径。当目标未保留帧指针时改由 `DWARF` 展开: 采样点采集一组寄存器与一段栈快照, 随样本带回, 交由用户态依据 `DWARF` 调试信息离线展开出完整调用栈。真机上 DWARF-1 得 1203 个样本, 每样本 6 个用户寄存器、6 个有效 (寄存器与栈均已采得)。两条路径合起来覆盖有无帧指针两种情形, 与 Linux 一致。

7.7 动态跟踪: `kprobe` 与 `tracepoint`

除按周期采样外, `perf` 还能在特定内核事件上打点。`kprobe` 是内核动态探针, 在运行时把内核任意函数入口处的指令替换为断点, 命中时陷入并执行探针回调, 无需重编译内核即可观察某函数被调用的时刻与参数; `tracepoint` 则是内核源码中预埋的静态跟踪点, 开销更低但位置固定。两者命中时都直接发出一条 `PERF_RECORD_SAMPLE`, 从而复用 `perf` 既有的采样、归属与符号化通路。对外, `perf probe` 与 `kprobe_events` 经 `tracefs` 暴露, 用户可在真机上按 `/proc/kallsyms` 的 `_stext+offset` 动态注册探针。

动态跟踪的一处关键在于过滤范围。早期未加过滤时，探针会把内核自身处理探针所触发的路径也一并采到，形成正反馈，一次 `perf record -e probe: -- cmd` 耗时达 233 s。引入每任务过滤只对被测命令采样后，同一用例 284 个样本在 12 s 内取毕（11.97 s、284 个样本），`perf report` 与 `perf script` 均返回 0（报告为 `probe:hsc` 事件的 284 个样本，脚本 284 行）。真机上 KPROBE-1 以 `/proc/kallsyms` 定位探针，命中 501 次并正确移除。这部分连同 `tracepoint` 采样一并实现，探针命中经 `perf` 采样通路产出样本。

7.8 ftrace 函数跟踪

`ftrace` 是内核的函数跟踪器，它借编译期在每个函数入口埋下的可打补丁桩，在运行时改写这些桩以记录函数进入，本内核共有 11512 个可打补丁入口。全量跟踪每个函数开销极大，实用形态是过滤跟踪，即以 `set_ftrace_filter` 只保留少数感兴趣的函数，例如只跟 `handle_syscall` 时得到 73 条记录，这也是 Linux 上的常用形态。

全量函数跟踪暴露出一个只在真机 8 核上出现的活锁，QEMU 始终未能复现，因为 `ftrace` 的测试都跑在单核，而全量跟踪在模拟器下又过慢、无法在超时内跑完。定位时先排除了两个错误假设（通知风暴、`stop_machine` 下的巨型 `protect`），`gdb` 抓到的现场显示四个核心全部停在 EL1 同步异常向量上，是一场递归异常风暴：每核的可重入保护本应在进入插桩回调时立即置位以阻断自触发，但它被安排在若干本身也已被插桩的、非内联的辅助函数返回之后才置位，这些辅助函数在保护尚未置位时被调用便再次触发插桩，层层嵌套直至死锁。修复方法是在回调序言最前处先以不带插桩的原子操作置位保护，随后在真机 8 核上确认置位即返回、不再卡死。过滤式函数跟踪在 QEMU 与真机上都实用可靠。

7.9 板级验证

`perf` 子系统在 OrangePi-5-Plus 上做了两套硬件在环验证，验证脚本即机器可判定的判定门，一次运行产出一个可比对的通过计数。第一套是 `big.LITTLE` PMU 拓扑矩阵，最终 39 项通过、0 失败，两簇各自独立编程（A55 type 9、A76 type 10）、跨簇迁移、每核溢出采样与 `rdpmc` 均通过。单核阶段的 18 项锚定了真机 A55 上的计数通路：真实 `PMCR.N=6`、用户态与内核态计数约 5600 倍之差、复用轮转、64 位周期计数器无回绕。第二套是完整的 `perf_event_open` 特性矩阵，56 项通过、0 失败，覆盖软件事件、缓存计数、丢样记录、真实 `tid` 归属、事件组与组采样读、系统级采样与旁带、帧指针与 DWARF 调用栈以及 `kprobe`。上游 `perf` 二进制直接测得 IPC 0.91。缓存计数、`tid` 归属、组调度与 `kprobe` 命中只能在真机上取得，QEMU 的 PMU 仅建模周期一项。

单核阶段的一项测量与直觉相反。压测所用的循环是一条内存串行的依赖链，A55 在这类负载上的 IPC 读到 1.60，高于 A76 的 1.30，顺序发射的小核在缺乏指令级并行时可以追平乱序的大核，A76 的 IPC 高于 A55 的假设只在指令级并行充足的负载上成立。核对后确认这是真实的硬件结果，遂如实记录，并把对应用例列为信息项。

7.10 硬件受限特性

一批 `perf` 特性受 RK3588 硬件限制无法提供，均按 Linux 在同一 SoC 上的行为如实拒绝，未伪造读数。分支栈采样 `PERF_SAMPLE_BRANCH_STACK` 与 `perf record -b` 依赖 BRBE 分支记录

扩展, 该 SoC 未实现。perf mem 与 perf c2c 依赖 PEBS 类的带权重与数据源的采样 (WEIGHT、DATA_SRC), 该 SoC 不具备。硬件断点的 mem: 事件因未接入调试异常向量而不提供, ref-cycles 因无真正的 ARM 参考周期事件而不提供。这些拒绝与 Linux 自身在这颗 SoC 上的不支持行为一致, perf 达到的是 RK3588 硬件实际支持范围内与 Linux 的功能对等。

7.11 板级运行中发现的 axtask 跨核互唤醒死锁

在这些板上跑 perf 期间, 还发现并修复了 axtask 调度器的一处跨核互唤醒死锁。追查一个疑似 smp8 USB IRQ 风暴的现象时, 一步对抗式验证先否定了该前提 (真机上只有 12 次良性的 USB 枚举中断, 启动照常完成), 进而定位到真正的故障: put_task_with_state 以 while task.on_cpu() { spin_loop() } 忙等一个远端核把任务切出, 而此时它正持有运行队列锁且关中断, 两个核同时这样做即冻结整机。修复删去自旋, 改为把入队推迟给拥有该任务的那个核, 并以一次 SeqCst 的 Dekker 握手保证恰有一侧入队, 不丢唤醒也不重复唤醒 (提交 611529498, 合并请求 #1495, 已合并)。该死锁在 QEMU 上无法复现 (QEMU 命中 0 次、真机 3 次里冻结 1 次), 是一处只在真机上暴露的问题, 这类问题正是构建 perf 的动因之一。此修复是负载均衡器的前置条件, 其窃取与唤醒路径都经由同一函数, 详见调度一节。

8 调度与大小核

调度器决定每个可运行线程在何时、在哪个核上运行; 在多核系统上, 一个线程被放到哪条运行队列, 就决定它在哪个核上执行。StarryOS 的 axtask 此前不做跨核放置: 新建的线程固定在派生它的核, 被唤醒的线程回到唤醒者所在的核, 运行期也没有负载均衡。RK3588 由 4 个 A55 小核与 4 个 A76 大核组成, 整机八核的算力因此始终压在启动所在的单个小核上, sysbench cpu 无论开多少线程都停在 160 ev/s 附近, 八线程落后同机 Linux 约三十倍。现已在 axtask 上补齐跨核派生分发、占用感知的派生与唤醒放置、Linux-CFS 式的 wake_affine 就近唤醒, 并为 CLONE_VM 线程负载补上一条无锁的用户拷贝快路径; 八线程 sysbench cpu 升到 **4987 ev/s**, 为 **Linux 5222 的 0.96 倍**, 单核算力与 Linux 对等, 核间驻留均匀。本节先说明调度器与运行队列的放置机制、以及大小核给放置带来的约束, 再逐一说明四层新增机制的原理与实现, 最后给出对等阶梯、核驻留与单核算力的板级结果。

8.1 调度器与运行队列

调度器的职责是把有限的 CPU 时间分配给众多可运行线程。axtask 为每个 CPU 维护一条独立的运行队列: 核空闲时从本核队列按调度策略取下一个线程执行, 线程时间片耗尽或主动阻塞时再重新入队。运行队列为每核私有, 一个线程当前排在哪条队列上, 就决定它在哪个核上运行。把线程指派到某条运行队列的这一步称为放置。

放置发生在两个时刻, 各对应 run_queue.rs 里的一处决策。其一是派生放置: fork / clone 新建一个线程时, 须为它选一条初始运行队列 (select_run_queue)。其二是唤醒放置: 一个阻塞在某事件上的线程被唤醒、重新变为可运行时, 须为它选一条运行队列重新入队 (select_wake_run_queue)。这两处决策共同决定负载在各核之间如何分布: 派生放置铺开新建的工作线程, 唤醒放置决定周期性阻塞与唤醒的线程醒来后落在哪。放置一旦失衡, 即便有空闲核也无线程可取, 整机吞吐会塌到被占满的少数核的算力上。

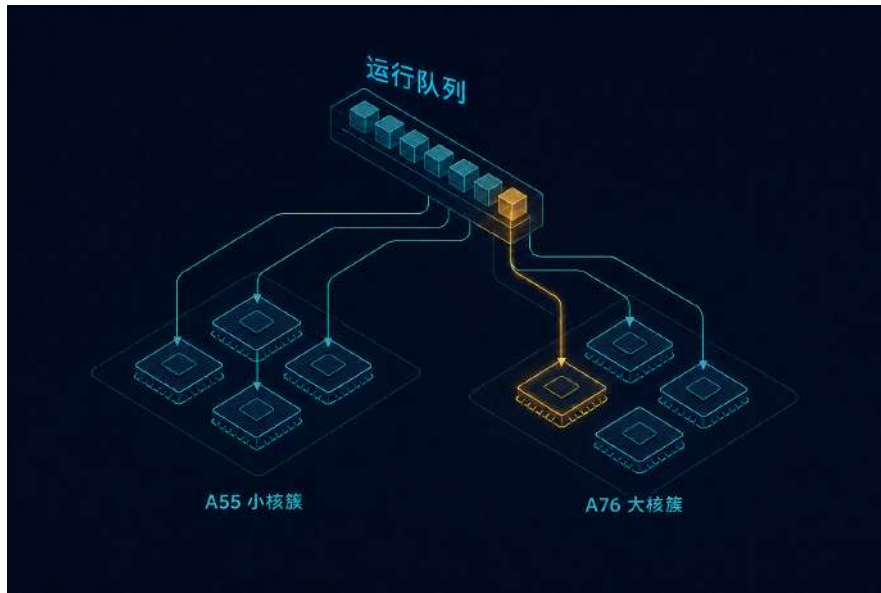


图 12: axtask 放置管线，青色为新增实现；顶部为待放置线程，各核持私有运行队列。派生分发在亲和性允许的核间轮转，单原子 occ 占用计数供派生与唤醒共用并每 tick 自愈，wake_affine 在唤醒者空闲时把被唤醒者就近交回，共同把负载在大小核间铺开

8.2 大小核与既有 axtask 的放置

RK3588 采用大小核结构，8 个核分属两簇：cpu0 至 cpu3 为 A55 小核，cpu4 至 cpu7 为 A76 大核，A76 的每周期吞吐与频率上限均高于 A55。启动核为 cpu0，属 A55 小核簇。异构给放置多加一层约束：同一线程放在 A76 与放在 A55 上吞吐不同，只有把可并行的线程铺满全部八核、并让繁重线程优先落在大核，才能发挥整机算力。

既有 axtask 在两处放置点都不跨核迁移，运行期也不做负载均衡，三点叠加使整机始终压在单个核上。

派生不跨核 派生放置把新线程固定在执行 clone 的那个核上。fork / clone 几乎总在启动核 cpu0 上发起，新建线程因而全部堆在 cpu0 这一个 A55 小核。

唤醒回汇聚 唤醒放置把被唤醒者交回唤醒者所在的核。栅栏同步的一组工作线程由同一个释放者唤醒，醒来后重新汇聚回释放者的核，散不开。

无负载均衡 运行期没有任何均衡逻辑。一条队列排满、其余七条全空时，没有机制把线程迁到空闲核。

三点叠加的结果是整机只在单个小核上排队，smp8 与 smp4 的读数逐字节相同，sysbench cpu 无论线程数多少都停在 160 ev/s 附近，八线程落后同机 Linux 约三十倍。这一差距纯由放置造成：一组强制每核各钉一份任务的对照读到接近八核理论上限的吞吐，确认单核算力本身并无问题，缺的是把线程铺开的放置逻辑。现已在这条放置路径上新增四层机制，自派生到唤醒依次生效，其结构与落点见图 12。

8.3 跨核派生与唤醒分发 (#1656)

新增的第一层是跨核派生与唤醒分发，提交为 #1656。派生时不再固定到启动核，而在亲和性允许的核之间轮转分发；唤醒时优先选线程上一次运行的核，保住缓存热度，不再一律交回唤醒者。轮转须只落在已上线的运行队列上，为此新增一个 `RUN_QUEUE_ONLINE` 位图门控，避免早期启动阶段把线程分派到尚未初始化的次核队列而触发 `DataAbort`。派生分发与唤醒改选两半缺一不可：仅做派生轮转时，栅栏唤醒的工作线程仍会被重新汇聚回唤醒者。

轮转把真并发暴露出来后，几处此前被启动核串行化所掩盖的缺陷随之显现，均按 Linux 语义修正并随 #1656 一并验证。一处是 SMP 下的 `ptrace` 缺陷：`wait_ptrace_resume` 会因一次善意的跨核 `interrupt()` 放弃 `trace-stop`，使刚执行 `TRACEME` 的子进程越过 `raise(SIGSTOP)` 继续运行。一处是 `riscv64` 的远程 `kick` 合并竞态：`REMOTE_RESCHEDULE_PENDING` 会压制掉排在在途 `reschedule` 之后落核任务的 IPI，把它搁在无 `tick` 的 `idle` 上，以 `force_kick_remote_cpu` 补发 IPI 修复。一处是 `pivot_root` 后关机时的卸载 `panic: init` 的 `unmount_all` 在共享挂载命名空间未静默时 `panic`，改为尽力卸载，并增设一把对应 `Linux namespace_sem` 的全局 `MOUNT_OP_LOCK`。此外补上一处 POSIX 亲和性缺口：`do_clone` 从不复制父线程的 `cpumask`，使一个 `taskset -c 0` 限定的 `sysbench` 仍可能运行到 A76 上，修正为在克隆时令子线程继承父线程的亲和性掩码。

跨核分发把上板 `sysbench cpu` 从平线 160 抬到八线程 3668 `ev/s`，确认此前的差距纯粹来自放置。

8.4 占用感知的放置

派生轮转把线程散开，但在异构核上仍散得不匀，因为轮转看的是队列长度 `nr_running`，而它不计入当前正在运行的任务。一个 A76 正跑着一个计算线程、队列却为空时，`nr_running` 读作 0，这个繁忙的大核被当成空闲，连续派生的兄弟线程于是全部压到它上面，形成聚集。

准确的占用需要把正在运行的任务也计入。一个直接的做法是在 `nr_running` 之外再加一个 `RQ_RUNNING` 标志、两者相加得占用；但这两个量无法被远端放置者一致地读到，属 `seqlock` 类竞态：运行态转空闲的切换从不触碰 `nr_running`，那次置 `false` 的存储没有配对的 `Release`，一次带序存储的尝试因此反而回归并被回退。最终以单个每核 `occ` 原子计数解决：`occ` 仅在线程进入和离开运行态时（在 `switch_to` 内）增减，把正在运行的任务计入负载，远端放置者只读一个原子量即可得到一致的占用。

发行版构建会移除针对 `occ` 下溢的 `debug_assert`，使 `occ` 在长时间运行后缓慢上漂，大核被误判为繁忙。为此在调度 `tick` 上加一层自愈：`scheduler_timer_tick` 每约 10 ms 重算一次占用：取 `nr_running` 与正在运行的非 `idle` 任务数之和，并配一个饱和递减，把漂移校正回真实占用。

唤醒放置同样接入 `occ`，对齐 Linux 的 `select_idle_sibling`：仅当目标核空闲 (`occ==0`) 时保留线程上一次运行的核，否则改选最空闲的核 (`select_least_loaded`)。此前唤醒把被唤醒者固定回 `last_cpu` 或唤醒者，栅栏释放的一阵唤醒会把八个线程聚到三个大核上，接入占用信号后八个线程才在全部八核上摊开。占用感知的派生与唤醒放置在大核频率提升带来的 3991 `ev/s` 基础上把八线程 `sysbench cpu` 推进到 4987 `ev/s`，并使核间驻留由集中转为均匀。

8.5 完整工作窃取均衡器的反面结论

在 #1656 基础上另前向移植过一套完整的容量感知工作窃取均衡器，含空闲核 idle-pull、繁忙核 push 与唤醒重定向，并从设备树读取每核容量（capacity-dmips-mhz，A76 为 1024、A55 为 530）按容量折算负载。主机测试全绿，但在板上它使目标负载回归：单线程吞吐由 350 压到 161，饱和时并无收益。其机制为迁移抖动，七个空闲核把唯一的工作线程反复弹移，无一核心得以保持缓存热度。配套移植的一次 Linux 式 nr_balance_failed 升级会让空闲 A55 抢走热 A76 上的任务，同样回退。运行期迁移（idle_pull_once / try_push_balance）因此仅以特性门控保留、默认关闭，出厂默认只采用容量感知的派生与唤醒放置这一安全子集。

8.6 wake_affine 就近唤醒

清除分页相关的 EFAULT（详见内存一节）后，整机 schbench 暴露出约 1 ms 的跨核唤醒下限：m1t4 的 wakeup p50 约 1042 μ s，同机 Linux 仅 6 μ s。

一次跨核唤醒的开销主要在处理器间中断投递。线程 A 在核 X 上唤醒线程 B、而 B 被放到另一个核 Y 时，核 X 须向核 Y 发一次 reschedule 的 GIC SGI（一种处理器间中断）通知它有新线程可运行，核 Y 收到后才切入 B。wake_affine 针对生产者唤醒单个消费者这类同步唤醒：当唤醒者近乎空闲（occ $\leq$ 1）时，把被唤醒者交回唤醒者所在的核，就近运行，省去这次跨核 SGI，且共享缓存仍热。这一层对齐 Linux CFS 的 wake_affine，默认启用；它把 m1t4 的 wakeup p50 由约 1042 μ s 降到 8 μ s，接近 Linux 的 6 μ s，判定持平。至于栅栏同步的一组并发工作线程被同一释放者唤醒的情形，就近交回并不适用，留待下文的唤醒铺开处理。

这约 1 ms 全部来自 IPI 投递本身。带内分类计时显示本地唤醒 p50 为 2 μ s，跨核项约 1048 μ s，而中断处理到切入仅 2 μ s，且 87% 的 reschedule SGI 指向真正处于 WFI 的核心。RK3588 的 reschedule GIC SGI 无法及时唤醒处于 WFI 的空闲核，该核要等自身约 1 到 2 ms 的 oneshot 定时器到期才前进。wake_affine 以就近唤醒绕开这条慢投递。对确需跨核唤醒的情形，idle-poll（WFI 前自旋约 50 μ s）把跨核唤醒中位数压低约 250 倍，下文的唤醒铺开正依赖这一点。该下限属固件与硬件层面的外因，软件仅能缓解，无从消除。

8.7 并发工作线程的唤醒铺开

wake_affine 的就近唤醒适合生产者唤醒单个消费者，对栅栏释放的一组并发工作线程却适得其反：它们由同一个释放者唤醒，就近交回会把整组重新聚拢到释放者的核，抵消派生放置本已把它们铺开的效果。这类聚拢在两线程 mem_bw2 与 schbench 上显现，一组本应各占一个大核的工作线程挤在同一个核上。直接套用 Linux 的 wake_affine_idle 加 select_idle_sibling 把每次唤醒都改为铺开，在缺 idle-poll 时反而更慢，因每次铺开都要支付前述约 1 ms 的跨核 SGI 代价；以固定时长自旋的 idle-poll 抹平这笔代价之后，铺开转为净收益。出厂放置配置据此一并启用 idle-poll，并以按线程 CPU 占用的 EWMA 区分两类唤醒：判为 CPU 密集的并发工作线程无条件经 select_idle_sibling 铺开到空闲兄弟核，生产者消费者仍走 wake_affine 就近；两者都在唤醒时决定落核，仍属唤醒放置的安全子集，不涉及前述已回退的运行期迁移。板级 A/B 中，schbench m1t4 的 RPS 由约 231 抬到 339，为同机 Linux 约 199 的 1.71 倍；两线程 mem_bw2（2 个 A76）由 0.34 抬到 0.78 倍 Linux，栅栏铺开由单核扩到全八核；sysbench mutex 八线程的墙钟由 1.69 s 降到约 0.85 s，对 Linux 的比值为 1.76 倍、残差在锁路径本身（详

见后文残余差距)。上述数值取自 2026-08-12 板级消融的三次重复中位数。

8.8 -T 线程负载的用户拷贝快路径

hackbench 的 -T (CLONE_VM 线程) 此前比 -P (fork 进程) 慢 3.6 到 4.2 倍, 方向与 Linux 相反。根因是一把共享、独占且会睡眠的 `Arc<Mutex<AddrSpace>>`: 每一次用户内存拷贝前的地址校验都要锁住整个地址空间, 且这把锁的持有跨越一次页表走查。CLONE_VM 线程共享同一个地址空间, 全部在这把锁上串行; fork 出的进程各自持有私有地址空间, 互不相干, 故 -T 慢而 -P 快。

新增的快路径对齐 Linux 的 `access_ok` 模型, 在取锁之前先做一次无锁的硬件页表探测。`access_ok` 的职责是在内核拷贝用户内存前确认目标地址在当前进程地址空间内可读或可写, 既有实现靠锁住地址空间并软件走查页表来判定。aarch64 提供地址翻译指令 `AT: AT S1E0R` 让硬件按 stage-1、EL0、读权限对一个虚拟地址做一次翻译与权限检查, 把结果写入 `PAR_EL1`, 其 F 位 (bit 0) 为 0 表示该地址在 EL0 可读、为 1 表示不可达; `AT S1E0W` 同理判可写。发出 AT 后以 `isb` 等翻译完成, 再读 `PAR_EL1`。`components/axcpu` 据此实现 `user_access_ok_page`: 关中断下对每个待访问页发一次 AT 探测 (上限 `FASTPATH_MAX_PAGES=16` 页), 全部可达则跳过取锁直接拷贝, 任一不可达再回落到取锁的慢路径, 接入 `check_region` 与 `prepare_user_memory`。前提均经核实: PAN 关闭 (`SCTLR_EL1.SPAN=1`) 使 EL1 得以按 EL0 权限访问用户地址, TBI0 关闭, COW 破坏集与 AT 接受集证明不相交。一次对抗审查发现一个校验绕过: 一个会回绕的 `UserPtr` 在 `page_start>page_end` 时跳过探测循环直接返回可达, 特性开启时构成校验绕过; 以 `checked_add` 修正后复核确认无绕过。

板级 A/B (同一 HEAD, 仅特性开关不同) 中, hackbench -T **g10** 提升 **10.2** 倍, -T 对 -P 的比值由 3.6 到 4.2 倍收敛到约 1, 恢复 Linux 的形态, -P 自身也加快 1.5 到 2 倍, 已并入出厂配置。另有一处 COW 引用计数溢出导致的 fork EFAULT 缺陷, 其修复详见内存一节。

8.9 对等阶梯与核驻留

固定频率隔离出放置各层的净贡献, 构成 sysbench cpu 的对等阶梯: 单核地板 (放置关闭) 约 361 ev/s, 补齐跨核分发即跃至约 4987 ev/s (为 Linux 的 0.96 倍, 放置是决定性的一层); 占用感知放置与出厂配置 (含唤醒铺开) 在此量级上持平, 分别约 4995 与 4987 ev/s, 为 Linux 5222 的 0.96 倍, 判定持平。`wake_affine` 单独的就近唤醒会把八线程回落到约 3523 ev/s (为压低唤醒延迟所必需), 唤醒铺开随即将其恢复。

放置均衡前, 八线程会塌陷到少数大核。此前 `/proc/stat` 的每核驻留是聚合值除以核数的虚构读数, 无法显出失衡; 将其改为真实的每核忙碌 `tick` (经 `ax_task::cpu_busy_ticks` 暴露) 后, 一次八线程测量给出 [cpu0 83, cpu1 0, cpu2 0, cpu3 0, cpu4 40, cpu5 2071, cpu6 1560, cpu7 851], A55 集群近乎空闲, 八个线程集中在三个 A76 上, 据此确认症结在聚集, 空闲的 A55 本可分担却无线程落入。唤醒接入占用信号后, 同一测量读到均匀驻留 [951, 988, 959, 968, 944, 954, 929, 891]。

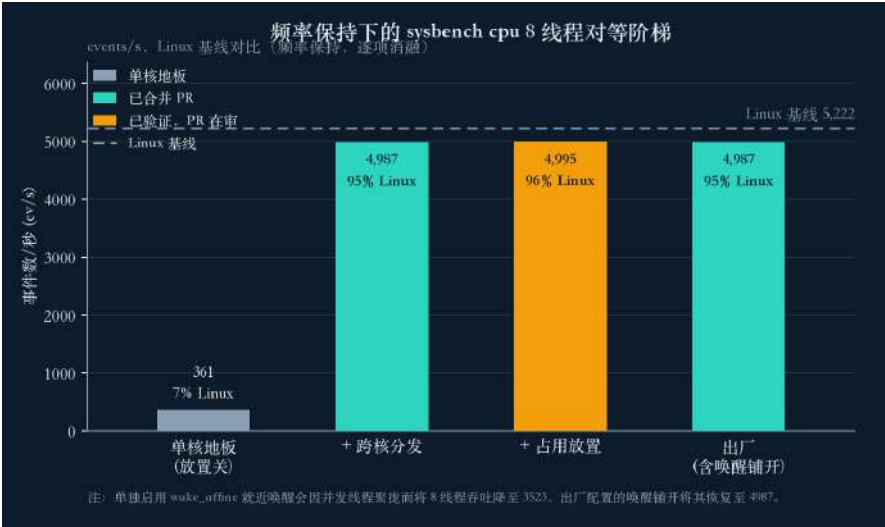


图 13: sysbench cpu 对等阶梯 (频率固定)。单核地板 (放置关闭) 约 361 ev/s, 补齐跨核分发跃至约 4987 ev/s, 占用感知放置与出厂配置在此量级持平, 分别约 4995 与 4987 ev/s, 为 Linux 5222 的 0.96 倍



图 14: 核驻留直方图。放置均衡前八个线程集中在三个 A76 核、A55 集群近乎空闲, 唤醒接入占用信号后在 A55 与 A76 八核间均匀铺开

8.10 单核算力与残余差距

把聚合读数拆分到单核，算力已然对齐。A55 单核为 359 ev/s，Linux 为 349，比值 1.03 倍，小幅领先；A76 单核为 896 ev/s，Linux 为 962，比值 0.93 倍，持平；综合每核约为 **Linux** 的 **0.98** 倍。八线程聚合 4987 ev/s 为 Linux 5222 的 0.96 倍，判定持平。

口径	StarryOS	Linux	比值	判定
单核 A55 (ev/s)	359	349	1.03×	小幅领先
单核 A76 (ev/s)	896	962	0.93×	持平
cpu 8 线程 (ev/s)	4987	5222	0.96×	持平

表 3: 每核与八线程 sysbench cpu 对等，p50 取 $N \geq 5$

其距满对等的残余来自频率上限：大核 DVFS 提升到 2126 MHz（上探 2256 MHz）、小核 1881 MHz 后，仍受 BL31 安全 CRU 与温控缺失所限，为外因所致，详见频率一节。锁竞争是后续继续优化的方向：放置与频率均已对齐，mutex 与 futex 的残差集中在锁自身的路径。总体而言，调度在结构与单核算力上与 Linux 对等，放置、跨核唤醒延迟与 -T 扩展均已恢复到 Linux 的形态。

9 系统调用入口开销

系统调用入口开销指用户态陷入内核、执行完毕再返回这条路径上，每次调用都要支付的固定成本，与调用具体做什么、是否触及调度或内存无关。一次系统调用无论是读一个字节还是取一个进程号，都要走同一段陷入与返回：用户态执行 SVC 指令触发同步异常陷入 EL1，内核保存现场、按调用号分发到处理函数、处理函数返回后再恢复现场、eret 返回用户态。这段路径的耗时是所有系统调用共同的底座，调用体本身越轻，它在总耗时中的占比越高。调度在结构与单核算力上追平 Linux 之后，剩余的 IPC 差距集中于此处。

最纯粹的探针是空系统调用 getpid，其处理体只取回一个已在内存中的进程号，几乎不做任何工作，测得的时延近乎全部来自陷入与返回路径。syscost.c 测得 StarryOS 的 getpid 需 1200 ns，同机 Linux 仅 167 ns，相差约 7 倍。一个不涉及调度、不涉及内存、仅进出内核一趟的调用仍慢 7 倍，说明差距落在这条路径本身。窗口化的 pipe-pong (ppong.c) 给出同一结论：两侧均以批处理摊薄单次开销，StarryOS 批处理后的地板仍为每条消息 13.4 μ s，Linux 为 1.87 μ s。本节先剖析这条路径的构成，指出每次调用都要支付而多数调用并不使用的三处成本，再说明各处的机制与对应优化，最后给出结果。

9.1 陷入路径的构成

在 aarch64 上，一次系统调用的时延由两部分构成。第一部分是陷入往返本身，即 uctx.run() 一整趟：SVC 进入、保存通用寄存器现场、按调用号查表分发、处理函数执行、恢复现场、eret 返回。这一段由 axcpu 的架构代码实现，是 ArceOS、StarryOS、Axvisor 三套系统共用的底座；getpid 这类空调用的处理体只占其中极小一部分，其时延几乎全部是这趟往返。第二部分是返回路径上的一组固定记账，在处理函数返回之后、恢复用户态现场之前执行，用于维护跟踪、定时

器与 CPU 时间等每进程状态。StarryOS 此前在这段返回路径上串接了三处工作，每次系统调用返回都完整执行一遍，而绝大多数调用并不需要其中任何一处。getpid 的 1200 ns 正是这两部分之和：陷入往返的架构地板，叠加三处每调用记账。三处记账构成架构地板之上唯一可摘除的冗余，后续优化集中于此。

9.2 每次调用固定支付的三处成本

以下三处均为 StarryOS 此前的既有行为，逐一说明其作用，以及在 getpid 路径上冗余的原因。

ptrace 停点 ptrace 是进程调试与系统调用跟踪机制，被跟踪进程在每次系统调用的进入与退出处停下，把控制权交给跟踪者（调试器或 strace）读取或修改其状态。StarryOS 此前在每次系统调用返回时都执行一遍这套停点判断与相关状态检查，即便当前进程未被任何跟踪者附加。对未被跟踪的进程，这段逻辑的结论恒为无需停下，其执行不产生任何效果。

POSIX 定时器轮询 POSIX 定时器（timer_create、setitimer 等）到期时须向进程投递信号，其到期检查此前由 poll_process_timer 在每次系统调用返回时执行。这段检查的代价并不小：它先获取全局 PROCESS_TABLE 的 SpinRwLock 读锁，再对当前进程做一次 Arc 升级，随后获取该进程 posix_timers 的 Mutex，最后遍历一遍存放定时器的 BTreeMap。即便进程未装配任何 POSIX 定时器，这一整套加锁、升级、再加锁、遍历仍照常走完，遍历的是一棵空树。对不使用 POSIX 定时器的负载，全程无一处能产生到期事件，冗余在于每次调用都重建并检查一份恒为空的状态。

utime/stime 记账 CPU 时间记账把每个线程消耗的时间区分为用户态（utime）与内核态（stime）两部分，供 getrusage、times 与 /proc 读取。其实现要求在每次进出内核的边界上完成一次 User 与 Kernel 之间的状态迁移并累加相应时段，而承载状态的 thr.time 为跨 CPU 共享，迁移须在一把 SMP 锁的保护下进行。这把锁架设于系统调用边界之上，每次进出内核各获取一次，锁本身即为这处成本的主体，锁内承载的时间累加相形之下可忽略。

9.3 三处优化

针对上述三处成本各实现一处优化，每项上线前均经过一轮对抗评审。三者只移除冗余，不改变对外可见的语义。

ptrace 快路径 将每次返回的停点判断收敛到一个预先算好的 is_ptraced 标志，对应 Linux 以 TIF_SYSCALL_WORK 位门控系统调用跟踪的做法：进程未被跟踪时整段 ptrace 停点逻辑不再进入，被跟踪时才照常执行。getpid 由 1200 ns 降至约 900 ns，降幅约 25%。

poll_process_timer 跳过 在 poll_process_timer 之前引入一个无锁的 PosixTimerTable::count 计数作为门槛。进程未装配任何 POSIX 定时器时该计数为零，前述的全局读锁、Arc 升级、Mutex 与 BTreeMap 遍历整段跳过，仅在确有定时器时才付出加锁与遍历的代价。此项在隔离测量下将 getpid 由 1026.5 ns 降至 853.3 ns，降幅约 17%。

无锁 `timer_state` 粗粒度 tick 记账 这是三者中收益最大的一处。先以二分定位成本：将 `set_timer_state` 改为空转，`getpid` 落到 458 ns，据此测得两次记账调用合计约 395 ns，占当时剩余开销的 46%。再区分是锁还是算术：仅按时钟 tick 做粗粒度记账、但仍保留那把 SMP 锁的版本只节省约 60 ns，确认成本主要在锁本身，锁内承载的时间累加无足轻重。据此把 User 与 Kernel 两个状态从受锁保护的 `thr.time` 迁移至每线程一个 `AtomicU8`：系统调用边界退化为一次 Relaxed 存储，不获取任何锁、不做记账，完整的时间轮询仅在该线程确有 `itimer` 装配 (`itimer_armed`) 时才执行。SMP 锁由此彻底移出系统调用边界。粗粒度记账改由时钟 tick 承担，`rusage` 的 `stime` 秒占比读到 0.71，与 Linux 的 0.72 对齐，精度对目标负载足够。7 组对抗评审得到 0 个确认缺陷。`getpid` 由 853 ns 降至 431 ns，降幅约 50%，此时已贴近陷入往返的架构地板。

9.4 tickacct 精确记账特性

记账口径本身另经一轮重做，形成 `tickacct` 特性。它把 `utime/stime` 的推进由每次系统调用的 `poll()` 迁移至时钟 tick 与上下文切换，对应 Linux 的 `TICK_CPU_ACCOUNTING`，为此新增一个全 CPU 的 `on_tick` 钩子。一次 15 组评审查出 5 个确认缺陷，修复其中 2 个 HIGH：一处陈旧的 `last_wall_ns` 会误触发刚装配的 `setitimer`，一处把系统调用前的用户态计算错记为 `stime`。评审同时暴露 `thr.time` 一处既有的跨 CPU 数据竞争，此前以 `AssumeSync<RefCell<...>>` 掩盖，现以 `SpinNoIrq<TimeManager>` 收口，并令 `poll` 返回 `[Option<Signo>;3]`，使锁不跨信号投递持有；7 组死锁评审得到 0 缺陷，上板确认无死锁。

该特性默认关闭。此前观察到的约 11% 降幅经核实为发射闭包带来的测量假象，SMP 安全重构之后关闭路径同样取得这份收益，故 `tickacct` 对 `getpid` 路径实为中性。已上线的无锁粗粒度 tick 计时给出的秒占比 0.71 已贴合 Linux；`tickacct` 额外提供的只是全 CPU 一致的精确记账，其每 tick 与上下文切换钩子带来维护与运行成本，在目标负载上并非必要，故保持默认关闭，作为需要精确 `rusage` 时的可选项保留。

9.5 结果

三处优化叠加，`getpid` 由 1200 ns 压至 431 ns，相对 Linux 由约 7 倍收敛至 2.6 倍，已并入板级出厂配置。

版本	getpid (ns)	相对 Linux
Linux	167	1×
StarryOS 基线	1200	7.2×
加 <code>ptrace</code> 快路径	900	5.4×
加 <code>poll_process_timer</code> 跳过	853	5.1×
二分参考 (<code>set_timer_state</code> 空转)	458	2.7×
无锁 <code>timer_state</code> 粗粒度 tick (上线)	431	2.6×
精确记账回退 (非 <code>tickacct</code>)	975	5.8×

表 4: `getpid` 微基准逐层下降，板级 p50，二分参考行为定位记账成本的中间探测点 (`set_timer_state` 空转)，精确记账回退行给出需要精确记账时的可选口径，高于上线值，不属该下降序列

残余的 431 ns 已贴近前述陷入往返的架构地板，包含 SVC 进入、保存现场、按调用号分发、

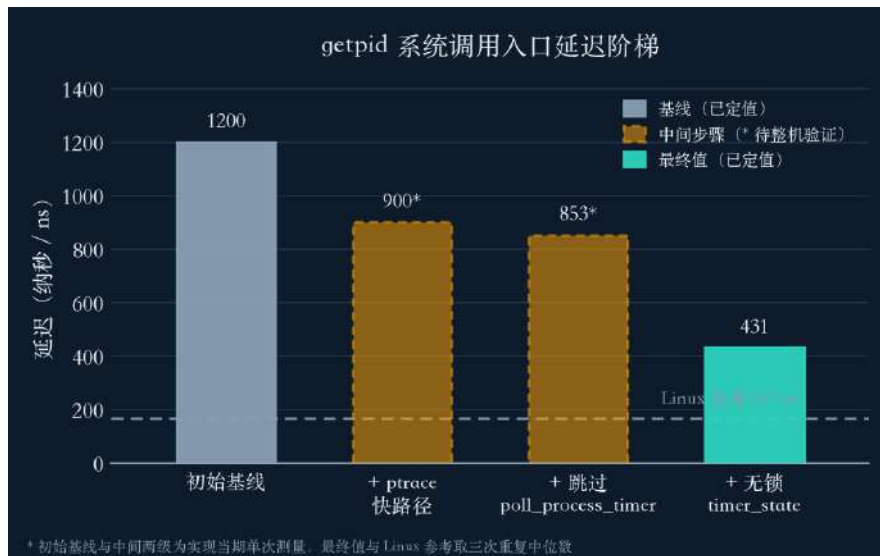


图 15: getpid 三步累计从 1200 ns 降到 431 ns, 为 Linux 167 ns 的 2.6 倍, 残余贴着 uctx.run 的陷入下限

恢复现场与 `eret` 返回。该路径属架构层, 由 ArceOS、StarryOS、Axvisor 三套系统共用, 其中不存在可摘除的冗余, 改动波及面大且无对应收益, 故不触碰。pipe 数据路径是下一步的优化方向, `pipe_wr` 约 5 μ s 对应 Linux 的约 0.4 μ s。

10 内存与分页

分页是现代内核管理内存的基础机制。进程看到的是连续的虚拟地址空间, 硬件的内存管理单元 MMU 把每个虚拟地址按页翻译到某个物理页帧, 翻译关系保存在页表中。RK3588 上一页为 4 KiB, 一次翻译要遍历一棵多级页表。为避免每次访存都遍历页表, MMU 内置 TLB 缓存最近用过的翻译。内核在进程创建时并不预先为全部虚拟地址建立映射, 仅在某页首次被访问时经缺页异常按需建立, 即首次触碰 (first-touch)。首次触碰这条路径的每页固定开销, 直接决定内存密集负载的表现: 一段大内存被顺序写过一遍时, 成千上万次缺页逐页累加, 每页多花几百纳秒便在秒级差距上显形。StarryOS 此前把 128 MB 私有匿名内存全部 fault-in 需 0.8 到 1.3 s, 约为 Linux 同机同规模读数的 9 到 15 倍。现已在首次触碰路径上补齐三项优化, 并在落地过程中修复一类潜伏于共享多架构 crate 的页表与 TLB 正确性缺陷。完成后 128 MB 私有匿名首次触碰为 0.030 s, 快于 Linux 的 0.086 s (2.87 倍), 是在 RK3588 上领先 Linux 的少数结果之一。本节先说明分页、首次触碰、多级页表与 TLB 各自的机制, 再说明既有路径的每页开销与新增的三项优化, 最后说明一类共享 crate 的正确性修复。

图 16 给出首次触碰这条路径及其上的多级页表遍历环节。一次缺页需分配物理帧、清零帧内容、写入页表项、并维护 TLB, 其中新建映射的 TLB 失效、帧的清零方式与映射的页粒度均为每页固定成本。三项新增优化分别压缩这三处成本, 共享多架构 crate 的页表遍历与 TLB 维护则是正确性修复所在。

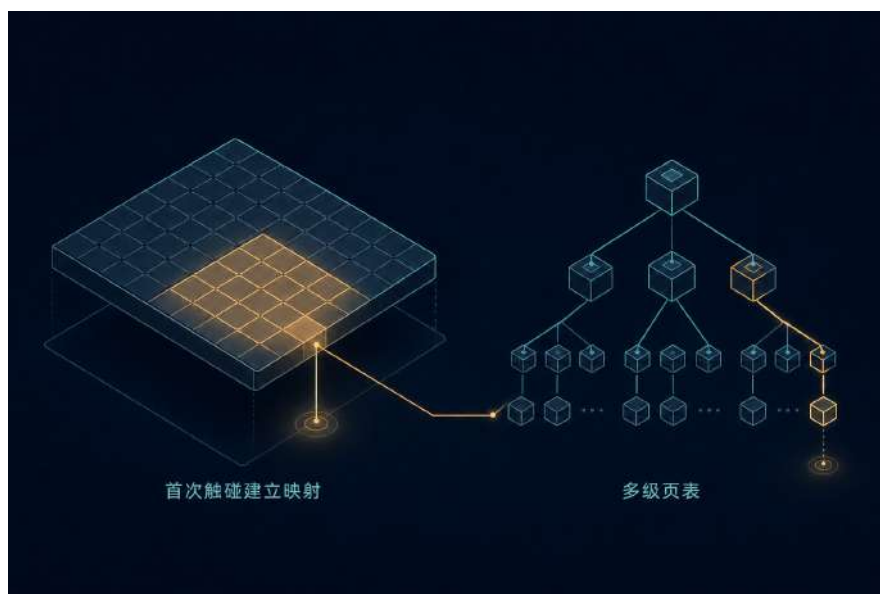


图 16: 内存首次触碰建立映射与多级页表遍历路径; 三项优化作用于每页固定开销, 正确性修复位于共享多架构 crate 的页表遍历环节, 青色为新增部分

10.1 分页、首次触碰、多级页表与 TLB

多级页表与地址翻译 aarch64 在 4 KiB 页、48 位虚拟地址下使用四级页表。一个虚拟地址被切成四段 9 位的索引外加 12 位页内偏移, MMU 从页表基址寄存器出发, 逐级用索引选出下一级表的物理地址, 走满四级得到叶级页表项, 其中的物理帧号与偏移拼出最终物理地址。中间各级的表项指向下一级表, 叶级的表项则描述一页的物理帧与权限。某些级上的表项可直接描述一个更大的块而不再下探, 例如二级上的一个块表项直接映射 2 MB, 这即大页的硬件基础。这套走查完全由硬件完成, 但每翻译一个未命中的地址都要多次访存读表, 代价可观。

TLB 与失效 为省去重复走查, MMU 用 TLB 缓存虚拟页到物理帧的翻译。命中时翻译近乎免费, 未命中时硬件走一遍页表并把结果填入 TLB。当内核改动一条已缓存的翻译时, 必须失效 TLB 中的对应条目, 否则硬件会继续使用陈旧翻译。aarch64 的失效指令 `tlbi` 有本地与广播两种形态: 本地形态 (如 `vmalle1`) 只失效当前核的 TLB, 广播形态 (如 `vmalle1is`、`vaae1is`) 经内部共享域通知所有核。SMP 下改动一条被多核共享的翻译必须用广播形态, 否则兄弟核会遗留陈旧条目。失效指令前后还需 `dsb` 屏障以确保描述符写入与失效在正确次序上对其余核可见。

首次触碰与按需缺页 内核为一段匿名内存建立虚拟区间时并不立即分配物理帧, 页表项处于 `not-present` 状态。进程首次访问该区间某页时, MMU 因该页无映射触发缺页异常, 缺页处理器分配一个物理帧、清零其内容以免泄漏残留数据、把帧写入叶级页表项使其转为 `present`, 随后返回重执触发访问的指令。这条路径对每页各走一遍, 因此其每页固定开销乘以页数即为触碰整段内存的总代价。StarryOS 已有 `fault-around` 机制, 在一次缺页中顺带为邻近的 32 页预建映射, 缺页次数因而受抑制, 差距落在每页开销本身。

每页固定开销如何决定内存密集负载 把 128 MB 私有匿名内存顺序写一遍要触碰约 32768 个 4 KiB 页。每页固定开销中的常量项在这个量级上被放大: 一次多余的广播失效、一次逐字节清零、

一次本可合并的缺页，单看只是几十至几百纳秒，乘以数万页即成秒级差距。sysbench 的默认 1K 块内存子测曾读到约 42 MiB/s、对 Linux 呈约 200 倍差距，此现象一度被疑为 DRAM 损坏。经核查，热态带宽为 7.3 GB/s，DRAM 本身并无问题，瓶颈落在首次触碰的缺页路径，其逐页开销即下文三项优化的对象。

10.2 首次触碰路径的既有每页开销

既有首次触碰路径的每页固定开销由三部分构成。其一，每建立一条全新映射均伴随一次广播 TLB 失效，128 MB 的连续触碰会产生约 32768 次 `tlbi vaael1is; dsb sy; isb` 的失效风暴；然而这些页刚从 not-present 转为 present，此前并不存在旧翻译可供陈旧，此次广播纯属浪费。其二，新帧的清零采用普通存储逐字写零，未使用 aarch64 提供的块清零指令。其三，映射以 4K 粒度建立，每 2 MB 需经历 512 次独立缺页，缺页处理的入口开销随之乘以 512。三项均为既有路径的固定成本，下文三项优化逐一对应。

10.3 三项首次触碰优化

新建映射跳过广播 TLB 失效 在共享的 `page_table_multiarch` 中为 `PagingMetaData` 增加 `NEED_FLUSH_ON_MAP` 标志，默认为真，在 aarch64 与 x86_64 上置为假。它只放行 not-present 到 present 的新建映射跳过失效，present 到 present 的 `remap`、`protect` 与 `unmap` 仍照常失效。跳过之所以安全，在于一条刚建立的映射此前不存在任何翻译进入 TLB，硬件不会命中一条尚未缓存的陈旧翻译；真正需要失效的情形是改写或撤销一条已可能被缓存的映射，这些情形未被放行。此项消除了整段首次触碰中约 32768 次逐页广播 `tlbi vaael1is; dsb sy; isb`。RISC-V 与 LoongArch 出于保守考虑保留默认的每次失效。此项将首次触碰由 0.8 到 1.3 s 压至约 0.37 s。

aarch64 dc zva 块清零 新帧的清零改用 aarch64 的 `dc zva` 指令，该指令按 DCZID_EL0 报告的块大小一次清零一整块缓存行，取代逐字存储的清零循环。清零是首次触碰中不可省略的一步（须消除残留数据），`dc zva` 在不改变语义的前提下降低其每页成本。此项将首次触碰由约 0.37 s 压至约 0.26 s。此外配套接入 `madvise` 的 `MADV_POPULATE_WRITE`、`MADV_POPULATE_READ` 与 `MADV_WILLNEED` 预取路径，令用户态可在访问前主动预建映射，把成批的首次触碰从关键路径上前移。

THP-lite 2 MB 私有匿名提升 feature 门控的 THP-lite 将 2 MB 对齐的私有匿名区提升为大页，机制是用一个二级块表项直接映射整块 2 MB，取代 512 个 4 KiB 叶级表项。大页对首次触碰的收益有两重：其一，一次缺页即建立整块 2 MB 映射，将该区间的缺页次数由 512 降为 1，缺页处理的入口开销随之降为原来的 512 分之一；其二，一块 2 MB 仅由一条页表项描述，原本需 512 条，页表本身更小、走查更浅、TLB 每条目覆盖的地址范围扩大 512 倍，TLB 命中率随之提升。实现将区间按“4K 头、2M 体、4K 尾”切分，仅对 2 MB 对齐的体段提升，头尾以 4K 兜底。它具备正确的 break-before-make 失效、拆分对 OOM 的原子性、无法提升时的 4K 回退与 COW 破坏路径，遵循 `MADV_NOHUGEPAGE` 与 `PR_SET_THP_DISABLE` 的逐区间/逐进程退出（后者跨 clone 继承），并兼顾 `MAP_NORESERVE`。其中 `MADV_NOHUGEPAGE` 供对大页敏感的负载按区间关闭提升，回落到 4K 路径。该实现在主机侧通过 54/54 测试。此项将首次触碰压至 0.030 s。

优化	128 MB 首次触碰
既有基线	0.8 到 1.3 s
新建映射跳过广播 TLB 失效	约 0.37 s
aarch64 dc zva 块清零新帧	约 0.26 s
THP-lite 2 MB 私有匿名提升	0.030 s

表 5: 首次触碰缺页路径每页开销的逐项压缩 (A76, 128 MB 私有匿名)

完成后的 128 MB 私有匿名首次触碰为 0.030 s, Linux 同机同频为 0.086 s, 快 2.87 倍, 板级多轮读数落在 0.024 到 0.033 s。收益来自硬件层面: 更少的 TLB 广播、dc zva 块清零、每 2 MB 减少 512 次缺页; Linux 侧缺乏等价的用户态手段获得同等收益, 因而这是一处真实的硬件领先。此处测量的是首次触碰延迟这一维度。多线程流式内存带宽属于另一维度, 受板级固件缺少 DDR 变频接口所限, 归入频率电压一节讨论。



图 17: THP 提升与新建映射跳过 TLB 失效把 128 MB 私有匿名首次触碰逐项压至 0.030 s, 快于 Linux 的 0.086 s (领先 2.87 倍)

10.4 页表与 TLB 的一类正确性修复

提升大页、跳过失效、引入 2 MB 块这几项改动落地时, 配套的多代理对抗审查 (每项改动 7 到 26 名独立审查者) 发现一批潜伏于共享多架构 crate 的 SMP 与 UAF 缺陷, 其中若干项为既有代码中长期存在的问题。这批修复已按单一关注点拆分, 准备为独立 PR 提交。逐处说明其机制如下。

not-present 巨页块误判为页表导致的 UAF 最主要的一处位于 `page_table_multiarch` 与 `page_table_entry`。一个 not-present 的巨页块 (例如一段被 `PROT_NONE` 设置的 2 MB THP) 在回读时 `is_huge()` 返回假, 页表遍历遂将该块对应的数据帧误当作下一级页表逐层下探。unmap 时 `dealloc_tree` 与 Drop 会顺着这条错误的下探将该数据帧当作页表帧释放, 若该帧正与另一

进程 COW 共享, 即构成一次 use-after-free。修复方式为新增 `GenericPTE::is_table()` (present 且非块方视为页表), 所有遍历与释放判定统一经由该接口, 并补齐 not-present 巨页块的 unmap 路径, 仅释放块自身的帧而不再深入。同一轮审查一并修复了两处相邻缺陷: 懒页 (从未 fault) 上的 `dealloc_frame(0)` 回归, 以 `paddr()==0` 守卫; 以及 unmap 与巨页重提升时对空的中间/拆分页表的回收, 其中一版基于 `is_present` 的丢弃自身即是 UAF, 已回退重写。

aarch64 的 break-before-make 与内存序 另有几处 aarch64 侧的隐患, 均处于内核范围, 波及所有 ArceOS 家族内核而不限于 RK3588:

- PTE 写入与 `tlbi` 之间缺少 `dsb ishst`, 失效可能在描述符写入落地之前被其余核观察到, 破坏内核中每一处 break-before-make 序列; 已将存储排序至 `tlbi` 之前。
- 全量刷新回退使用了本地的 `vmalle1`, 本应使用广播的 `vmalle1is`, 一次触发全量刷新的大 `mprotect` 会在兄弟核上遗留陈旧 TLB; 已改为广播。
- THP 拆分对 OOM 不具原子性, 先 unmap 再分配叶级页表, 分配失败或 SMP 下 BBM 的 TLB 冲突窗口会遗留撕裂映射或泄漏; 已改为先预留叶表的免分配拆分原语, 提交阶段不可失败且可回滚。
- 回收阶段的多次 TLB 失效批处理至单个屏障之后 (`flush_tlb_nosync` 配合 `sync`), 拆卸时每 GiB 的 `dsb` 由约 512 次降至约 16 次。

fork 路径上的 EFAULT 内存工作还在板上暴露并修复了数处 fork/EFAULT 缺陷。其一, COW 引用计数使用 `u8`, 当一个只读的 `libc/text` 帧被父进程外加约 254 个 fork 子进程共享时发生溢出, `clone_map` 返回 `BadAddress` 转为 EFAULT; 扩展为 `u32` 并采用 `checked_add` (溢出转 ENOMEM) 后, 板级 `hackbench -P g10` (400 进程) 由 240 s 超时恢复至 2.5 s。其二, `populate_area` 将调用方的 4 KiB 对齐范围直接传入 2 MiB 的 THP(COW) 区, `CowBackend::populate` 报 `InvalidInput` 被误映为 EFAULT, 表现为板上偶发、随负载出现的 `hackbench/schbench` 失败 (4 字节的 `futex` 字与 `clone` 的 `tid` 指针命中按需路径), 将填充范围对齐至区的页大小即修复, QEMU `smp8` 复测后 `BadAddr=0`。此外在板上复现并修复了两例与 `PROT_NONE` 巨页相关的路径: fork 一个曾对 THP 区做过 `PROT_NONE` 的进程触发的 EFAULT, 以及穿过一个 not-present 巨页块的部分操作触发的 EFAULT, 二者均随上文 `is_table()` 与 not-present 巨页帧回收一并收敛。整区 `PROT_NONE` 再放开后 `mprotect` 返回 0、随后访问触发 `SIGSEGV` 的一例, 此前一度挂起, 现以整区放开时对 not-present 巨页块的重新提升修复, `thp_reenable` 的 A 至 F 六个用例在板上与 Linux 一致通过。另修复两处 SMP 下的数据完整性缺陷: 匿名区在 `mprotect(+W)` 放宽写权限时未破坏 COW, 可致跨进程写穿, 改为在放宽写权限时对共享帧执行 COW 破坏; `futex` 字此前为非原子读取, SMP 下可能读到撕裂值而丢失唤醒, 改以 `atomic_read_user_u32` 原子读取。

这批修复均位于 ArceOS、StarryOS、Axvisor 三套系统共享的多架构 crate 中, 波及四种架构, 一处改动即令三套系统受益。此类页表与 TLB 的 UAF 及乱序能够通过随手测试, 却在 SMP 叠加 COW 与 THP 时悄然破坏内存, 将其定位是原子 PR 与多代理对抗审查这一纪律的直接产物; 一轮 8 代理的取巧审查在最终交付集上未发现取巧实现。



图 18: 页表与 TLB 修复位于共享多架构 crate，一处改动波及 ArceOS/StarryOS/Axvisor 与四种架构

11 频率与电压 DVFS/PVTPLL

频率与电压调节 (DVFS) 依负载动态调整 CPU 的运行频率与供电电压，在功耗约束下取得算力，是真机上算力对齐的前置条件。RK3588 的调频链路有其特殊之处：CPU 时钟是电压耦合的 PVTPLL，SCMI 只选定一个环形振荡器的频带，真正交付的频率随供电轨压变化，改频须按档设定电压；且 CPU 簇的时钟单元 CRU 由 BL31 保护，软件无法直接改写其分频。StarryOS 此前将全部 CPU 固定于上电时的约 816 MHz 工作点，既无 governor 也无电压管理，sysbench cpu 在任意线程数下均维持于 160 事件/秒附近，逐核约为同频 Linux 的 0.75 倍。现已实现按簇的 ondemand 调频、完整的 PMIC 电压标定与 PVTPLL 环长切换 (#1657, 已合并)，取回对齐频率，逐核算力与同频 Linux 持平；多线程内存带宽是唯一尚未对齐的一项，其根因在主线固件，驱动已实现。本节先说明 RK3588 调频链路的三级机制，再说明既有与新增各部分的实现与板级验证。

图 19 给出这套调频的三级结构。最上层是 ondemand governor，依每核忙碌率评分并选定目标工作点；中间是 PMIC 电压子系统，按该工作点经 I2C/SPI 标定 CPU 供电轨压；最底层是 SCMI 及其下的 PVTPLL，依请求的速率选定环形振荡器的环长与频带。三级中，governor、PMIC 子系统与环长切换均为本队新增，SCMI 时钟通路则为既有。三者协同方能取回与 Linux 对齐的交付频率。

11.1 电压耦合的 PVTPLL 与 BL31 保护的时钟

RK3588 的 CPU 时钟并非普通的整数分频 PLL。SCMI 在低档 (约 600 MHz 以下) 选定一个整数 PLL，在中高档则选定一个 PVTPLL 环形振荡器，其振荡周期取决于工艺、电压与温度。SCMI 只选定该环的环长与所属频带，环真正交付的频率随供电轨压开环变化：同一环长下抬高轨压即抬高交付频率。因此 RK3588 上的改频不能只写时钟，须为每一档设定与之匹配的电压。

时钟单元 CRU 由 BL31 保护这一点可在板上直接读出。在 Linux 上经 /dev/mem 读取 B0PLL 寄存器仅得一份冻结的上电影子值 (m=272/p=2/s=2, 对应 816 MHz)，而核心实际运行于 816、

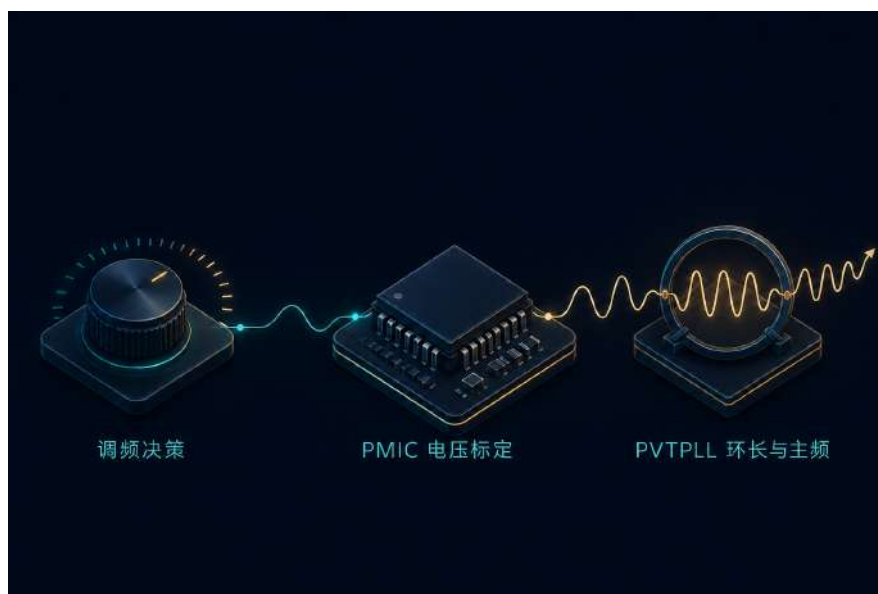


图 19: RK3588 调频链路: `ondemand governor` 依每核忙碌率评分选定工作点, PMIC 按该档标定供电轨压, PVTPLL 依请求的 SCMI 速率选定环长与频带; 各段均为新增的内核侧实现

1200、1608、2016 MHz 时该寄存器逐字节保持不变。软件无从经 CRU 直接读写实际分频, 交付频率只能经 PMU 周期计数器间接测得。电压耦合本身则以固定时钟、只扫轨压的方式判定: 将 SCMI 时钟固定于 1200, 交付频率随电压单调上升。

SCMI 时钟 (固定)	供电轨压	PMU 实测交付频率
1200 MHz	675 mV	1.19 GHz (恰为 1200)
1200 MHz	725 mV	1.32 GHz
1200 MHz	800 mV (上电)	1.49 GHz

表 6: SCMI 时钟固定于 1200、只扫轨压, 交付频率随电压单调上升, 判定时钟与电压开环耦合

由此可解释上电默认状态。上电轨压为约 800 mV (boot VSEL 经 U-Boot i2c md 读出为 0x30, 即 48, 按 $V = 500 + VSEL \times 6.25 \text{ mV}$ 折算为 800 mV), 只上调 SCMI 时钟而沿用该电压时, 设定的 1200 MHz 实测约为 1490 MHz, 属一次安全的过压: 约 1490 MHz 仅需约 710 mV, 上电轨的 800 mV 提供约 90 mV 余量, 未构成欠压。Linux 取得精确频率, 在于其 OPP 框架按每档设定 PMIC 电压; StarryOS 沿用上电电压, 因而出现在约 1.24 倍的一致过冲。欲取回精确频率并解锁更高档位, 须自行掌管电压轨。

11.2 既有的 SCMI 时钟通路

既有的 SCMI 通路只能经 SMC 下发时钟频率, 无 governor、无电压管理, 沿用上电电压。它决定了 StarryOS 未调频时的默认表现: Linux 的 A55 OPP 扫描将 StarryOS 的默认 160 事件/秒钉定于 816 MHz 上电档 (408 MHz 78 事件/秒、600 MHz 118、816 MHz 161、1008 MHz 202), 即上电工作点、无 DVFS。第一版仅上调 SCMI 时钟的驱动 (A55 至 1008 MHz、A76 至 1200 MHz, 沿用上电电压) 取得 A55 约 1.46 倍、A76 约 1.83 倍的真实提升, 但因前述电压耦合, PMU 读出核心运行快于所设定值, 一致过冲约 1.24 倍。取回精确频率、并向更高档位攀升,

须补齐电压侧，这是 #1657 所做的工作。

11.3 ondemand 按簇调频 (#1657)

调频侧新增一个按簇的 **ondemand governor**。每核忙碌 tick 在调度 tick 中以一个每核原子计数低开销累计 (BUSY_TICKS, Relaxed)，真正的 apply 推迟至一个可睡眠的周期内核任务 (cpufreq-gov, 周期 GOV_PERIOD_MS=100 ms)，因为 apply 需下发 SCMI-SMC 时钟并执行 PMIC 的 I2C/SPI 电压斜坡，无法在关中断的调度 tick 中完成。周期任务据每核忙碌率评分，以簇内最忙核心提升整簇频率，采用快升慢降策略，与 Linux 的 schedutil 一致。评分窗口最初取固定的 100 ms，一次迟到的轮询会高报负载并触发无谓的顶档跳变；现改为测量单调时钟给出的实际窗口 (LAST_POLL_NANOS)，迟到轮询不再高报。

11.4 PMIC 电压标定

电压侧新增完整的 PMIC 电压子系统，依 rk3588-cpufreq 特性开关编译。StarryOS 缺少 i2c/spi/pinctrl 基础设施，每条 PMIC 总线均需三层初始化：时钟解门控、核心软复位去断言、引脚复用。两簇分走两条总线：

A76 两路电压轨 经 i2c@fd880000 (总线 0) 上的 rk3x I2C 主控，驱动 RK8602@0x42 与 RK8603@0x43 的寄存器 0x06。写入钳位于 675 至 1000 mV、按 6.25 mV 整步进，升频时先写电压，并回读校验 (set_vsel_verify)。

A55 电压轨 经 spi@feb20000 (CS0) 上的 Rockchip rk3066 SPI 主控，驱动 RK806 的 DCDC2 (BUCK2_ON_VSEL=0x1B)。

频率读数通道 在 components/axcpu 的 aarch64 初始化中于 EL0 开启 PMCCNTR_EL0，使 cpuprobe mhz_pmc 读出真实交付的时钟。每一档 OPP 的电压均据此在板上标定与验证。

事务式 apply OPP 的 apply (apply_opp) 采用事务式流程：升频时先设电压后设时钟，降频时先设时钟后设电压，任一步骤失败即在该步终止；由此任何部分失败最坏只会过压，不会欠压。时钟变更在下调电压前先经 scmi::clock_rate 回读确认已生效。

A55 电压轨采用带上下限的开环写方式下发 (force_write_dcdc2, 钳位于 675 至 950 mV)。SPI 主控在时序、CTRLR0、引脚复用、时钟与复位上逐字节对齐 Linux，经 SPI 向 RK806 的 DCDC2 写入目标档位，事务完整下发，TX 回显、写入抵达芯片，可观测到相应的电压下降。开环写以钳位区间保证该轨不欠压：governor 仅凭 A76 回读成功即启用，A55 各档在 1008 MHz 以下最坏只令电压偏高，不会欠压。开环诊断路径默认关闭。

11.5 PVTPLL 环长切换

补齐电压后，仅凭抬压将 A76 推至 1725 MHz、A55 至 1523 MHz 即达电压天花板，逐核吞吐 (sysbench cpu 8 线程 3991) 约为同型号 Linux 的 0.75 倍。AVS/PVTM 分箱曾作为抬高天花板的候选，经验证被排除：A76 顶档 (2256、2304、2352、2400) 全部硬锁于 1000 mV、无 AVS 箱，主线亦无 Rockchip AVS 驱动。

真正决定同电压下主频的是 PVTPLL 的环长，其表见于 TF-A 的 rk3588_clk.c。低档（约 600 MHz 以下）走整数 PLL，环长为 0；1608 至 2400 MHz 整段 A76 频带的环长恒为 11，各档之间只有电压不同。StarryOS 原先将 SCMI 固定于 1200（环长 17），凭抬高该长环的电压方达 1725；请求更高的 SCMI 速率会把环重编为长度 11，同等电压下交付频率随之提升。同电压对照可见此机制：850 mV 下 ring1200（环长 17）的 1592 转为 ring1416（环长 11）的 1830（+238），**925 mV 下 ring1200 的 1725 转为 ring1608 的 2126（+401，约 +23%）**。

沿此实施四步 **OPP** 提升，全部在板上以 sysbench cpu 8 线程验证(--cpu-max-prime=20000)，零压降。

步骤	变更	A76 MHz	A55 MHz	8 线程事件/秒	对 Linux 5222
基线	ring1200, 1725/1523	1725	1523	3991	0.76
第 2 步	A76 ring1608 @925	2126	1523	约 4570	0.88
第 3 步	+ A55 ring1608 @950	2126	1881	4987	0.96
第 4 步	A76 ring1608 @1000	约 2256	1881	约 5137	0.98

表 7: PVTPLL 环长切换的四步 OPP 提升，逐核吞吐 0.76 倍提升至 0.98 倍，A55 1881 MHz 已超过 Linux 自身 A55 的 1800 MHz 上限

board 上采纳进 cpufreq.rs 的 OPP 阶梯，两簇在 1608 MHz 处由长环切至 11 环：

- **A76**: 408@675 / 816@675 / 1189@675（上电）/ 1318@725 / 1491@800 / 1592@850 / 1830@850(ring1416) / **2126@925(ring1608, governor 默认上限)** / 2256@1000(ring1608, 顶档，默认封闭)。
- **A55**: 408@675 / 816@675 / 1021@675（上电）/ 1212@762.5 / 1285@800 / 1372@850 / 1695@850 (ring1416) / **1881@950 (ring1608, 顶档，可达)**。

此前将高频挂起归因于 925 mV 下超过 1700 MHz 的压降，经查为误判：每一次板上挂起均为工具链错配（分支锁定 nightly-2026-05-28，强用 nightly-2026-07-15 会误编译 axcpu/axio 而在启动处卡死）。2126 MHz 已在 925 mV 下稳定达成。

11.6 逐核对齐与 governor 上限

取得对齐频率后，单核算力与同频 Linux 持平，多线程残差即频率比，单位频率的算力已对齐。逐核 A55 为 359，Linux 为 349，比值 **1.03**（超过 **Linux**）；逐核 A76 为 896，Linux 为 962，比值 0.93（图 20）。单线程为 0.94 倍，4 线程为 0.98 倍；8 线程在环长顶档为约 5137 事件/秒（0.98 倍），在占用式调度配置下为 4987 事件/秒（Linux 5222，0.96 倍）。0.72（1725/2400）为对齐前的基线频率比；两个 8 线程读数对应两种经验证的调度配置：部署默认档（A76 2126 MHz）八线程达 0.96 倍，环长顶档（A76 2256 MHz）收敛至 0.98 倍。频率固定关闭 cpufreq 时 8 线程约 2006 事件/秒，开启后约 4987，DVFS 一层约 2.49 倍，全为频率。逐核对等的完整对照表见调度一节，此处不再重列。

顶档保留一条未经验证的安全上限。2256 MHz @ 1000 mV 的 A76 顶档默认封闭（A76_CAP_TO_ALLCORE_SAFE=true, gov_max_idx 对大核返回 len-2），governor 封于 2126 MHz @ 925 mV。

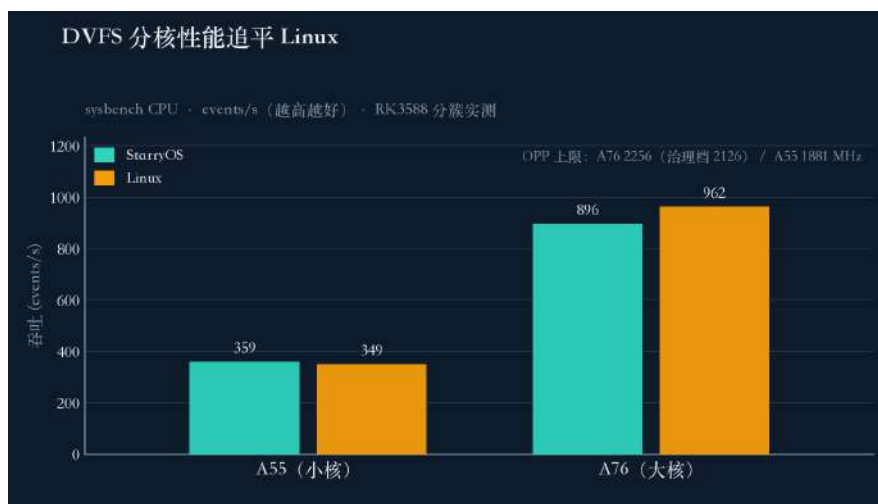


图 20: A55 与 A76 两簇逐核 sysbench CPU 吞吐对照 (events/s, 越高越好); 对齐频率后逐核算力与同频 Linux 持平 (A55 1.03、A76 0.93)

原因在于唯一冻结过的全核零压降快照位于 1592 MHz, 而 StarryOS 无温度调节器, 无温控封顶下不宜将全核 1.0 V 设为默认, 待温控就位后方放开。

11.7 内存带宽的 DDR 变频固件外因

CPU 侧对齐后, 多线程内存带宽是唯一尚未对齐的一项, 其根因在主线固件, 驱动已实现。经唤醒铺开与占用式放置将内存线程分散至多个控制器, 聚合差距的大部已收回。单流 memcp 受 DDR 上电低频所限, 约为 Linux 的 0.36 倍; 多线程写带宽经放置铺开已回升至 0.61 倍 (sysbench memory 写入 8 线程 34330 MiB/s)。

内存带宽指标	StarryOS	Linux	比值
sysbench memory 写入, 8 线程 (MiB/s)	34330	56046	0.61
1M 块 memcp 原始带宽 (GB/s)	13 到 20	约 55	0.36

表 8: 多线程写带宽经放置铺开已回升至 0.61 倍, 单流 memcp 残差 0.36 倍同出于 DDR 未变频这一根因; 首次触碰延迟属另一维度, 见内存一节的 2.87 倍领先

为此已实现 **DDR/DMC 变频驱动** (drivers/ax-driver/src/soc/rockchip/ddr_dvfs.rs, 特性 rk3588-ddr-dvfs), 逻辑本身正确。RK3588 的 DDR 频率不由 SCMI 控制, SCMI 的 DDR 时钟在 TF-A 中仅为空操作桩; 真正的接口是 Rockchip 的 SIP DRAM 接口 (SMC 0x82000008 加共享页 0x82000009)。驱动按 GET_VERSION、SHARE_MEM、DRAM_INIT、GET_FREQ_INFO、SET_RATE、MCU_START、POST_SET_RATE 的顺序执行, DDR 步进 {528, 1068, 1560, 2112} MHz, 电压先行 (vdd_dds0 经 RK806 BUCK5 至 0.875 V, vdd_log_s0 经 BUCK3 至 0.75 V)。

该驱动在板上受主线固件限制, 驱动已实现。板上验证中 GET_VERSION 返回 -1 (SMC_UNKNOWN), 而同机 SCMI 的 SMC (0x82000010) 正常, 此即主线 TF-A 缺少 DRAM SIP 处理器的特征。打通须更换一版将 DRAM SIP 置于 EL3 的 rkbin BL31 并重刷开发板。驱动因此仅

停留于探测态 (ENABLE_SET_RATE=false)，约 0.36 倍的单流 memcpy 带宽差距源于板级固件缺少 DDR 变频接口，属 StarryOS 侧实现之外的制约。

12 启动与首帧推理 TTFI

首帧推理时延 (TTFI) 是网球机器人上电后实际感受到的等待：从 U-Boot 打印 Starting kernel 交出控制起，到 live 模式下首帧 RKNN 推理完成为止。它跨越内核早期初始化、副核启动、设备探测、init 抵达应用、模型加载与相机启动五段，任何一段的串行等待都直接累加到用户可感的延迟上，因而是一项对整条启动链高度敏感的端到端指标。此前 StarryOS 在该指标上落后 Linux，同机同负载下内核到首帧为 19.22 s，Linux 中位数为 11.28s，慢约 1.7 倍。现已将 TTFI 降至 **9.83s**，领先 **Linux 约 1.45 s**。这与 THP first-touch 内存（快 2.87 倍，见内存一节）同为幅度较大的领先；连同单核 A55 与 schbench 唤醒吞吐的领先，构成 StarryOS 在 RK3588 上对 Linux 取得领先的几处方向，其余方向多为持平。本节先说明 TTFI 的度量口径与启动链各段的机制，再按段说明既有行为与我们的改动，最后给出真机结果与诚实边界。



图 21: TTFI 启动链的分段：从 U-Boot 交接到 live 首帧 RKNN 推理，本队改动的段与消除的两处主要开销以青色标出

图 21 给出这条启动链的分段与各段耗时。度量以内核交接为零点，以应用打出 TENNIS_FIRST_INFERENCE 为终点，由主机在 1.5 Mbaud 串口上加时间戳采得；两侧运行同一份 glibc 网球二进制，模型为 ReLU 480×640，推理亲和固定于大核 4-7。

12.1 启动链的组成

启动链从 U-Boot 交出控制延伸到首帧推理，可分为五段，其骨架为既有实现，以下按执行顺序说明各段的机制。

最先是内核早期初始化。someboot 在 MMU 与缓存关闭的窗口内解析扁平设备树 FDT、为各核复制 per-CPU 数据、建立整块 RAM 的线性映射，随后 rust_main 完成内存与调度子系统的初始化。此窗口全程非缓存执行，本身构成约 1.43 s 的开销。

其后是副核启动与设备探测。RK3588 为 4 个 A55 小核加 4 个 A76 大核的大小核结构，副核启动后设备探测方能并发；探测阶段逐一 bring-up PCIe 控制器、块设备与 USB 相机。该板有四个空 PCIe 连接器，空槽的链路训练需等待 LTSSM 状态机超时；块设备的写完成依赖 dwmmc 主控的完成中断，中断若不按时到达则退回轮询。

接着是 init 抵达应用。init 从 shell 或自动脚本拉起网球应用，应用装载并动态链接 librknrt (7.7 MB)、libmpp 与 librga，这一装载与动态链接阶段本身约耗 4.1 s。

再往后是模型初始化。rknn_init 建立 NPU 上下文并加载权重，其内部由 librknrt 的数百次顺序生产者消费者线程握手串联，每次握手都是一对协同线程间的交接。

最后是相机启动与首帧推理。UVC 相机送出 MJPEG 帧，经 JPU 解码、RGA 预处理、NPU 推理的零拷贝流水线得出首个检测结果。19.22 s 与 11.28s 之间的差距即分解为设备探测段的串行等待、模型初始化段在相机负载下的饥饿，以及一段与 Linux 等量的应用冷启动。

12.2 基线的构成：19.22 s 的分解

提交版本的 TTFI 为 19.19 至 19.23 s，稳定在 19.22 s 附近，而 Linux 从 10.09、11.15、11.42 到约 12.43 s，中位数为 11.28s。二者约 8 s 的差距由两处可修复的内核开销加一段与 Linux 等量的应用冷启动构成。第一处是四个空 PCIe 槽的串行链路探测：控制器 fe150000、fe170000、fe180000、fe190000 均报 link down LTSSM=0x3，每个空槽把链路训练等到超时，四者串行约 5.6 s。第二处是块设备完成中断的退化：写完成中断在 2 s 内未触发后退回轮询，约 2.1 s，其成因是提交分支尚未包含 dwmmc 的 wait_card_not_busy 修复。这两处均落在内核侧，是 StarryOS 独有的开销；其余部分为两侧共有的应用冷启动。

12.3 相机初始化在流负载下的 32 s 停滞

将提交分支 rebase 至上游 dev (156 个新提交) 后，TTFI 回退至 49.29 至 50.03 s，其中 model_init 膨胀至 31.8 至 32.6 s，解码退回 CPU 路径。恢复经板级验证的 NPU/JPU crate 后仍为 50.03 s，故驱动 crate 并非成因。这一 32 s 停滞的定位构成本节最关键的一段根因排查。

征象与逐项排除 按调用计时，全部 32 s 落在设备 ioctl 处理内，该路径被调用 32719 次 (f.ioctl 分派累计 38002 ms，而文件描述符解析仅 33 ms)。按阶段打点，约 30 s 全部落在 init_yolov8_model (即 rknn_init) 内，RGA 与 JPU 建立仅 0.3 s。决定性的一组对照是：同一模型初始化在 --mode validate 无相机负载下为 1.5 s，而在 UVC 边流边初始化时约 32 s。任务转储显示 NPU-init 线程长期处于 Ready 态、utime 近零而 stime 持续累积数秒，属线程饥饿，计算本身并不耗时。逐项上板测试排除了一批嫌疑：移除网卡后无变化 (否定 NIC/TCP)，驱动 crate、缺页、文件描述符路径均无关，线程放置以 taskset -c 0、4、4-7 三种绑定测得完全一致的 32 s，延后入队与自旋两种唤醒机制也测得一致。“dash 在空闲时占满一个核”一度被疑为成因，核对后确认这是 stime 计入阻塞在系统调用中的墙钟时间所致的记账假象，并非真实 CPU 占用。

根因 成因是 RR 调度器 50 ms 时间片配合前端重排队，与相机 URB 完成中断洪流叠加。librknrt 在初始化中做数百次协同线程间的顺序交接，每一次交接的接收方都须等到当前时间片在 URB 洪流下耗尽才被调度，数百次串联即塌缩为数十秒。

消除 解决包含两项，一项部分缓解、一项彻底消除。时间片从 50 ms (5 个 tick) 降至 10 ms (1 个 tick) 使停顿减半，从 32 s 降至 16.9 s (提交 5360b2722，改动 axtask 的 MAX_TIME_SLICE)。先完成模型加载再启动相机将其彻底消除：把 cam.start() 移至 det.init() 之后，使模型初始化不再与 URB 洪流争抢，model_init 从 16870 ms 降至 140 ms，是单项最大的收益 (提

交 870fd59f0)。相机延后在原地测得 `capture_init_ms=756`、`model_init_ms=139.97`、`first_frame_wait_ms=1183`、`first_detection_ms=25`，应用内部冷启动从 18.0 s 降至 1.35 s。该修复保留了全套驱动，未以裁剪功能换取速度。

12.4 设备探测段的两处内核停顿消除

基线中两段最重的探测开销由两处驱动改动直接消除。空槽 PCIe 链路等待从 800 ms 降至 100 ms、重试从 80 次降至 10 次，已由固件训练完成的链路借助提前返回得以保留，四个空连接器各省约 700 ms，合计约 2.8 s（提交 aa25afaa3）。`dwmmc` 在完成一次写之前先等待卡退出 `program-busy` 状态，去除了完成中断在 2 s 内未触发后退回轮询的约 2.1 s 启动停顿，此项移植自块设备一节的 8eb3ba417（提交 594152ee8）。

12.5 启动到 shell 的三处架构改造

内核到 shell 的时段由 8.22 s 压至 4.46 s，由六项改动驱动，其中三项为架构性改造，以下逐项说明其机制。

先启动副核后探测 (SMP-before-probe) `rust_main` 原先在 `probe_all_devices` 之后才启动副核，全部设备 `bring-up` 仅在 `cpu0` 上单核执行，任何被迁移至副核的任务都会命中未初始化的运行队列，触发 `AxRunQueue::resched` 空指针解引用（FAR VA:0x10）。这是整轮排查中“设备工作仅落在 `cpu0`”这一制约的唯一成因。将副核启动与 `INITED_CPUS` 屏障移至探测之前、恢复 Linux 的顺序后，`PROBE_JOB 0..3` 得以在 `cpu1` 至 `cpu4` 上并发执行，内核到 shell 从 8.22 s 降至 7.50 s（提交 3d91d372a）。这项改造同时使后文的探测任务绑核成为安全操作。

借用设备树而不复制 (FDT-borrow fdt_ref) 并发之后单个 PCIe 控制器的耗时仍偏高，根源是每次 `phandle` 查找都经 `live_fdt()` 调用 `rdriver::with_fdt(Clone::clone)`，将约 280 KB 的整棵扁平设备树深拷贝一次，与硬件等待无关。新增 `fdt_ref() -> Option<&'static Fdt>` 借用静态设备树，并改动三处 `live_fdt()` 调用点后，PCIe 的 `phy=` 阶段从 248 至 657 ms 降至 0 ms，PCIe 段墙钟从 1.98 s 降至 0.59 s，内核到 shell 缩短至 6.04 s（提交 cc3a9207b）。这是本段收益最大的一项改动。

PERST 时序、someboot 记忆化与单遍 FDT 在上述基础上再叠加三项。`PERST#` 保持时间从 200 ms 降至 100 ms，取 CEM r2.0 的 `Tpvperl` 下限，与 Linux 一致。`someboot` 记忆化已解析的 RAM 区间，避免在 MMU 与 D-cache 关闭下重复遍历 FDT。`init_memory_map` 由两遍遍历 FDT 改为单遍，省约 0.68 s。三项使内核到 shell 由 6.04 s 收敛至 5.14 s、再至 4.46 s。改动后单个控制器的探测已可读为 `PCIE_PHASE 0xfe180000 parse=0 prep=2 map=1 init=302`（毫秒），四者在 `cpu1` 至 `cpu4` 上并发，PCIe 段墙钟从 1.98 s 经 0.59 s 降至 0.18 s。

12.6 应用冷启动的两项通用改动

模型初始化在消除饥饿之后仍受制于权重文件的冷读。两项通用改动改善了这一段：页缓存批量读把多页的回填合为一次填充（提交 10871196e），文件回填缺页预读在首次缺页时提前读入邻近页（提交 fe54ed470）。两项与相机延后合并后，`model_init` 从 139.97 ms 的最优点上升

到含权重冷读的约 910 ms 稳定区间：提交版本边流边初始化时为 1540 至 1667 ms，rebase 后为 31760 至 32622 ms，validate 模式无相机时低于 1800 ms，相机延后为 139.97 ms，四机制齐备的内核加相机延后加预读为 906 至 910 ms。两项改动对两侧 OS 同样有效，属通用的应用冷启动改进。

12.7 init 自动执行与真正的对等差量

一次六维对抗式审计修正了度量口径。两侧运行的是同一份 glibc 可执行文件，任何应用阶段的加速对 Linux 同样有效，净差约为零。真正在 StarryOS 与 Linux 之间产生差量的仅有两处，一是 init 抵达应用的方式，二是内核到 shell 的启动速度。同一审计还纠正了一处此前的高估：早先“约 9.7 s、领先 Linux 3 至 4 s”的复合口径漏计了 shell 到 proc_start 之间约 4.1 s 的应用装载与动态链接阶段（librknrt 7.7 MB 加 libmpp 加 librga），据实计入后端到端约为 11.37 s，与 Linux 持平。

真正可辩护的领先来自 init 的公平性。使两侧均经各自 init 自动执行启动脚本后，Linux 走 systemd，StarryOS 走 `auto-exec /autoexec.sh` 自动拉起 `run_ttfi.sh`，从而消除手动进入 shell 引入的约 1 s 采样噪声（提交 bceb8f3a9）。叠加单遍 FDT 后，一次自动执行运行测得 `TENNIS_COLD_START capture_init_ms=825.50 model_init_ms=955.07 first_frame_wait_ms=1068.39 time_to_first_command_ms=2059.04`，端到端 TTFI 为 9.83s。

阶段	Linux	StarryOS 手敲进入 shell	StarryOS auto-exec
内核到 shell	约 9.0 s 到应用启动	5.14 s	4.46 s
Welcome 到应用启动	不适用	2.65 s	1.66 s
TTFI（内核到首帧推理）	11.28s	11.37 s	9.83s

表 9：同机同负载的 TTFI 分阶段对比，两侧均经各自 init 自动启动应用。StarryOS 精简 init 约 6.1 s 抵达应用，Linux systemd 约 9.0 s

12.8 中性与否定结果

四项改动的结论为中性或否定，一并如实记录。相机重叠（提交 5883051d2）把相机 open 与协商同模型初始化重叠，隐藏了 `capture_init`（825 降至 584 ms），但并发的 USB 建立扰动了 `cpu0` 上的 `rknn_init`（955 升至 1150 ms），该变体基线约 10.49 s，净差约 -0.15 s，使 TTFI 落至 10.34 s，落在噪声内，仍领先 Linux，留在代码树中作为可回退或可绑核的候选。早启动 MMU 的改造是一处严谨的否定结果：boot-timing 诊断测得 MMU 关闭窗口 `total_uncached=1431ms fdt_parse=674ms percpu_copy=637ms table_build=10ms other=110ms`，该窗口真实存在，故 `f4185acdf` 在 aarch64 引导恒等映射上提前开启 MMU 与缓存（因 someboot 无堆，围绕 UART 静态映射），改动正确且不损坏启动，QEMU `smp1` 通过 22/22、`smp4` 通过 36/36、真机零缺页启动，但板级为 4.446 s 对 4.46 s，收益约为零。前提有误：这 1.43 s 非缓存窗口在低启动频率下受 CPU 制约（A55 引导核在 `cpufreq` 调频前运行于低频），对计算密集型工作加缓存无从节省，故予以回退，boot-timing 诊断保留；真正的早启动改进应指向引导核频率，已记录未追。让步式探测延时（提交 6af543890）为净负：tick 粒度把众多小等待各自向上取整到 10 ms，反而放大了每个并发探测，已回退（68e5623ae）。探测任务绑核的首次尝试在副核上崩溃，正是先启动

副核后探测的改造使其随后成为安全操作。

12.9 QEMU 验证

四项通用审计改动 (PERST、someboot 记忆化、页缓存批量读、缺页预读) 经 amd64 容器下的 QEMU 验证: 9 个子用例、约 264 项检查、0 失败。其中 `syscall-test-mmap-readahead` 通过 9/9, 正是预读的判定用例, 逐字节校验批量填充、EOF 零填充与映射中部缺页; `syscall-test-mmap-family` 通过 79/79, `syscall-test-read` 通过 22/22, 另有 9 次干净启动验证启动表的正确性。实现期间捕获并封堵了两处真实隐患: 页缓存批量读的写缺页 COW/RSS 处理, 以及缺页预读在页回收时的脏页丢弃。

12.10 结果与诚实边界

整条曲线由 19.22 s (落后 Linux 约 1.7 倍) 起, 经 rebase 一度回退至约 50 s, 定位并消除 32 s 的 `rknn_init` 停顿后, 经约 11 项公平改动逐层回落至 13.25 s、11.37 s (手敲进入 shell), 最终经 `auto-exec` 降至 9.83s。最终 StarryOS 内核到首帧为 9.83s, Linux 为 11.28s, 领先约 1.45 s。一次代表性的稳态流水线为 `frame_to_detection_ms_avg=7.385`、`p50=6.604`、`p95=10.499`, 解码 `decode_ms_avg=1.282` `p50=0.961` (JPU 加 RGA 零拷贝), 推理 `run_ms_avg=5.320`、`rknn_perf_run_ms_avg=4.964`, 解码与推理错误均为 0。

口径一并声明。该领先来自精简 `init` 相对 `systemd` 约 5.5 s `bringup` 的机制优势, 两个板级样本均清晰领先, 内核到 shell 与 `model_init` 的方差较小。作为边缘 AI 设备的实用指标, “StarryOS 更早得到首帧推理”这一结论真实、板级可复现, 其口径限定在上电到首帧这一整机指标。连同 THP `first-touch` 内存 (快 2.87 倍)、单核 A55 与 `schbench` 唤醒吞吐, 这些构成 StarryOS 在 RK3588 上领先 Linux 的方向。

13 原生显示栈 VOP2 / HDMI-QP / HDPTX

显示栈是从内存中的一帧像素到点亮一台物理显示器的整条驱动通路: 一段帧缓冲经显示控制器按视频时序扫出、经发送器打包为传输协议、经物理层串化为差分信号, 最终在连接器上驱动屏幕。StarryOS 在 OrangePi-5-Plus 上此前没有通向真实 HDMI 帧缓冲的路径。现有的 `card0` (`starry-simplifiedrm`) 只是叠加在 `axdisplay` 之上的 KMS 前端, 真正经 `rdif-display` 注册一块真实帧缓冲的后端仅有 `virtio-gpu`; 因此在板端 `has_display()` 返回 `false`, `/dev/fb0` 并不存在, 任何依赖标准帧缓冲的图形程序都无从运行。现已从零实现一套 `#[no_std]` 的 RK3588 显示驱动栈, 覆盖 VOP2 显示控制器、HDMI-QP 发送器与 HDPTX 组合 PHY, 目标是使一份标准的 Qt6 QtWidgets 指针时钟经 `/dev/fb0` 显示于物理显示器。此项工作属于驱动开发, 其证据为 QEMU 渲染通过、主机寄存器级测试计数与逐字节核对主线的寄存器数值, 硬件点亮的时序已写就并推导完毕, 并已在 RK3588 上点亮验证通过 (HDPTX ROPLL 锁定)。本节先深入说明这条通路的四段链路及每一段的内部机制, 再区分既有与新增, 最后给出主机与板级的验证结果。

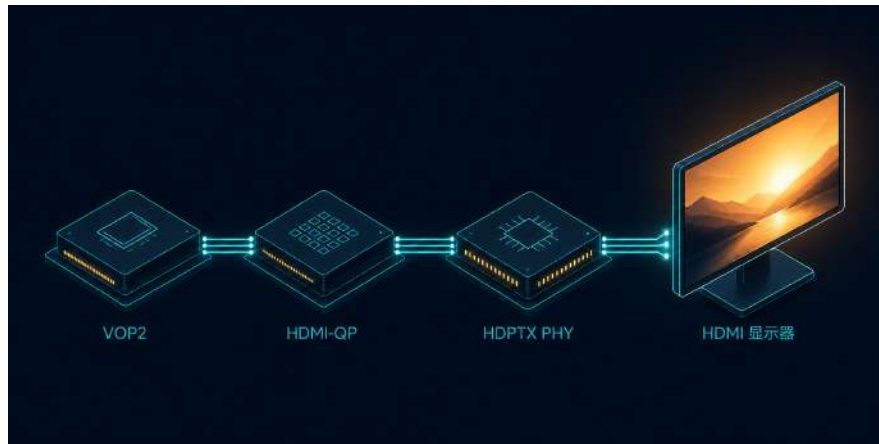


图 22: RK3588 四级显示链路: VOP2 输出显示时序, HDMI-QP 打包, HDPTX PHY 串化为 TMDS; 148.5 MHz 像素时钟由 HDPTX 的 ROPLL 产生, 该 ROPLL 像素时钟已在 RK3588 上板锁定验证通过

13.1 显示通路: 四段链路及其机制

图 22 给出这条通路的四段链路。像素从帧缓冲出发, 经 VOP2 生成显示时序, 经 HDMI-QP 打包为 HDMI 协议, 经 HDPTX PHY 串化为 TMDS 差分信号, 最后经连接器抵达显示器。四段各对应一个独立的 driver-core crate, 以下逐段说明其内部结构与工作机制。

VOP2 显示控制器 VOP2 是像素与时序的源头。它以内部的 AXI 主设备从 DRAM 中的帧缓冲取像素, 并驱动一个视频端口 (VP) 按光栅时序扫出。其内部有三层结构。其一是视频端口, 每个 VP 负责为一路输出产生完整的行场时序, 即水平总长、有效区、同步与前后消隐, 垂直方向同理; 1080p60 下水平总长为 2200、垂直总长为 1125, 配合 148.5 MHz 的像素时钟得到 60 Hz 刷新。其二是图层与窗口, VOP2 提供 Esmart 与 Cluster 两类窗口, 经 overlay 与 layer-sel 选择哪个窗口馈入哪个 VP; 本驱动使用 Esmart0 窗口, 格式为 XRGB8888, 其 REGION0_CTRL 置 0x1 使能窗口, CTRL1 的 AXI 读写 id 打包为 0xB0A0。其三是 config-done 影子锁存, VOP2 的时序与图层寄存器为影子寄存器, 软件把配置写入影子后置 config-done 位, 硬件在下一帧边界原子地把影子锁存为活动值, 从而避免半改动的时序被扫出; VP0 的 config-done 字为 0x18001, 其中 GLB_CFG_DONE_EN (BIT15) 是使能该锁存的关键位。VOP2 另提供帧起始中断用于同步。本驱动完成的是 VP0 至 HDMI0 的整套 modeset, 即时序计算、Esmart0 窗口装配、层选择与锁存的一次性编程。

HDMI-QP 发送器 HDMI-QP TX 接收 VOP2 送出的并行像素与时序, 将其打包为 HDMI 传输协议。它先经 LINK_CONFIG0 设定工作模式, 区分 DVI 与 HDMI 两种链路语义; 随后在消隐区插入信息帧, 其中 AVI infoframe 携带视频标识码 VIC, 1080p60 对应 VIC-16, 信息帧按定长打包为若干 DWORD 并附带校验和, 由 PKTSCHED 调度其在消隐区的插入时机。内容保护 HDCP 在此旁路 (HDCP2 bypass), 使链路进入无加密的直通传输。QP 是 RK3588 上较新一代的 DesignWare HDMI 控制器变体, 其寄存器布局与旧代不同, 本驱动按该变体的 LINK_CONFIG0、AVI 打包与 PKTSCHED 使能三处逐一编程。

HDPTX 组合 PHY HDPTX 是物理层，基址 0xfed60000，为 Samsung 组合 PHY IP。它承担两件事：把并行 TMDS 符号串化为高速差分信号，以及产生像素时钟。像素时钟由其内部的 ROPLL（环形振荡器锁相环）从参考时钟合成，经 mdiv、refdiv、sdiv 三个分频参数确定输出频率，1080p60 下取 mdiv=123、refdiv=1、sdiv=3 得到 148.5 MHz。串化路径由若干 TMDS 通道构成，各通道及公共部分需装载大量模拟微调值。PHY 的上电经 GRF（通用寄存器文件）配置完成，锁定状态经两次轮询确认，先查询 0_PHY_CLK_RDY（BIT2），再查询 0_PHY_RDY（BIT1）与 0_PLL_LOCK_DONE（BIT3）。这两次轮询是判断 PHY 是否真正拉起时钟并锁相的唯一运行期判据。

连接器与显示器 四段链路的末端是连接器与显示器。连接器一段仅做定频直通。

13.2 像素时钟归属

这条通路结构上的关键在于像素时钟由谁产生。RK3588 上 148.5 MHz 的显示时钟 dclk 由 HDPTX PHY 的 ROPLL 产生，以时钟名 clk_hdmiphy_pixel0 暴露；CRU 仅将该时钟选通至 VOP，V0PLL 与 CRU 本身不产生此频率。由此可知 PHY 位于整条链路的最上游，也是其中唯一真正不确定的环节，冷启动时须首先把它拉起并锁定，其后才谈得上 CRU 选通与 VOP2 扫出。CRU 侧的接线由新增的 `Cru::vop_hdmi0_clocks_setup` 完成：把 DCLK_VOP0 选通到 clk_hdmiphy_pixel0（选通字 0x01800080），并放开 VOP 的时钟门控（CLKGATE_CON52 掩码 0x230F）。像素时钟归属的这一事实决定了下文冷启动的编程顺序。

13.3 既有与新增

既有的部分是上层的 KMS 抽象与注册框架。card0 前端、axdisplay 中间层与 rdif-display 注册接口沿用 StarryOS 原有实现，它们定义了帧缓冲如何向系统注册、如何暴露为 /dev/fb0，此前只对 virtio-gpu 一个后端生效。新增的部分是整条硬件链路的四个 driver-core crate 及其 OS 胶合层，把这套注册接口接到了真实的 RK3588 硬件之上。OS 胶合层落在 ax-driver: display/rockchip_vop2.rs 完成 FDT 探测、DMA 帧缓冲分配与 register_display_with_info 注册，display/hdmi_coldinit.rs 承担冷启动编排，soc/rockchip 下的电源域与共享 CRU 访问器提供上电与时钟原语。四个 driver-core crate 均为 #[no_std]，各自附带主机测试，合计 39 项通过，构成本章目前的主要证据。

Crate	职责	主机测试
rockchip-vop2	VP 时序计算、Esmart0 窗口、层选择、config-done 锁存、VP0→HDMI0 全量 modeset	23
dw-hdmi-qp	HDCP2 旁路、LINK_CONFIG0 工作模式、AVI infoframe (VIC-16)、PKTSCHED 使能	6
rockchip-hdptx-phy	Samsung 组合 PHY 的 193 条主线初始化写、ROPLL 148.5 MHz 行、GRF 上电、两次锁定轮询	7
rockchip-soc CRU 扩展	DCLK_VOP0→clk_hdmiphy_pixel0 选通并放开 VOP 门控	3

表 10: 显示栈四个 driver-core crate 的职责与主机测试计数，合计 39 项

rockchip-vop2 的测试随实现阶段逐步扩充，由核心的 8 项增至加入复核护栏后的 9 项，再至加入 Stage-2 时序与中断的 17 项，最终随全量 modeset 达到 23 项。

13.4 193 条 PHY 写序及其判据

HDPTX PHY 的初始化是对主线 phy-rockchip-samsung-hdptx.c 的逐字移植，共 193 条寄存器写，取自该驱动的 reg_sequence 数组，按类别为 69 条 common-cmn、43 条 tmds-cmn、4 条 sb、5 条 lntop-lowbr、52 条 common-lane 与 20 条 tmds-lane，寄存器偏移为下标乘四。这些数值属于无文档的模拟微调值，其正确性判据只有一条，即与主线逐位一致，任何偏差都无从从语义上验证。运行期的唯一判据仍是前述两次 HDPTX-GRF 锁定轮询，因而上板时判定 ROPLL 是否锁定的板级 oracle 就是那两条串口读数，辅以一次 VOP2 寄存器转储。

modeset 侧的寄存器数值由一名独立复核代理重新推导，全部与主线吻合。表 11 列出 1080p60 下的核心值，均为独立再推导后与主线匹配的结果。

寄存器	1080p60 值	含义
DSP_HTOTAL_HS_END	0x0898002C	水平总长 2200、 HS_END 44
DSP_HACT_ST_END	0x00C00840	有效区起 192、止 2112
DSP_VTOTAL_VS_END	0x04650005	垂直总长 1125、 VS_END 5
DSP_VACT_ST_END	0x00290461	有效区起 41、止 1121
CFG_DONE (VP0)	0x18001	含 GLB_CFG_DONE_ EN (BIT15)
Esmart0 REGION0_CTRL	0x1	窗口使能、 XRGB8888
Esmart0 CTRL1	0xB0A0	AXI 读写 id
AVI 打包 DWORD	0x000D0200 / 0x00281027	校验和 0x27
ROPLL (148.5 MHz)	mdiv=123、refdiv=1、 sdiv=3	PHY 像素时钟
CRU 选通字	0x01800080	DCLK_VOP0→clk_ hdmiphy_pixel0
CRU 放门掩码	0x230F	CLKGATE_CON52

表 11: 独立再推导后与主线匹配的 1080p60 核心寄存器值

13.5 两条启用路径与安全取舍

驱动提供两个互斥的可选特性，对应两条不同的启用路径。adopt 路径 (rockchip-vop2) 复用 U-Boot 已经建立的 dclk、GRF、TX 与 PHY，仅重指 VOP2 并重编 modeset，其风险面最小，可在共享的 OrangePi 板级配置上按需启用。from-scratch 冷启动路径 (rockchip-vop2-coldinit) 自行掌管 PHY、TX、CRU 与 GRF，按像素时钟归属所定的依赖顺序依次拉起：先给电源域上电 (VOP 电源域 24、VO1 电源域 26)，再由 HDPTX PHY 完成 power_on_1080p60 并锁定，随后经 CRU vop_hdmio_clocks_setup 把像素时钟选通到 VOP 并放门，再配置 GRF 选路，接着执行 VOP2 modeset，最后使能 HDMI-QP TX。

默认启用 adopt 路径，复用 U-Boot 已建立的 dclk 与 PHY；冷启动路径作为可选特性按需启用，暂不纳入共享的板级配置。做出这一取舍是因为该 OrangePi 配置同时供 rknpu 与 tennis 板级测试使用，不应因显示驱动而受损。冷启动因此仅在有人值守的显示点亮会话中启用，该点亮会话已于 2026-08-12 完成。两种内核均可干净链接：adopt 路径 16.26 s 构建、无回归，冷启动路径 31.28 s release 构建、已将 rockchip-hdptx-phy 编入。

13.6 主机与板级验证

主机与板级两侧的证据现已齐备。Stage 0 为 Qt 在 QEMU 上的渲染：日志显示 linuxfb 平台插件加载成功，完整经历 QT_DEMO_STARTED 至 QT_DEMO_PASSED，客户机为 Alpine v3.23 配 qt6-qtbase-6.10.3-r0 (与构建容器 ABI 对齐)，时钟为 82 KB 的 aarch64 musl 动态 PIE，运行

于原生 arm64 的 qemu-system-aarch64 11.0.2；为使 Qt 就位，离线安装了 109 个包共 276 MiB，并把 rootfs 由 2048 MiB 扩至 3072 MiB。在此基础上，2026-08-12 于 OrangePi-5-Plus 上完成整栈上板点亮：Stage 0 至 Stage 4 已在 RK3588 上点亮验证通过，HDPTX 的 ROPLL/PHY 锁定 (HDPTX-GRF STATUS=0x0e, pll_lock、clk_rdy、phy_rdy 全置位)，VOP2 VP0 输出 1080p60 光栅并实时扫描，注册为 /dev/fb0 并绘出彩条，glibc Qt6 时钟在该原生显示栈上运行通过。板级判据（两次 HDPTX-GRF ROPLL 锁定轮询读出 STATUS=0x0e，辅以一次 VOP2 寄存器转储）全部通过。

上板之前的对抗式复核发现并修复了两个真实的 **HIGH** 缺陷，均在任何板级运行之前完成。其一为帧缓冲缓存一致性：扫描输出的帧缓冲起初分配为 ContiguousArray（带缓存的流式内存）且缺少缓存维护，又与非缓存的 /dev/fb0 mmap 别名同一段物理内存，在 ARMv8 上构成可缓存性不匹配，会把陈旧的 RAM 内容送上面板，改为 CoherentArray（非缓存）后消除。其二为 config-done 锁存：cfg_done_value() 遗漏了缺乏写掩码保护的 GLB_CFG_DONE_EN (BIT15)，裸写会将其清除，使影子至活动的锁存被禁用，此后每次重指都静默失效，补成 0x18001 后修复。对抗式复核共进行四轮：第一轮 19 个代理在分支上提出 23 项，收敛为 3 项确认、8 项判为误报；时序再推导由新代理复核 6 项寄存器全部匹配；modeset 再核对由第二名新代理确认全部匹配、含两处次要修正；第二轮针对冷启动胶合层的 5 项发现中，1 项 HIGH 经核验为误报、3 项修复、1 项记为文档说明。经核验为误报的那项声称冷启动的 PHY 复位 id 解码到了错误的 CRU 寄存器，实际上厂商寄存器编码的 id (0xc003b 至 0xc003d) 经 +0x30000 的 PHP-CRU 模型正确解码到 php_softrst_con(3) 的 11 至 13 位，已记录以免再次误报。

开发过程中另有若干路线更正。片上直接安装 qt6-qtbase-dev 会膨胀到 250 余个包而不可行，故改为在与分支匹配的 Alpine 容器中交叉编译时钟并只投递二进制；一处“27 errors”安装失败根因为 rootfs 满（空闲 135 MiB 而需 276 MiB），扩容至 3072 MiB 后解决；apk 在字体配置触发器的良性失败下仍返回非零，故把成功判定门由退出码改为 libqlinuxfb.so 产物是否存在。此外，落后于 upstream/dev 的主检出多次带来模式漂移，plat_dyn 字段已被移除、ax-feat/ 更名为 ax-runtime/、RockchipPM 现由 rdif_power::Power 包裹（使裸 get_one::<RockchipPM>() 的上电成为静默空操作，相应死代码已删除），均已随之校正。

就技术细节据实说明如下。PHY 的 193 个模拟微调值、全部 reset/GRF/MMIO 常量以及整套时序均为逐字对齐主线的移植，其在板端的正确性由前述两次 HDPTX-GRF 锁定轮询与一次 VOP2 寄存器转储确证。板载 DTB 中 route_hdmi0 标记为 disabled，冷启动据此采用完整的原生初始化路径，由驱动自行依序拉起整条流水线。当前作用域为 1080p60，DDC/EDID 与热插拔属后续功能范围。该显示栈是一套完整、已上板点亮验证的原生显示子系统：主机寄存器级测试 39 项通过，板级 HDPTX ROPLL 锁定与 VOP2 VP0 的 1080p60 光栅点亮验证通过；约 24 至 25 个提交暂存于工作树分支，待整理为独立 PR 回馈上游。

14 图形合成器栈：Weston 于 StarryOS

Wayland 是取代 X 窗口系统的显示服务器协议，其中的合成器是介于内核与各图形应用之间、独占显示硬件的进程：它从各客户端收集绘制好的窗口缓冲，叠合为一帧整屏画面交给显示设备，同时把键盘、鼠标与触摸输入分发回对应窗口。让一个完整的合成器在一套新内核上启动并接入真实客户端，等于一次性把 Linux 图形与进程间通信栈的一大片纳入内核的支撑范围：它

对内核的要求横跨显示输出、输入设备发现、套接字传递文件描述符与事件循环四个彼此独立的子系统，任一处不合规整套界面即无法立起。StarryOS 此前不具备承载这类合成器所需的内核接口，图形应用因而无从运行。现已在 **StarryOS** 上完整启用 **Weston 14** 图形合成器，Weston 是 Wayland 的参考实现，它以 DRM 加 pixman 软件后端启动、接受客户端连接、经 GTK4 客户端与 VNC 呈现界面，并在四种体系结构上通过自动化判定。本节先结合图 23 说明合成器压在内核上的四个子系统及其机制，再说明逐层的内核启用与修复，最后给出验证。

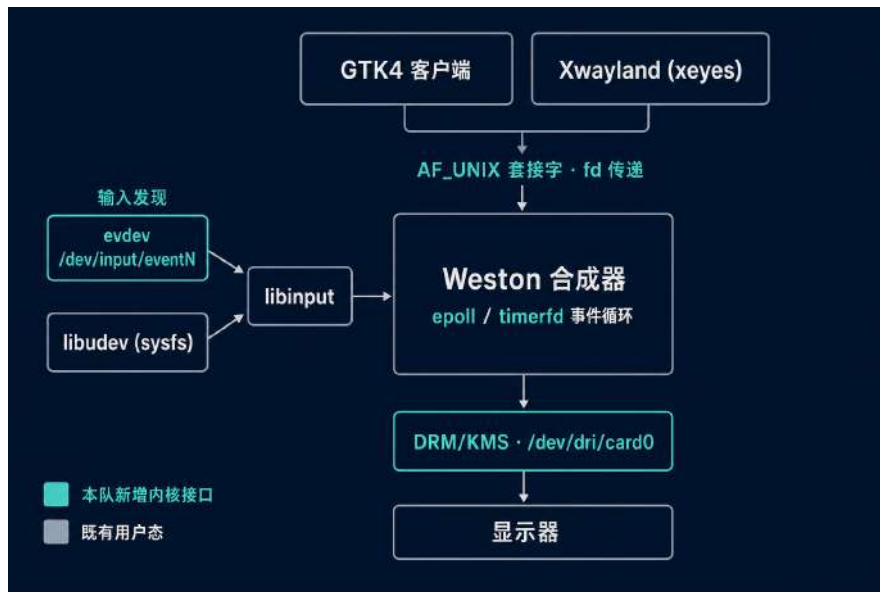


图 23: Weston 于 StarryOS: 客户端经 AF_UNIX 套接字连入并提交窗口缓冲，输入沿 libinput 与 libudev 自 sysfs 与 evdev 发现，DRM/KMS 承载扫描输出，整套事件经 epoll 与 timerfd 汇聚轮转

14.1 合成器压在内核上的四个子系统

Weston 的四个子系统各自对应一组内核接口，以下逐一说明其机制。

显示输出 DRM/KMS 显示后端 `drm-backend` 经 `libdrm` 操作 `/dev/dri/card0`。它要协商设备能力、枚举 KMS（内核态显示模式设置）资源、分配显示缓冲，并以原子方式提交一次模式设置与翻页。模式设置须兼容传统的 `SETCRTC` 路径与原子化的 KMS 两种语义，承载像素的显示缓冲则由 GEM dumb 缓冲提供，合成器把叠合好的整屏画面写入这块缓冲后交由显示控制器扫出。

输入发现 evdev 与 libinput 输入由 `libinput` 经 `libudev` 沿 `sysfs` 遍历发现 `/dev/input/eventN` 节点，再以一组 `evdev ioctl` 查询各设备能力并读取事件。其中 `EVIIOCGPROP` 用于取回设备属性、`EVIIOCGABS` 用于取回绝对轴信息，二者是 `libinput` 判定一个节点是否为可用输入设备、以及该设备属键盘、指点还是触摸的依据；缺此则该节点被静默跳过。

协议传输 AF_UNIX 与文件描述符传递 协议服务由 `wayland-server` 在一个 AF_UNIX 流式套接字上监听，接受客户端连接并接收其提交的窗口缓冲。其中最关键的是随消息原子地传递文件描述符：客户端把承载像素的共享内存以文件描述符的形式经辅助控制消息交给合成器，套接字须在字节流中以正确的标记原子地携带这一控制消息，缺此则合成器无从取得客户端的窗口缓冲。

共享内存一侧还依赖 memfd 的密封语义，F_SEAL_SHRINK、F_SEAL_GROW 与 F_SEAL_WRITE 分别封住缩小、增长与写入，令合成器得以安全映射客户端提交的缓冲而不虞其被抽换。

事件循环 epoll 与 timerfd 合成器靠 epoll 与 timerfd 把上述三路事件汇入一个统一的事件循环轮转。epoll 多路复用显示、输入与套接字三类文件描述符的就绪状态，timerfd 则把定时器表达为一个可被 epoll 等待的文件描述符，用于驱动重绘节拍与各类超时。这一层要求就绪通知与实际就绪的数据严格一致，否则事件循环会在虚假就绪上空转。

14.2 逐层的内核启用与修复

此次适配遇到的问题落在三个深浅不同的层次，内核改动据此逐层推进。最浅的一层是接口不存在，打开即以明确错误码回绝。系统内核侧补上了 /dev/dri/card0 节点，兼容 SETCRTC 与原子 KMS 两条模式设置路径 (#506)，并以按缓冲粒度分配的 GEM dumb 缓冲与带引用计数的 mmap 承载像素 (#514)；输入侧如实暴露多个 eventN 设备并实现 EVIOCGPROP 与 EVIOCGABS (#513)，设备发现所依赖的 AF_NETLINK 上 rtnetlink、uevent 与 genl 几路也一并补全 (#512)，以上均已合并。

第二层是接口看似齐备、编译与打开均成功而行为错误且不报任何错。sysfs 的 /sys/class/input/ 须是指向真实设备路径的符号链接，libudev 靠 realpath() 与 dirname() 沿链接回溯，若做成普通目录回溯即中途断掉；/dev/input/eventN 的次设备号须从 EVDEV_MINOR_BASE (64) 起编，若从 1 起编，libinput 以 fstat() 的 st_rdev 反查 /sys/dev/char/ 便对不上号。这些隐式约定集中在一处修订内补齐：sysfs 类目录改为符号链接、evdev 次设备号改从 64 起编、并预先在 /run/udev 下播下 udev 的初始化种子文件，libudev 方肯认定设备已配置 (#508，已合并)。memfd 的密封检查被分布到截断、映射与写入各条路径，缺一处密封即形同虚设 (#507 与 #515，已合并)。Weston 的整体启动收尾、把输入接到中断唤醒以压下延迟、以及让 AF_UNIX 流式套接字按字节标记原子地携带控制消息送达文件描述符，同在一处合并完成 (#509，已合并)。这一层的失败是沉默的，libinput 只报一句跳过未配置的输入设备，要查出真因须回头去读用户态库的源码。

最深的一层是并发与性能，它不挡启动却使界面难以使用。epoll 的水平触发注册无条件唤醒等待方，造出每秒约 500 次的空转忙循环；输入若退回内核节拍驱动会带来约 16 毫秒的输入延迟，在单核上还叠加中断上下文中的锁争用。系统内核侧把 epoll 的水平触发改为如实排干就绪事件，收敛了那条 500 赫兹的空转 (#504)，并补上 timerfd 这一计时事件源 (#503)，二者均已合并。输入延迟已由中断唤醒压下 (#509)，单核上中断上下文中的锁争用仍是有待优化的残留限制。

14.3 验证

这套支撑经端到端验证为可用。Weston 以 DRM 加 pixman 后端启动后，一个客户端可成功连接，一个 GTK4 程序能把界面绘制出来并经 VNC 远程查看，一套自动化用例产出 WAYLAND_TEST_PASSED 判据，在四种体系结构上一致通过 (#1160，已合并)。为防回归，另引入以黄金帧作视觉比对的持续集成哨兵守护该能力不回归。Xwayland 亦成功运行 xeyes 这个经典 X 程序，使既有的 X 应用得以在 Wayland 之上运行 (#516，已合并)。该图形栈的启用与视觉感知流水线相互独立，为设备侧的实时监控与可视化界面提供了用户侧通路。

15 QEMU RKNPU 功能级仿真与模型结构协同优化

RKNPU 是 RK3588 上的神经网络加速器，其寄存器接口、命令编码与算子语义均无公开文档，RKNN 运行时以闭源二进制驱动它。此前对这块加速器的驱动 bring-up、RKNN 的持续集成与模型结构调优只能在开发板上进行，每一次改动都要占用一台真机，逐算子的成本也无从读出。功能级 QEMU 设备模型在模拟器中重建这块加速器的寄存器、命令流水线与算子执行，在无板环境下算出与真机一致的计算输出，把闭源加速器变成一个寄存器精确、可进 CI、可逐算子度量的开发与优化载体。此非一个仅接受写入而不计算的桩：模拟器真实执行 INT8 与浮点卷积并回写正确张量。仿真侧现已实现完整的功能级 **RK3588 RKNPU 模拟器**，以两个合并请求（#19、#26）合入上游 QEMU，配套 106 个 qtest 回归，接入 Rockchip IOMMU，并支持三核协同会合；网球检测模型的结构协同优化在该模拟器上完成。本节先说明 RKNPU 与功能级仿真的必要性，再逐块说明命令流水线、IOMMU 接入、算子执行、回归验证与多核协同的机制与实现，最后给出基于该模拟器的模型结构协同优化。本节的每一个模型数字均为该模拟器的逐推理实测，属结构成本与运行时计数口径，非真机读数。



图 24: 功能级 RKNPU 模拟器的分层结构，青色为仿真侧新增的设备模型

图 24 给出该模拟器的分层结构。既有的部分是 QEMU 主线提供的设备模型框架、标准 IOMMU 的 MemoryRegion 翻译接口、迁移状态机制与 qtest 基础设施。新增的部分是 RK3588 RKNPU 的设备接入（MMIO、中断、复位、设备树绑定）、命令流水线各功能块的寄存器与状态机、真实的算子执行路径、多核会合逻辑，以及为模型优化服务的逐推理 profiling 计数。RKNN 运行时二进制在模拟器上未经改动直接运行，模拟器对齐其可观察的寄存器与命令行为到足以让整条推理链正确执行。

15.1 RKNPU 与功能级仿真的必要性

RKNPU 由一条命令驱动的定点/浮点计算流水线构成，RKNN 运行时把编译好的模型翻译成一串寄存器写入与命令表，提交给加速器执行卷积、激活、池化等算子并回收输出张量。由于寄存器布局与命令编码闭源，任何脱离开发板的开发都无从下手：驱动无法在 CI 中回归，模型改动

无法在无板环境下验证，逐算子的成本更无法读出。功能级仿真的价值在于同时满足两点。其一是寄存器精确，模拟器复现命令流水线的可观察行为，使未经改动的 RKNN 运行时能在其上跑通整条推理链。其二是功能正确，模拟器在维护寄存器状态之外据命令算出真实的 INT8 与浮点输出，因而可用作精度评估的参考。二者合起来使这块加速器可脱离开发板被调试、被度量、被优化。

15.2 设备接入与命令流水线

模拟器以 `-machine rk3588-evb,rknpu=on` 启用，机器随之发出配套的设备树绑定，向访客暴露 RKNPU 的 MMIO 区、中断线与复位。命令处理沿加速器的功能块展开：RKNN 运行时把张量与权重的地址、算子参数与协同模式写入各块的寄存器，再以命令表把任务串成任务链提交，模拟器按块推进状态机，从核提交、寄存器待决状态、复位与迁移状态一并建模。各功能块的职责如下。

PC 程序控制器 PC 是命令流水线的入口，负责从命令表取出任务、解出各算子的寄存器配置并驱动其余各块，同时维护任务链与待决寄存器状态。模拟器重建这条取指与派发路径，使一次推理对应的整串任务按序推进。

CNA 卷积前端 CNA 负责卷积的输入组织，把特征图与权重按卷积核的取数模式喂入计算核，并承载协同模式的配置位。模拟器复现其取数几何与配置寄存器，其中协同模式位（见多核协同一节）即由 CNA 的控制寄存器解出。

CORE 卷积计算核 CORE 是乘累加阵列，对喂入的特征与权重做卷积与深度卷积的乘累加。模拟器在此真实执行定点与浮点的乘累加，覆盖 INT8、量化 INT8 与 FP16/FP32，算出的部分和交由后处理。

DPU 后处理 DPU 承接卷积输出，完成激活、量化、逐元素运算与查表（LUT）等后处理。激活函数中形如 SiLU 的非线性会被编译为查表算子在此执行，模拟器复现 LUT 与逐元素路径的功能行为，这一点直接关系到下文模型优化中激活函数替换的成本变化。

DPU-RDMA 读通道 DPU-RDMA 是后处理块的读 DMA，将待处理张量从内存读入 DPU。模拟器建模其读取几何与面选择，多核路径下的双端口面选择与有效面缺口布局在此处理。

PPU 池化 PPU 负责池化与平面处理。模拟器复现池化算子及其输出布局，与卷积、后处理一同构成完整的算子集合。

15.3 IOMMU 接入

RKNPU 经 Rockchip IOMMU 访问访客内存，模拟器通过 QEMU 标准 IOMMU 的 `MemoryRegion` 接口接入这条翻译路径。加速器发出的每一次 DMA 都经 IOMMU 做地址翻译与权限检查，翻译失败或权限不足时如实上报 DMA 翻译错误，不会静默访问宿主内存。此接入使模拟器可验证驱动侧的地址映射是否正确，也把畸形地址的越界访问挡在翻译层。

15.4 算子的功能执行

功能执行路径是模拟器中难度最高的一环，模拟器在维护寄存器状态之外据命令算出真实输出。执行覆盖 INT8、量化 INT8 与 FP16/FP32 三种数值路径，算子覆盖卷积、深度卷积、逐元素、查表、池化以及张量布局转换。量化 INT8 路径按加速器的定点约定做乘累加与再量化，浮点路径按 FP16/FP32 计算，查表路径复现激活函数编译所得的 LUT 语义，布局转换复现加速器在各块之间对张量摆放格式的重排。因输出与真机一致，该路径既支撑驱动 bring-up 的正确性验证，也可直接作为模型精度评估的参考基准。为拦截畸形或超大的访客负载，模拟器对宿主内存占用与单次算子规模设定了可配上限，越限即被安全地拒绝而不致耗尽宿主资源。

15.5 QTest 回归

模拟器附带 106 个 QTest 子测试，即机器可判定的判定门，一次运行即回归整套设备行为与算子正确性。覆盖面包括 MMIO、复位、IRQ 与 IOMMU 行为，命令解码与任务生命周期，整数与浮点执行的数值正确性，DMA 错误与不支持控制字的处理，任务链，迁移状态，以及 DPU-RDMA 与 PPU 的布局。这套回归使模拟器的每一处行为改动都可自动核验，是把它用作 RKNN 的 CI 与精度参考的前提。上述设备接入、IOMMU、算子执行与 QTest 以合并请求 #19 合入上游 QEMU，已合并。

15.6 多核协同

RK3588 的 RKNPU 由三个物理核构成，卷积任务可按协同模式在一至三核上并行并在完成边界会合。模拟器对照真实硬件逆向出这套协同行为并加以复现。协同模式自 CNA 的 CONV_CON2 寄存器 [30:28] 位解出：模式 0 为单核，模式 1 与 4 为双核，模式 2 与 5 为三核，模式 3、6、7 另有区别。每个 NPU 核在模拟器中运行于独立的宿主 worker 上，各自推进本核的任务，在协同完成边界处同步。逆向所依据的硬件证据一并记录：双核模式 4 等待 core0 与 core1，三核模式 2 与 5 等待 core0 至 core2；参与核数相同的模式即便模式值不同也在同一点会合；各核本地命令表的任务下标无须跨核对齐；只要物理核集合与参与核数属同一类，独立分配的命令链与输出缓冲亦满足同一硬件会合条件。扩展至多核时一并修正了若干数值正确性问题：INT16 unpool 的几何与存储映射、FP16 DPU-RDMA 的双端口面选择与有效面缺口布局、通道分组下 FP16 深度卷积权重的紧凑索引，以及跨步输出回写仅落在有效输出通道面上。这部分以合并请求 #26 合入上游 QEMU，已合并。下文模型优化所依赖的逐推理 profiling 计数由一个在审的合并请求提供。

15.7 基于模拟器的模型结构协同优化

模拟器暴露的逐算子成本计数，使网球检测器得以逐层修改结构，每一步改动都可读出 MAC、命令字、任务取指、LUT 任务数与逐段耗时的变化，这是板级黑盒跑分无法提供的信息。度量口径固定为 RKNN 运行时 2.3.2，精度在 285 图 / 382 框的保留集上按 INT8 评估，置信度 0.25，NMS IoU 0.45。表 12 给出累计的结构成本，每一行均在上一行基础上再叠加一层改动。

阶梯步骤	MAC (GMac)	RDMA 输入 (M)	任务取指	命令字 (k)	LUT 任务	逻辑输出 (KiB)
原始 640×640 SiLU	3.15	27.21	443	95.15	107	541
输入缩放 480×640	2.36	20.41	422	92.42	86	406
改用 ReLU	2.36	19.60	502	43.56	3	406
去掉 score-sum (9→6 输出)	2.36	19.50	484	42.62	3	400
类别头 64→32	2.07	19.09	484	42.62	3	400
头框合并 (隐藏 32 / reg_max 8)	2.06	18.89	484	42.62	3	203

表 12: 模拟器实测的累计结构成本, 逐步压缩 MAC、命令字与逻辑输出

对全链路成本影响最大的一步是激活函数的替换。原始 640×640 模型采用 SiLU, SiLU 在 DPU 上被编译为大量查表算子。输入缩放后单次推理含 86 个 LUT 任务, `rknn_run` 为 12.31 ms; 替换为 ReLU 后 LUT 任务降至 3 个, 命令字由 92.42k 降到 43.56k, `rknn_run` 由 **12.31 ms** 减半至 **5.79 ms**。其余各步分别修正一处结构冗余。输入由 640×640 调整为模型原生的 480×640, 节省 RDMA 输入与 MAC。去掉冗余的 score-sum 将 9 路输出并为 6 路。类别头由 64 收窄至 32。检测头与框回归头合并, 隐藏维取 32, `reg_max` 取 8, 逻辑输出随之由 400 收窄至 203 KiB。上述改动累计将 MAC 由 3.15 压缩至 2.06 GMac, 逻辑输出由 541 收窄至 203 KiB, RDMA 输入与命令字一并下降。

精度在整条阶梯上保持稳定。如表 13, mAP50 由 92.9% 变化至 91.1%, 中途最低降至 89.5%, mAP50-95 略有上升, 由 69.9% 至 70.6%; Precision 由 93.8% 至 94.8%, Recall 由 91.6% 至 90.3%。耗时方面 `rknn_run` 由 14.66 ms 降到 5.68 ms, `outputs_get` 由 0.286 ms 降到 0.125 ms, `postprocess` 始终保持在 0.03 ms 以下。以可忽略的精度代价把单次推理的运行耗时压缩到约三分之一 (`rknn_run` 14.66→5.68 ms), 加速器 MAC 降至约三分之二 (3.15→2.06 GMac)。

阶梯步骤	mAP50 (%)	mAP50-95 (%)	rknn_run (ms)	outputs_get (ms)
原始 640×640 SiLU	92.9	69.9	14.655	0.286
输入缩放 480×640	91.0	69.7	12.314	0.251
改用 ReLU	89.5	70.0	5.786	0.250
去掉 score-sum	89.5	70.0	5.839	0.157
类别头 64→32	90.4	71.6	5.643	0.154
头框合并	91.1	70.6	5.677	0.125

表 13: 同一阶梯上的 INT8 精度与逐推理耗时, 精度基本持平而 `rknn_run` 降至约三分之一

本节的 MAC、LUT 任务数、`rknn_run` 与 mAP50 均为 QEMU RKNPU 功能模拟器的逐推理实测, 属结构成本与运行时计数口径, 非真机测量。优化后模型在开发板上的端到端首帧与逐帧时延由网球应用的板级运行给出, 见启动与推理相关各节。模拟器本身是一项可复用的产物, 使这套逐算子的成本与精度分析, 连同 RKNN 的 CI 与驱动 bring-up, 均可脱离开发板进行。

16 板级 I/O 与外设驱动

板级 I/O 与外设驱动是 StarryOS 直接驱动 RK3588 板载外设的一层。SoC 把 UART、USB 主机控制器、PWM、网络控制器与引脚复用等外设都映射为内存中的寄存器块, 这一层通过 MMIO



图 25: 模拟器实测: 网球模型结构协同优化把 MAC 3.15→2.06 GMac、LUT 任务 107→3、rknn_run 14.66→5.68 ms, 同时 mAP50 保持在 92.9→91.1%

对这些寄存器块编程, 并以设备节点与 sysfs 属性把它们交给用户态, 从而为机器人的感知取流与执行器控制提供硬件通路, 也为上板调试提供控制台、网络与重启原语。本报告中每一项在真机上测得的数据, 都以这一层能在 RK3588 上稳定驱动外设为前提。两条板级现实决定了这一层的工作重心。其一, StarryOS 的用户 shell 一旦占用串口 tty, 内核的 warn! 与 info! 便不再抵达控制台, /dev/kmsg 又仅可写入, 第二条登录通路 (SSH) 因此成为硬性前提。其二, 经 1.5 Mbaud 串口以 ymodem 刷写约 14 MB 的 FIT 映像长期不可靠, 反复出现 Error: NAK 损坏, 一夜运行中 8 次尝试全部失败, 而任何一次物理下电都会重新枚举 USB 串口适配器, 并可能在 8 核启动时导致电源欠压。两者共同要求一套以网络为先、暖重启为主的上板流程, 这套流程又依赖可用的 reboot 系统调用、UART 与 USB 串口, 以及板载网络。本节先说明这一层的构成与既有骨架, 再逐组说明各外设驱动的机制以及既有与新增的分界, 最后给出使能效果与刷写通路的实测。

16.1 子系统构成与既有骨架

StarryOS 与 ArceOS 此前已提供这一层的通用骨架。tty/pty 子系统提供字符设备到串口硬件之间的行规程 (line discipline), 负责规范模式下的行缓冲、回显与信号字符, 并以等待队列在缓冲区为空时挂起读者、在字节到达时将其唤醒。USB 主机栈与 UVC 类驱动框架负责设备枚举、端点管理与传输请求的提交回收。sysfs 提供以属性文件暴露内核对象的通用模型。ax-net 提供网络协议栈与套接字接口。pinctrl 负责按引脚号把功能选择写入 IOMUX 寄存器。本队在此骨架上补齐 RK3588 的具体外设驱动, 并修复其中若干时序、锁与引脚路由缺陷, 使骨架在真机上可用。整层按功能分为四组: 控制台与执行器、双系统切换的重启原语、相机取流前端、从网卡上电到 SSH 的使能链。以下逐组说明, 并标明各组中既有骨架与新增实现的分界。相关工作归本队板级侧, 除引脚路由外均已在 rcore-os/tgoskits 合入。

16.2 控制台与串口: UART 与 USB 串口 tty

串口控制台是其余一切上板操作的前提。RK3588 的 UART 是一组 DW_apb_uart 兼容的 MMIO 寄存器块, 含发送保持寄存器、接收缓冲寄存器、线路状态寄存器、FIFO 控制与波特率分频寄存器。RK3588 UART 初始化 (#1921) 完成串口资源的建立: 映射寄存器窗口、按源时钟设置分频以得到目标波特率、配置 FIFO 与线路控制、开放收发。这一步构成串口控制台的地基, 此前缺少对 RK3588 UART 资源的初始化, 控制台无从建立。

调试 UART 之外，第二条控制台通路来自 USB 串口。USB 串口 tty (#1378) 在 USB 主机栈枚举出串口适配器后将其绑定，以批量 IN/OUT 端点承载串行字节流，并把它作为一个 tty 暴露为 /dev/ttyUSB 设备。这条通路承担两件事：其一是提供第二个控制台，其二是机器人机械臂的串口控制器经此接入，机械臂在 /dev/ttyUSB 上呈现为一个 tty，用户态经该 tty 下发臂的动作指令。

交互式 shell 的可用性取决于 tty 输入路径的正确。TTY 输入刷新与读者唤醒修复 (#1922) 补齐了 TCFLSH 对输入队列的刷新，并消除了一处丢失唤醒：阻塞在 tty 上的读者在缓冲区为空时睡在等待队列上，输入路径每收到一个字节都应将其唤醒，此前某些情形下字节已入队而唤醒未发出，读者在尚有待读数据时继续睡眠，shell 因此挂起。修复后唤醒与数据到达严格对应，交互式 shell 自此可用。以上三项均已合并。

16.3 执行器控制：PWM 的 sysfs 接口

执行机构的控制面来自 PWM。PWM 以可编程的周期与占空比生成周期性方波，输出的平均电平由占空比决定，据此可设定舵机转角、电机转速与风扇转速。RK3588 的 PWM 控制器是一组 MMIO 寄存器块，含周期寄存器、占空比寄存器与控制寄存器，由源时钟驱动，编程时写入以时钟周期计的周期计数与占空比计数。PWM 的 sysfs 接口 (#1468) 按 Linux 的惯例在 /sys/class/pwm 下以 period、duty_cycle、enable、polarity 等属性文件暴露该控制器，用户态写入这些文件即可驱动执行器。这是机器人底盘电机与舵机的支撑点，已合并。

16.4 双系统切换：reboot 系统调用

暖重启回路依赖一个可用的重启原语。reboot(2) (#1358) 在系统调用层实现 Linux 语义的 reboot：调用方传入两个魔数与一个命令码（重启、关机、下电等），内核据此触发系统复位。在 RK3588 上，复位经固件的 PSCI SYSTEM_RESET 调用落到硬件。有了这一原语，StarryOS shell 中一条 reboot -f 即可复位开发板并进入下一阶段，物理按键不再必要。这是暖重启回路成立的前提：StarryOS 经 reboot 进入 Linux，Linux 经 SSH 触发 reboot 进入 U-Boot，U-Boot 以 fatload 载入预置的 FIT 映像，两系统之间的切换全程不触碰电源按键。暖重启之所以关键，在于前述物理下电对 USB 串口重新枚举与 8 核上电欠压的影响。该系统调用已合并。

16.5 视觉取流前端：UVC 相机异步传输

USB 相机是整条视觉流水线的源头。UVC 是 USB 视频类标准，相机的帧经 USB 端点以流式方式传入：主机向端点提交一批传输请求，硬件在帧数据到达时逐一填充这些请求，完成回调将数据上交并立即重新提交请求以保持管道持续满载。这一提交、完成、重提交的循环即异步传输生命周期。UVC 异步传输生命周期修复 (#1924) 纠正了这一循环中的时序错误：生命周期处理不当会使已完成的请求未被重新提交，或在停止取流时状态错乱，取流因此停止或数据错乱。相机既是流水线的源头，这一修复对整条网球流水线直接承重。其后将 UVC 测试迁移至 ax-sync (#1961)，使相机测试路径改用正确的同步原语，消除测试中原有的同步语义错误。两项均已合并。

16.6 从网卡上电到板上 SSH

板上 SSH 取代了此前对单一串口通道的依赖，是上板迭代调试得以进行的关键。SSH 的成立取决于一条使能链，链上任一环缺失均无法登录。

链的底端是引脚路由。pinctrl 按引脚号将功能选择写入 IOMUX 寄存器，每个引脚的复路由 GRF/IOC 寄存器窗口中的若干比特决定，驱动需据引脚号与所在 bank 算出目标寄存器与位偏移。RK3588 的 pinctrl 原先误算了 IOMUX 的寄存器窗口偏移，Starry 写入了错误的窗口，GPIO、regulator 与外设的引脚复用因此错位。修正偏移计算后引脚复用方才正确，这一步是使能链的前置条件，现已实现并在板上验证，正作为独立 PR 整理中。

其上是网卡的上电、枚举与地址分配，由 rtnetlink 一组改动（#1497）整合到位。网卡的电源轨由一个固定电压 regulator 门控，其使能是一个 GPIO，设备树声明该 GPIO 为 active-low，驱动须将其驱动至断言电平方能给电源轨供电，极性写反则网卡不上电；该组改动修正了固定 regulator 的 active-low 极性，使电源轨上电。其次，板上有多颗 RTL8125 2.5G 网卡，多网卡枚举将每一颗暴露为一个网络接口。最后，rtnetlink 是内核与用户态之间基于 netlink 的网络配置接口，实现其 IPv4 地址操作后，用户态得以经 ip addr 为接口分配地址。三者串起，电源轨上电、端口枚举、地址分配依次到位，配合 TTY 修复（#1922），脆弱的单一串口通道被替换为真正的登录，从网卡上电到 SSH 登录的整条链路在板上闭合。

网络路径另有一处正确性修复。ax-net 数据报锁修复（#1963）避免了在数据报自旋锁下对互斥锁执行睡眠：持自旋锁时睡眠在原子上下文中不合法，负载下会引发错误睡眠，修复令数据报路径不再在持锁时睡眠于互斥锁。除引脚路由外，本组改动均已合并。

16.7 使能效果与刷写通路

这一层交付的是板级与外设的 bring-up，其效果集中体现在两处。其一，板上 SSH 登录取代了此前对单一串口的长期依赖，配合 reboot 原语构成不触碰电源按键的暖重启回路。其二，映像刷写从不可靠的串口 ymodem 转为以网络为先的通路。表 14 给出三条刷写通路的实测用时。

表 14: 三条映像刷写通路的实测用时

通路	用时	说明
网络 scp FIT 至 /boot	约 1 s	md5 校验一致，替代失败的 ymodem
串口 ymodem (1.5 Mbaud, 约 14 MB FIT)	90 s 以上, 频繁失败	间歇 Error: NAK, 一夜 8 次尝试全部失败
U-Boot fatload 预置 FIT 后 bootm	约 1.4 s	相较 ymodem 的约 5 分钟

网络 scp 至 /boot 再经 fatload 与 bootm 启动的通路已在板上跑通：载入修复后的内核，md5 校验一致并完成自检。这一层不含对 Linux 的性能主张，其内容为板级与外设的 bring-up。相机取流（#1924/#1961）与 PWM 控制面共同构成机器人的感知与执行通路，感知前端的模型结构优化与首次推理时延各自单独成章。本层的定位是把 RK3588 开发板变为可 SSH、可暖重启的实用目标，其余章节相对 Linux 的持平与领先方由此得以在真机上被测得。

第四部分 方法、分工与展望

17 评估后未采用的尝试与适用条件

本节汇总一批经板级 A/B 对照评估、在当前平台未见收益因而未采用（已回退）的尝试，覆盖调度、内存、启动、频率与 NPU 五处路径。每一项都在开发板上完成 A/B 对照，有收益的并入正文，无收益的连同其原因如实记录于此。明确记录这些尝试，既为当前平台划清了边界，也便于在条件具备时直接复用而无需重走弯路。多数尝试在更换硬件类型或补足某项前置条件后有望见效，其适用条件随条目一并列出。

下表按调度与内存、启动与频率、NPU 三组，列出各项尝试的现象、原因与适用条件。

尝试	现象	回退原因	适用条件
全量 work-stealing 负载均衡器 (idle-pull、push 与 wake-redirect)	单线程吞吐减半；八线程饱和时与关闭时基本持平	七个空闲核心上，三条迁移路径反复搬动唯一的工作线程，无核心能维持缓存热度；大小核架构上，分离一对热任务与冷独立任务保持缓存热度两个目标直接冲突。已回退，运行期迁移仅以默认关闭的 feature 保留	核心众多、任务长期饱和、迁移成本相对计算量可忽略的同构机器，或补足按缓存热度与容量设定的迁移门控之后
自适应 haltpoll	唤醒延迟相较固定时长轮询未见稳定优势	停机时长信号受 GIC-SGI 与 WFI 交互的异常特性干扰，判据失真。已回退为固定时长自旋 (idle-poll) 后再进入 WFI；idle-poll 现随并发工作线程的唤醒铺开在出厂放置配置中启用	平台停机时长信号可信、且无 SGI-WFI 干扰时
按需缺页拷贝 (去掉 fork 前的 populate_area)	fork 密集的 -P 负载显著变慢，线程路径仅获小幅收益	-T 线程路径的瓶颈源自并发度，持锁时长与缺页模型均不构成主因。已回退	缺页并发不构成瓶颈、且惰性建立映射能显著节省未触碰页开销的场景
early-MMU 启动重排 (提前开启 MMU 与缓存)	kernel 到 shell 用时与未改动基本相同	启动早期的未缓存窗口在启动低频下受限于 CPU 计算，对计算密集工作启用缓存不省时间。已回退，保留启动计时诊断	该窗口由内存延迟主导，或启动核心已运行于高频时
探测让步延时 (yield 式探测等待)	并发设备探测耗时增加，整机启动更慢	tick 粒度将众多小等待分别向上取整至一个 tick，逐条放大每条并发探测。已回退	单次等待远长于一个 tick，或改用高分辨率定时器进行让步时
AVS/PVTM 顶频抬升	A76 顶档上限无法提升	此为固件与硬件的外部约束：顶部 A76 OPP 全部锁定于固定电压，无 AVS 分箱，主线亦无对应的 Rockchip AVS 驱动。真正提升顶频的机制是 PVTPLL 环长切换，已在正文采用：A76 治理上限 2126 MHz (保留待温控可至 2256 MHz)、A55 1881 MHz，逐核约达 Linux 的 0.98，见频率电压一节。AVS 分箱抬升在当前芯片配置下暂不适用，已回退	芯片提供非空 AVS 分箱、且存在可读取分箱电压的驱动时
NPU 张量零拷贝 (把张量重排移到用户态)	端到端推理时间未降反升	单核推理下仅改变重排的执行位置，重排开销未减少，且新增一次搬运。已回退	重排能真正并行至另一核心，或交由硬件承担时
第三 NPU 核	由两核增至三核推理未见收益	当前模型负载约占用一个核心，增配核心无法充分占用。已保持两核	模型足够大、单核算力成为瓶颈时
NPU 定频 (锁住 governor)	工作点无变化	NPU 已稳定运行于其额定工作频率，锁频未改变实际工作点	NPU 频率原本会被 ondemand 治理压低或产生抖动时

表 15: 经板级 A/B 评估后未采用 (已回退) 的尝试: 现象、原因与适用条件

几条回退指向同一批平台事实。大小核架构上，分离一对热任务与令独立任务保持缓存热度两个目标天然冲突，全量迁移在七个空闲核心的场景下只反复搬动唯一的工作线程，这也是占用感知的放置仅在 spawn 与 wake 两处执行、运行期迁移默认关闭的原因。自适应 haltpoll 的不稳定与朴素唤醒铺开的高代价同源，均受制于该 SoC 毫秒量级的 reschedule SGI 唤醒 WFI 核心的异常特性；改以固定时长自旋的 idle-poll 抹平这一异常之后，唤醒铺开转为收益并随之出厂，粗粒度 wake_affine 仍把生产者消费者的唤醒延迟收在 8 μ s (Linux 6 μ s)，铺开机制见调度一节。NPU 三项暂未见收益同源单核已能容纳当前模型负载，改变重排位置、增配核心或锁频都不触及实际工作点。AVS 顶频受固件与硬件的外部约束所限，真正提升顶频的机制是 PVTPLL 环长切换，已在正文采用，见频率电压一节。这些结果不改变整体判断，反而明确了后续的着力方向，也标出更换平台类型或补足前置条件时应当重试的候选。

18 AI 辅助开发方法与正确性保障

本队采用一套以机器可判定的判定门为核心的 AI 辅助开发方法。方法把开发分工划为两侧：团队一侧确定架构方向、划定原子改动的边界、制定验收标准；AI 一侧在真机与 QEMU 的闭环内反复迭代实现，把每一次改动运行至判定门，由判定门给出通过与否的判定。其关键在于全部验收标准都落实为机器可判定的判定门，一次运行即产生一个可对比的数值，以该数值取代肉眼读取日志得出结论的环节。以往判断一次内核改动是否正确，依赖人工阅读串口日志逐条比对，既慢又易漏；现将正确性的判据固化为数值化的判定门之后，判定既快也可复核。

18.1 方法的架构

这套方法由三个部分构成：团队掌握的规格与验收、AI 在闭环内的实现迭代，以及横在两者之间的一组机器可判定的判定门。团队给出一次改动的目标、边界与验收阈值；AI 将实现运行至判定门，读取判定门返回的布尔判定与数值，据此继续迭代或收敛；判定门运行在板上或 QEMU 上，产出的数值同时用于放行改动与否定设想。图 26 给出这一双侧分工与中间的四类判定门。

判定门指一段能够自行判定对错的运行：给定输入，运行完整流程，将结果与同一条基线比对，输出一个表示通过与否的布尔量以及一个可追溯的数值。sysbench 与 schbench 是既有的上游基准，Linux 同机数据是既有的比对基线；将其组织为一次运行即给出放行判据的判定门，以及贯穿全部改动的评审与暖重启回路，是本队新增的部分。

18.2 四类判定门

第一类是 sysbench 对等回归门。每一次触及内存或调度的改动，都在板上重跑 cpu、threads、mutex、memory 的完整矩阵，与 Linux 同机数据逐轴求比值，任一轴掉出既定区间即判为失败。DVFS 的四步 OPP 提升依靠这道门逐步放行，每一步以 sysbench cpu 8 线程无压降为通过条件，最终 8 线程达到 4987 事件/秒，为 Linux 5222 的 0.96。

第二类是 schbench 延迟阈值门。唤醒延迟相关的改动完成后，以 schbench 测量 p50/p99 唤醒延迟，直接与一个数值阈值比对。跨核唤醒发送 IPI 的原有路径把 p50 抬到约 1042 微秒，wake_affine 改动完成后降至 8 微秒，接近 Linux 的 6 微秒，该数值即为放行凭据。

第三类是板上无死锁的多智能体对抗式评审门。触及 SMP 同步的改动落地之前，先拆为单一关注点的原子改动，再交由多个相互独立的评审智能体对抗式复核，每次改动配备 7 到 26 名



图 26: 团队掌握规格与验收、AI 在真机闭环内迭代实现，中间由四类机器可判定的判定门裁决

评审，正是这道门查出了两处高危缺陷。其一，DVFS 的 `apply_opp` 将一次电压写入失败静默吞没，可能在提升频率之后使核心停留于高频欠压这一存在硬件风险的状态，改为失败即中止之后，出错至多导致过压；其二，`perf` 组采样读路径的一处 `use-after-free` 在复核中被定位并修复。这类缺陷多数潜伏于 ArceOS 家族共享的跨内核 crate，能够通过随手测试而在 SMP 叠加 COW、THP 时损坏内存，页表与 TLB 一类的修复同时惠及全部三个系统。

第四类是 `s_frac` 计时正确性 oracle。CPU 时间记账既要快也要准。这道门以 `rusage` 的 `s_frac`，即系统态时间占比，作为可比对的正确性判据。一处遗漏的 `retick` 曾把被调度出去的睡眠时段计入 `utime/stime`，整秒乃至整小时的睡眠都被记为 CPU 时间；以一个无锁的 `resume_ns` 下界修复之后，`s_frac` 落在 **0.71**，接近 Linux 的 **0.72**，表明记账既准确又高效。

18.3 真机闭环与带外电源控制

这些判定门要产生价值，前提是能在真实 RK3588 上低成本地反复运行，且这条闭环须无人值守。`boardctl.py` 把常规一轮迭代收敛为暖重启：从 StarryOS shell 执行 `reboot -f` 返回 Linux，经 ssh 触发重启进入 U-Boot，以 `fatload` 载入部署的映像并 `bootm`，配合部署、同步、校验与实时失败监视，把一轮迭代压缩到秒级且无需断电。暖重启承担常规路径，但它有一处结构性盲区：只在被测系统仍能响应软件命令时才能恢复。这条闭环恰恰用于验证会令内核挂死的改动，SMP 死锁、早期 panic 与活锁都在其列；一旦某次改动在 shell、SSH 与 U-Boot 三者均不可达之前就挂死，暖重启无从恢复，回路只能停下等人到场物理断电。一套用于验证挂死的闭环，其恢复手段若本身依赖被测系统不挂死，便不成其为无人值守。

要让闭环真正无人值守，恢复手段必须独立于被测系统的软件状态。板级唯一具备这一性质

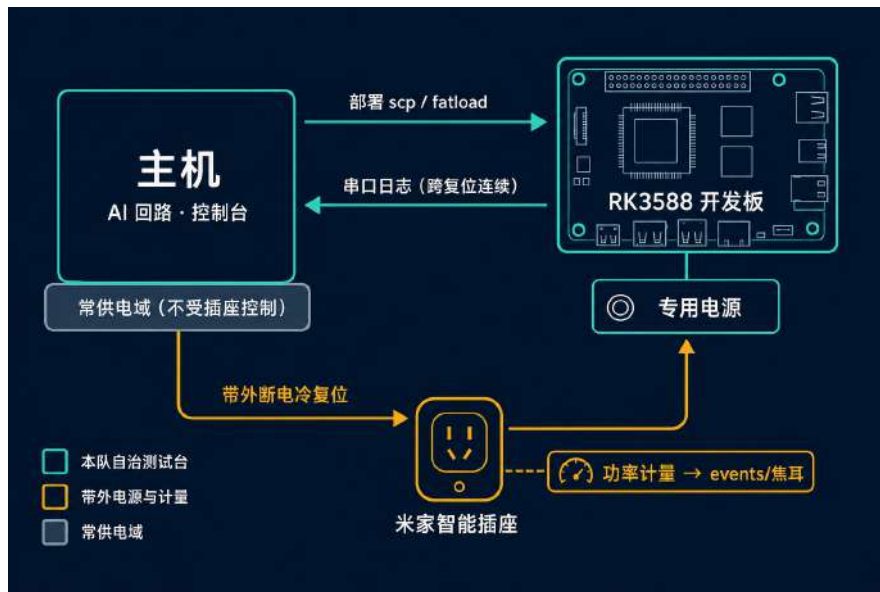


图 27: 带外电源控制的真机自治测试台：主机与串口适配器接常供电，板卡专用电源经计量式智能插座切换；暖重启为常规快路径，智能插座硬断电冷启为与被测软件状态无关的恢复原语

的原语是电源。为此在板卡供电与市电之间接入一只可编程的计量式智能插座（米家智能插座），主机侧的回路经其 miIO 本地协议在任意状态下对板卡冷复位，全程不经云端。恢复据此分四级逐级退化：L0 暖重启，被测系统存活，秒级，承担常规一轮；L1 智能插座硬断电冷启，被测系统挂死而主机存活，与被测软件状态无关，可从任何挂死恢复；L2 经 U-Boot 启动计数回退到金身映像，用于部署的内核连冷启都过不去时；L3 上报人工，仅当金身映像冷启仍失败，此类情形属真实硬件故障，是这条回路唯一需要人的环节。

图 27 给出这套测试台的拓扑，它令暖重启的两处顾虑一并消解。插座只切板卡专用电源一路，主机与 USB 串口适配器另接常供电。断板电因此不会令主机侧串口适配器重新枚举，控制台日志在复位间保持连续，即便某次改动在 SSH 建立之前就挂死，回路仍能读到确切的挂死点；专用且足额的电源为 8 核冷启提供浪涌余量。硬断电只在少数挂死时触发，常规仍走暖重启，反复上下电既慢又损板的代价由此不再构成约束。

四类判定门中凡标注为板上的，都运行在这条测试台上，QEMU 先行验证逻辑，真机负责给出最终数值。

18.4 测量驱动的对抗式验证否定了若干设想

判定门给出的数值既用于放行改动，也用于否定基于推断的设想，后者同样重要。DDR 带宽差距最初被推测为 THP 或 tick 记账所致，经板上 schbench 与 sysbench 测量，两个假设都被数据否定，真正的根因是 DDR 总线固定于上电频率、变频由 Rockchip 的 SIP DRAM 接口负责而主线固件缺少这一 EL3 处理程序，属固件层面的外部因素。fork 在约 250 个进程处出现 EFAULT，曾被判断为缺页处理的问题，板上复现后定位到 COW 引用计数 u8 溢出，将 u8 改为 u32 即解决。自适应 poll-idle 一度被寄望于胜过固定 50 微秒空转，板上测量表明其并无优势，因而未予采纳。每一个被否定的设想背后都对应一次真机测量。

18.5 团队掌控架构与验收

这套方法的边界清晰：AI 负责在闭环内把实现迭代至通过判定门，架构决策与最终验收始终由团队负责。划定原子改动的边界、确定每道判定门的阈值、判定一个数值是否足以放行，均为团队做出的判断。团队成员能在无 AI 在场时口头阐明每一处关键设计的动机，并当场修改对应的内核代码：PVTPLL 环长切换为何能在同一电压下换取更多主频、break-before-make 为何需先执行 dsb isbst 再执行 tlbi、s_frac 为何可作正确性 oracle，均非黑箱。收尾阶段一轮投机实现排查覆盖了整套出厂改动，未查出投机取巧的实现，剩余唯一尚未对齐的一项是 DDR 变频，受主线固件限制，驱动已实现。

19 队员分工

本队共三名成员，按子系统分工，各自负责所辖模块的设计、实现、板级测量与复现。洪世金（Joseph Joshua Anggita）为队长，承担内核性能、内存与驱动的主体工作：性能观测 perf、调度器与大小核、频率与电压 DVFS、内存与 THP、页表与 TLB 正确性、RGA 与 JPU 驱动、原生显示栈、sysbench 基准套件与整机对比，并负责本报告的撰写与整体协调。王乙凡承担功能级仿真，将 RK3588 的 RKNPU 实现为可执行的 QEMU 功能级模型。徐子航承担板级使能，补齐板上运行真实负载所需的 I/O、网络与相机驱动，实现 reboot 系统调用。机器人拾球应用由王乙凡与徐子航共同完成。表 16 按成员列出各自负责的模块与交付物，各贡献的 PR 编号与合入状态见增量贡献一节。

三名成员的工作在若干处交汇。板级测量依赖徐子航打通的 UART、SSH 与相机链路方能取得读数；洪世金的调度、DVFS 与内存结论为机器人应用提供算力预算与页表基础；机器人模型的推理性能在王乙凡的 RKNPU 模型上测得；机器人应用由王乙凡与徐子航在板级链路上共同实现。分工按模块划分，验证由全队共用一套板级与仿真环境完成。

成员	负责模块与交付物
洪世金（队长）	perf 观测单核 ARM PMUv3 硬件 PMU perf，多核与 big.LITTLE，软件事件与 HW_CACHE，真实 tid 采样归属与 LOST，事件组、SAMPLE_READ 与 -a，调用图，ftrace（板级 39/0、矩阵 56/0）。调度器与大小核跨核互唤醒死锁修复、新建与唤醒线程的跨核分发、占用感知的派生与唤醒放置、CFS 对齐的 wake_affine（跨核唤醒 1042 至 8 μs）、CLONE_VM 用户拷贝快路径（hackbench -T 10.2×）。 DVFS 与 cpufreq RK3588 ondemand CPU DVFS 加 PMIC 电压标定，PVTPLL 环长切换与 OPP 顶推（A76 2126 MHz、A55 1881 MHz）。内存与 THP 2MB 私有匿名 first-touch（0.030 s，快 2.87×）、新建映射跳过 TLB flush、MADV_NOHUGEPAGE 语义、COW 引用计数 u8 至 u32。页表与 TLB 正确性共享跨内核 crate 上的 not-present huge block 误判 UAF 类修复、dsb ishst 存序、vmalle1is 广播回退、BBM 无分配拆分、空中间表回收与批量失效合并。 RGA 与 JPU 驱动 RGA2 二维加速器、JPU VDPU720 硬件 JPEG 解码、rknpu DRM ioctl 归并。原生显示栈 VOP2、HDMI-QP、HDPTX PHY、CRU 扩展与 Qt6 冷启动编排。sysbench 与整机对比、报告撰写与整体协调。
王乙凡	QEMU RKNPU 功能级模拟器 PC/CNA/CORE/DPU/RDMA/PPU 全流水线及 IOMMU 与 INT8/FP 执行（106 qtests），多核协同执行，每次推理的 profiling 计数。机器人拾球应用（与徐子航共同）。
徐子航	板级 I/O 与网络驱动 RK3588 UART 初始化、USB-serial tty、TTY 输入 flush 与唤醒、PWM sysfs 执行器面、rtnetlink IPv4 与 RTL8125 多网卡及 regulator GPIO 共同打通板级 SSH、ax-net 数据报睡眠锁修复、RKNPU GEM 遵循 cache 标志的 mmap。 UVC 相机 USB 相机异步传输生命周期与测试迁移到 ax-sync。 reboot reboot(2) 系统调用，支撑板级热重启循环。机器人拾球应用（与王乙凡共同）追球到投放的五阶段状态机、执行器后端、底盘电机编码器里程计定位回桶。

表 16: 三名成员按模块与交付物的分工，各贡献的 PR 编号与合入状态见增量贡献一节

页表与内存的这些修复落在多套 OS 共享的跨内核 crate 上，风险面较大，每处改动在合并前均经过对抗式代理复核，每处由 7 至 26 名评审代理压测。页表 UAF 类误判、dsb ishst 存序、vmalle1is 广播回退、cpufreq 高频欠压这些真实缺陷均在复核中被定位并修复，故将其独立列为一组交付物。

QEMU RKNPU 模型已合并且功能完整，机器人模型的推理数据均在其上测得。原生显示栈的四个 driver-core crate 主机寄存器级验证通过，并已在 RK3588 上点亮验证通过（ROPLL 锁定、VP0 1080p60 光栅、/dev/fb0 彩条、Qt6 时钟原生运行）。

20 开发过程与时间线

这一个月的工作自 2026 年 7 月初持续至 8 月中旬。此前在应用层于单线程侧追平 Linux 的工作已经完成，这一个月目标是使 StarryOS 在 RK3588 上成为通用的对等系统。工作分两条线推进。系统内核侧负责 perf、调度器、DVFS 与 cpufreq、RGA/JPU 驱动、sysbench 与原生显示栈；仿真与板级侧负责 QEMU RKNPU 模拟、相机 UVC 与 UART/USB-serial/PWM 驱动、rtnetlink 转 SSH、RKNPU GEM 缓存与 reboot 系统调用。硬件平台为 OrangePi-5-Plus，cpu0

至 cpu3 为 A55 小核，cpu4 至 cpu7 为 A76 大核，配备三核 NPU、硬件 RGA/JPU 与 HDMI 输出。整体进度按周推进，各项改动均以开发板实测为准；本章仅列出时间线，各项工作的现象、根因与验证在对应章节展开。

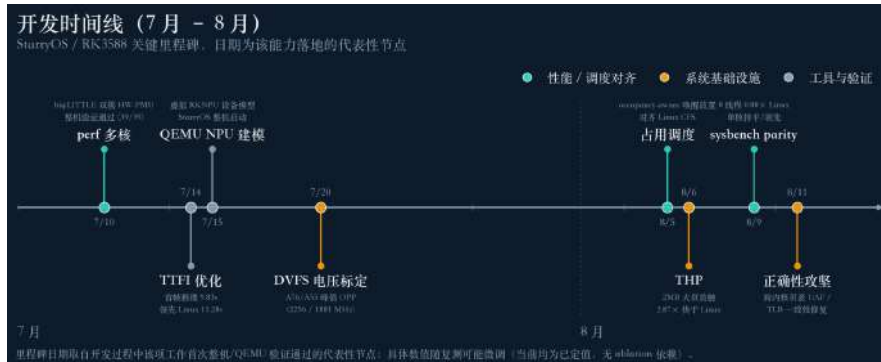


图 28: 约一个月的按周里程碑，自 perf 多核历经调度器、DVFS、THP，至 sysbench 对等与 TTFI 领先

第一周（约 07-01 至 07-14）以将硬件 PMU 的 perf 推进至多核为起点。单核 A55 首先通过 18 项检查，判定为 PARTIAL；随后 smp8 big.LITTLE 全量 39 项通过、0 项失败，判定为 FULL。开发板压力测试暴露出 axtask 跨核相互唤醒的死锁，改为延迟跨核唤醒，移除对远端 on_cpu 的自旋等待，该修复并入 PR #1495 已合并。多核与大小核的 PMU 支持落于 PR #1577，依据 MIDR 区分 a55/a76 两类 cluster 身份，经 IPI 逐核扇出；在此基础上直接运行未经改动的上游 perf 6.x 与 PERF_SAMPLE_CALLCHAIN 火焰图。同周仿真与板级侧完成的 rnetlink IPv4 与多网卡支持打通开发板 SSH 登录（PR #1497 已合并），调试自此摆脱串口控制台的约束，显著提升了整段工作的效率。

第二周（约 07-11 至 07-16）清空 perf 待办事项。软件事件、HW_CACHE、真实 tid、PERF_RECORD_LOST、事件分组与 PERF_SAMPLE_READ 分列于 PR #1601/#1602/#1603，DWARF 回溯与 kprobe/tracepoint 在 QEMU 上验证通过。TTFI 由提交版本的 19.22 s 降至 9.83s（同机 Linux 为 11.28s），并经开发板确认。仿真与板级侧完成的 QEMU RK3588 RKNPU 功能模拟合并（processmission/qemu #19），附带 106 项 qtest，提供板外的 RKNN 开发与模型优化基准。原生显示栈 VOP2、HDMI-QP 与 HDPTX 同周启动，主机测试通过，板级点亮已完成（ROPLL 锁定）。

第三周（约 07-14 至 07-22）建立 sysbench 度量与基线分解，将 StarryOS 落后 Linux 的约 24 至 33 倍差距拆解为 DVFS、A55 至 A76、SMP 分布三个可攻克的因子。RK3588 cpufreq 的 ondemand 治理与 PMIC 电压标定并入 PR #1657 已合并，axtask 在 spawn 与 wake 处实现 SMP 分布并入 PR #1656 在审，cpu 八线程吞吐由约 160 提升至 3668。前向移植的工作窃取负载均衡器在开发板 A/B 测试中呈无收益，因空闲拉取抖动而回退，收益集中于放置一侧，运行期迁移引入额外干扰。

第四周（约 07-16 至 07-31）收尾 perf CLI 与 ftrace，函数追踪器记录 11512 条，开发板 perf 矩阵 55 至 56 项通过、0 项失败。PVTPLL 环长与电压配合分四步上调 OPP 上限，per-core 由 Linux 的 0.75 倍提升至 0.98 倍，A76 上限达 2126MHz，A55 上限达 1881MHz。同期为 first-touch 缺页路径省略 TLB 刷新，为后续 THP 首触作准备。两条线合作在 QEMU RKNPU 模拟器上对网球模型进行结构协同优化，MAC 由 3.15 降至 2.06 GMac，rknn_run 由 14.66 降至 5.68 ms，mAP50 由 92.9 保持在 91.1%。

第五周(约 07-28 至 08-05)落地占用感知调度器, fork 与 wake 对齐 Linux CFS, 占用改为单一原子量, 按 tick 在锁下经 nr_running 自愈, 唤醒采用 select_idle_sibling 风格。/proc/stat 补充每核 BUSY_TICKS 的真实计账, 修正多线程聚簇导致的误读。THP 的 2 MB 私有匿名首触将内存 first-touch 由约 0.26 降至 0.030s, 相较同机 Linux 的 0.086s 领先 2.87 倍。DDR/DMC 变频驱动已实现, 但受固件所限, 板上运行的主线 TF-A 缺少 SIP DRAM 接口; 多线程写带宽经放置铺开回升至 0.61 倍, 单流 memcpy 仍约 0.36 倍。仿真与板级侧的板级 I/O 驱动, 包括 UART、USB-serial tty、PWM sysfs、reboot 系统调用与 UVC 相机生命周期, 本周批量合并。

第六周(约 08-05 至 08-10)攻克唤醒延迟与系统调用开销。wake_affine 配合 GIC-SGI-WFI 根因分析与 poll-idle 将唤醒延迟压至 $8\mu\text{s}$ (同机 Linux 为 6), 接近对等。CLONE_VM 的 access_ok 用户拷贝快路径在线程路径上开发板实测达 10.2 倍。系统调用入口成本沿 ptrace、poll_process_timer、tickacct 与无锁 timer_state 逐层削减, getpid 由 1200 降至 431ns, 相较 Linux 由约 7 倍收窄至 2.6 倍。sysbench 的最终计分卡落于 PR #1658, cpu 八线程 4987 对 Linux 5222, 比值 0.96 判定为持平。

第七周(约 08-09 至 08-11)进行独立 PR 拆分与内核正确性专项工作。SD 卡本地加载的开发板 harness 将单次刷入由串口 ymodem 的约 90 s 缩短至约 1 s, profiling 循环因此加速。多架构页表中的一类 UAF 得到定位, 未建立映射的大块被误判为页表而释放了一个 COW 数据帧, 经 GenericPTE::is_table() 修正; 同期修复 aarch64 缺失 dsb ishst 与本地 vmalle1 误用非广播两处潜伏的 SMP 顺序 bug。cpufreq 电压安全经 26 组对抗式审计完成 fail-closed 加固。约 31 项改动已实现并按同一原则拆分为独立 PR 提交, 正确性细节见对应章节。

一个月内落地并验证的工作包括硬件 PMU perf 从单核到多核再到大小核、QEMU RKNPU 功能模拟、axtask SMP 分布、CPU DVFS 与 PVTPLL 电压调节、占用感知的 CFS 对等调度器、THP 首触、CLONE_VM 快路径、系统调用成本下降与 TTFI 领先。原生显示栈主机测试通过, 板级点亮已在上板会话中通过 (ROPLL 锁定)。另有三项经板级验证在当前平台未见收益、已回退, 各有其因: 工作窃取负载均衡器因迁移抖动回退, early-MMU 启动重排在低频启动窗口无增益, DDR/DMC 变频受主线固件限制; 后者是多线程写带宽这一项差距的根因。

21 可复现性与后续工作

本报告的板上数字全部出自同一块 OrangePi-5-Plus (RK3588, cpu0 至 cpu3 为 A55 小核、cpu4 至 cpu7 为 A76 大核, 三核 NPU, 硬件 RGA/JPU, HDMI 输出)。本节记录这些数字的复现方法与后续工作的推进方向。

21.1 硬件接线与板上运行

该板具备两条访问通道。其一为 USB 转串口的调试控制台, 工作于 1.5 Mbaud。其二为经 RTL8125 网口的 SSH 登录 (#1497 合入后可用)。两条通道均需保留, 一旦 StarryOS 的用户 shell 占用串口 tty, 内核的 warn/info 便不再到达控制台, 而 /dev/kmsg 仅可写入, 此时只有 SSH 可观测运行态。

上板迭代须无人值守。常规一轮走暖复位回路: 从 StarryOS shell 执行 reboot -f 返回 Linux (依赖 #1358 的 reboot 系统调用), 在 Linux 侧执行 ssh reboot 转入 U-Boot, 随后 fatload 预置的 FIT 并 bootm, 全程无需断电。改动挂死到 shell、SSH 与 U-Boot 均不可达时, 板卡专

用电源经一只计量式智能插座带外硬断电冷复位，恢复与被测软件状态无关；插座只切板卡一路，主机与 USB 串口适配器另接常供电，故断板电不致令串口适配器重新枚举、控制台日志跨复位连续，专用足额电源亦为 8 核冷启提供浪涌余量。整套测试台的构成与分级恢复见 AI 方法一节图 27。

回路由 `scripts/board/boardctl.py` 驱动。其依据串口占用情况自动选择传输方式，串口服务在设备被占用时经由本地 WebSocket 桥接，否则直接连接 `pyserial`。`deploy` 执行 `scp` 并强制一次 `sync`，随后校验 `PT_INTERP` 加载器，此步修复了此前部署后启动挂死数百秒的故障。控制台数据流附带主机时间戳，并接入实时的失败监视器。这套流程直接压缩了烧写路径的耗时，三条刷写通路的实测用时见板级 I/O 一节表 14。

内核与测试镜像由 `cargo xtask starry` 产出 `rootfs` 与板上 FIT，`perf` 与 `sysbench` 在板上运行的是上游同名二进制，成功判定依据各自的 `success_regex`。全部性能数字集中于 `data/numbers.tex` 这一份权威表，图表由各自脚本从 `figures/data/FNN.json` 渲染，正文仅引用宏名，不写入字面量。

21.2 后续方向

调度侧的下一步是实现全量负载均衡。当前上板运行的是安全的纯放置子集（容量感知的产生与唤醒放置，不含迁移），其原因在于完整的空闲拉取、推送与唤醒改向在 RK3588 上使 `sysbench` 单线程降至 0.46 倍，根因为迁移抖动。实现路径是为迁移引入代价门限与迟滞，使其仅在收益为正时跨核迁移任务。

DVFS 的 Phase 2 是电压闭环。A55 轨以带上下限的开环电压写入下发，仅保证不欠压；Phase 2 将接入闭环电压回读与温控，方可放开全核 1.0 V 的顶档。A76 顶档亦封定于 2126 MHz @ 925 mV，因为唯一冻结过的全核零压降快照位于 1592 MHz，且板上尚无温度调节器。同一层的 DDR 变频受主线固件限制，驱动已实现，待主线补齐 DRAM SIP，或替换为将其置于 EL3 的 `rkbin BL31`，届时即可着手收敛写带宽这一项差距。

内存侧的下一步是把首次触碰 THP 强化为通用 THP。首次触碰的 THP 已将 128 MB 私有匿名内存压缩至 0.030 s，快于 Linux 的 0.086 s，领先 2.87 倍，这是一项真实的领先。后续需补足后台合并、`madvise` 提示，以及真实负载下 `split/merge` 的稳健性。

加速器侧的下一步是将 NPU 三核并发推进到真机。三核协同的正确性已由仿真与板级侧在 QEMU RKNPU 模拟器上完成多核验证，将其接入真机推理即可把吞吐按核数横向扩展。与之配套的是将这套 QEMU 设备模型上游化。RKNPU 模拟器已合入 `processmission/qemu` (#19 与 #26 多核，106 个 `qtest`，含真实 INT8/FP 执行与 IOMMU)，下一步是将其构建为主线 QEMU 中一个规整的平台总线设备，使后续开发者无需真机即可开发 NPU 驱动。